RumiCar

RumiCarハンズオン！

自動運転アルゴリズムに挑戦しよう

講師：千葉　瑠実佳
RumiCar開発部
プロジェクトオーナー

RumiCar開発部　紹介


千葉　瑠実佳


千葉　吉輝


片岡　秀明


小野寺　修


石垣　翔子


稲玉　繁樹

目次

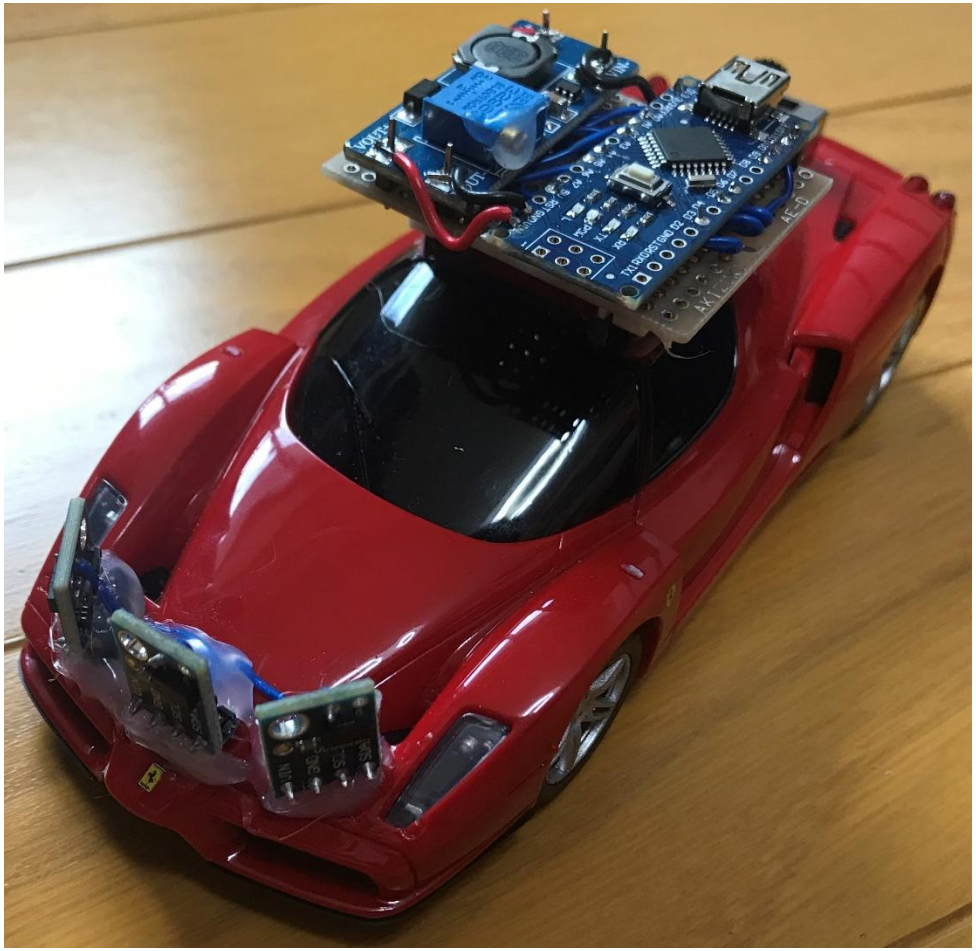
- 学べる内容
- RumiCarとは
- 準備
- 距離の測定
  - Exercise-1.1　中央のセンサで測距しよう！
  - Exercise-1.2　３つのセンサで測距だ
  - Exercise-1.3　シリアルプロッタを使ってみよう
- モータ制御
  - Exercise-2.1　ハンドルを切る
  - Exercise-2.2　速度制御
  - Exercise-2.3　前進と後進
  - Exercise-2.4　ジグザグ走行
- 自律走行基礎
  - Exercise-3.1　安全に停止する車
  - Exercise-3.2　市街地を走る車
- AIと一緒にプログラムを改良
  - Exercise-4.1　スリップしない前進と後進
  - Exercise-4.2　はまったら切り返す車

青字のところで実際にプログラミングするよ

学べる内容

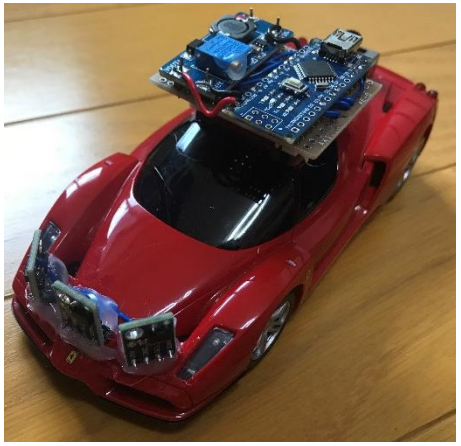
- RumiCarの仕組み
  - センサー
  - モータドライバ
- レーザー測距モジュールを使った距離測定
- モータドライバ制御
- 自律走行基礎
- AIと一緒にプログラムを改良する方法（頼み方と確かめ方）
- その他として、良いプログラム開発のための関数化の意義、なども学べます

RumiCarとは

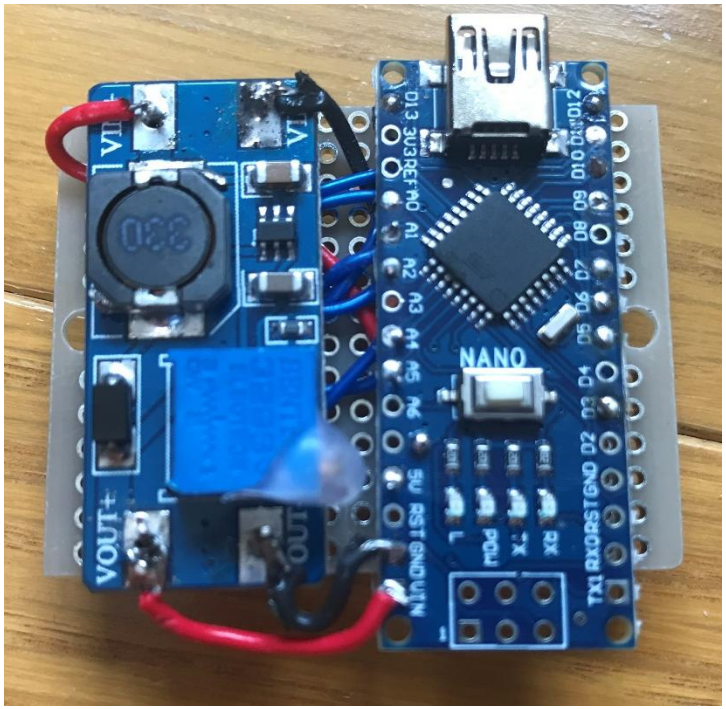


RumiCarとは

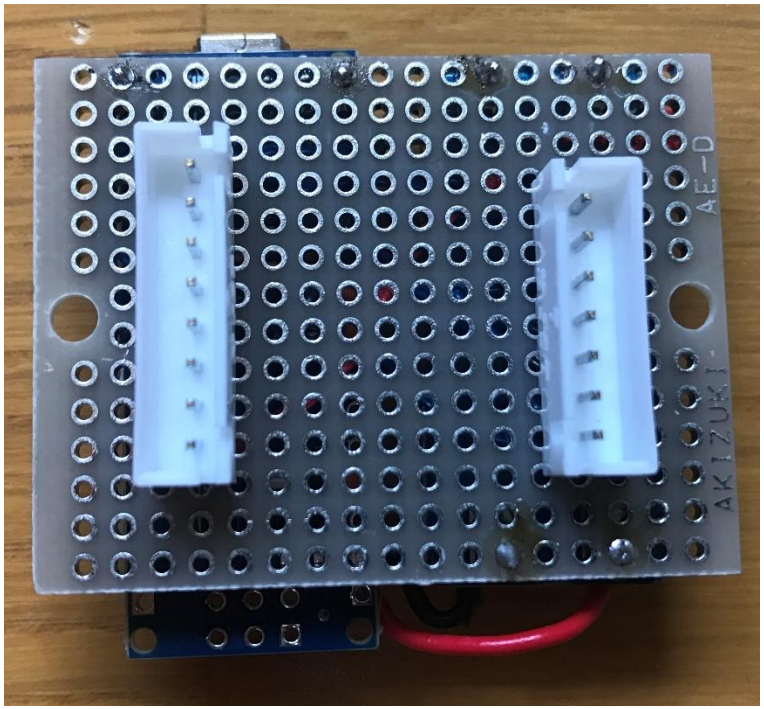
- 複数の車体と多様な種類のコンピュータモジュールを組み合わせて自律走行プログラムを開発可能なプラットフォームです
- レーザー測距モジュールを搭載していて障害物迄の距離を計測することができます
- モータの回転を制御することにより車速制御できます



コンピューターモジュール (CM)

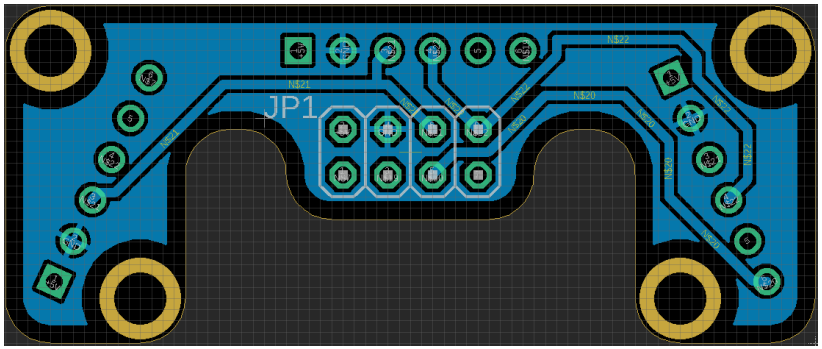


表面



裏面

コンピューターモジュール(CM)基板化

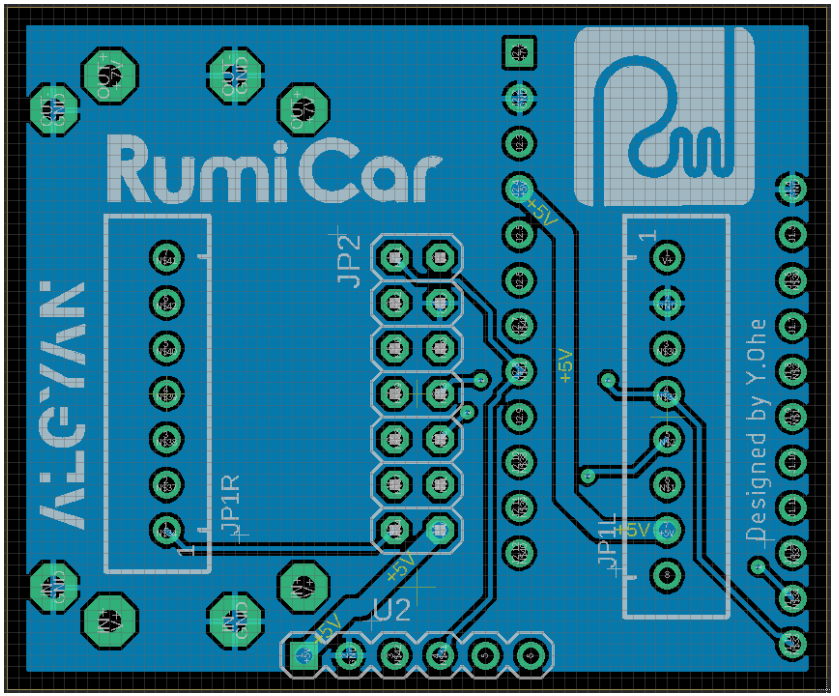


センサー基板

開発・設計してくださったのは
大栄　豊　氏



SeeedのFusion PCBに発注。
納品された。



コンピューターモジュール基板

組み合わせ自由自在

Arduino Nano



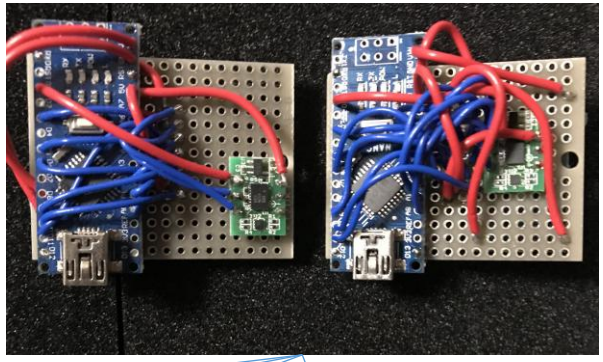
ESP32

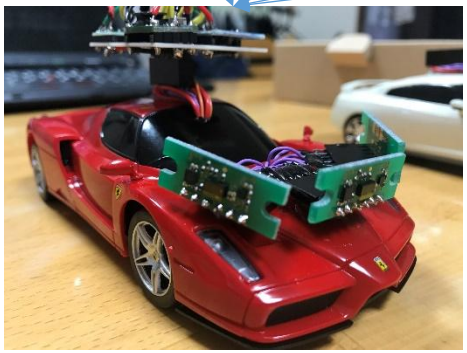


ESP32+EasyVR3



Arduino Nano+BMX055







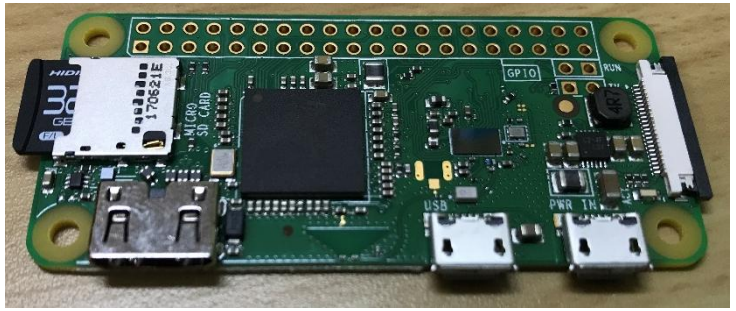




組み合わせ自由自在

次なる取り組み(挑戦)
Raspberry Pi

- **コンピューターモジュール化**
- **モーター制御**
- **画像認識**



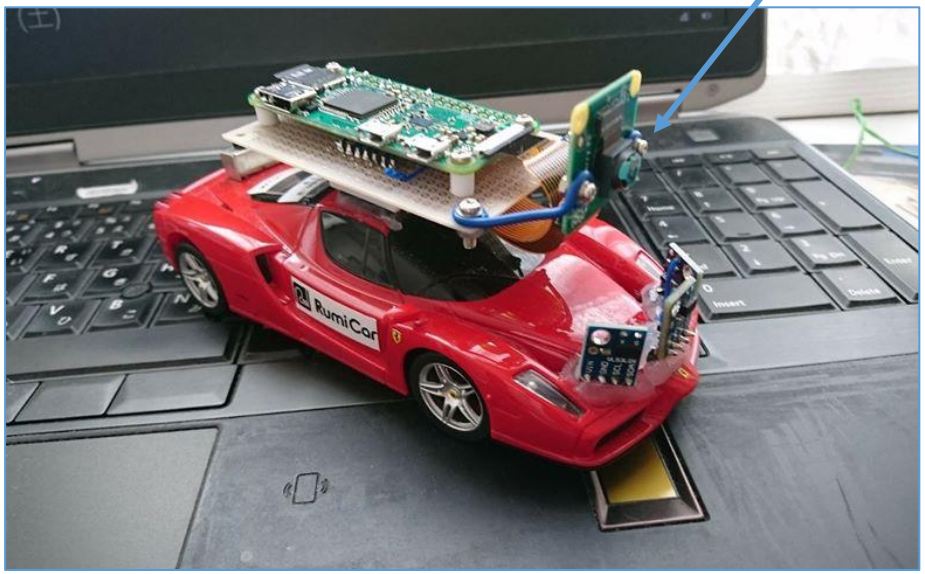


Pythonでコードを書き、
モーター制御などしてくださる
石垣　翔子　氏

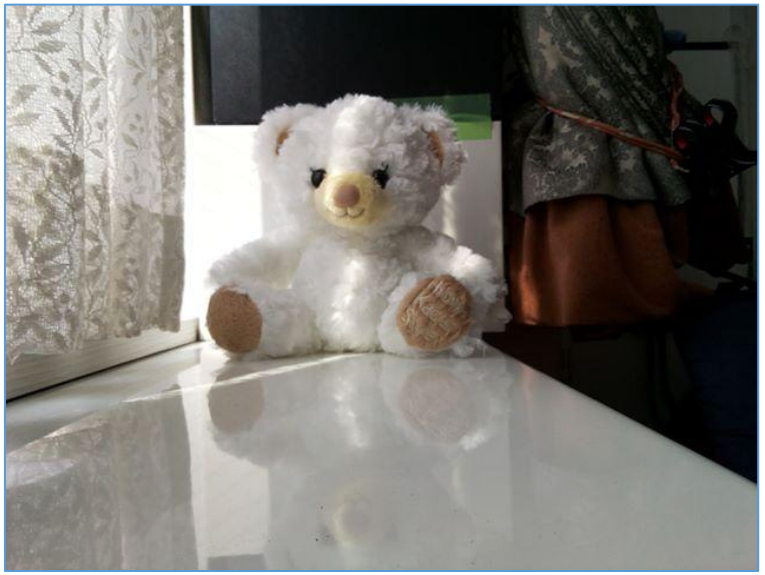
ラズパイ版CM（カメラ付き）



ラズパイ版CM（カメラ付き）



ラズパイ版CMをRumiCarに装着



ラズパイ版CMのカメラで撮影

Obniz版CM

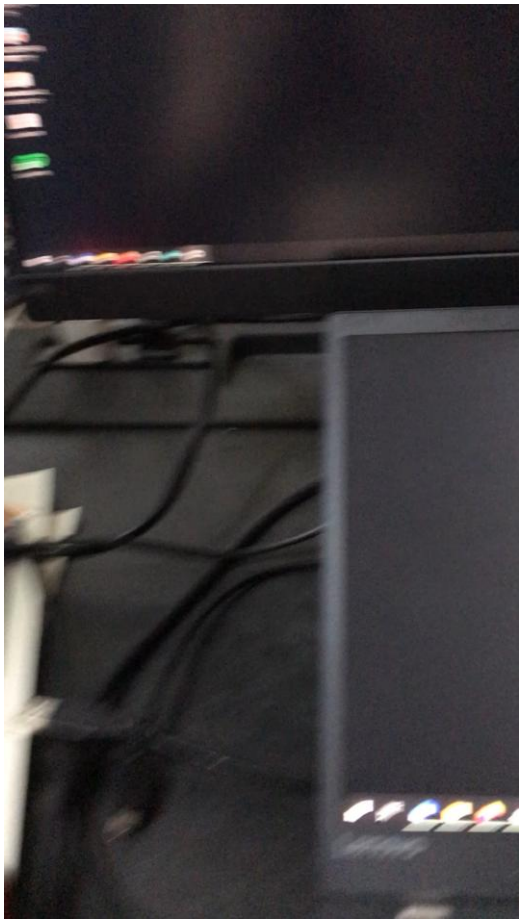


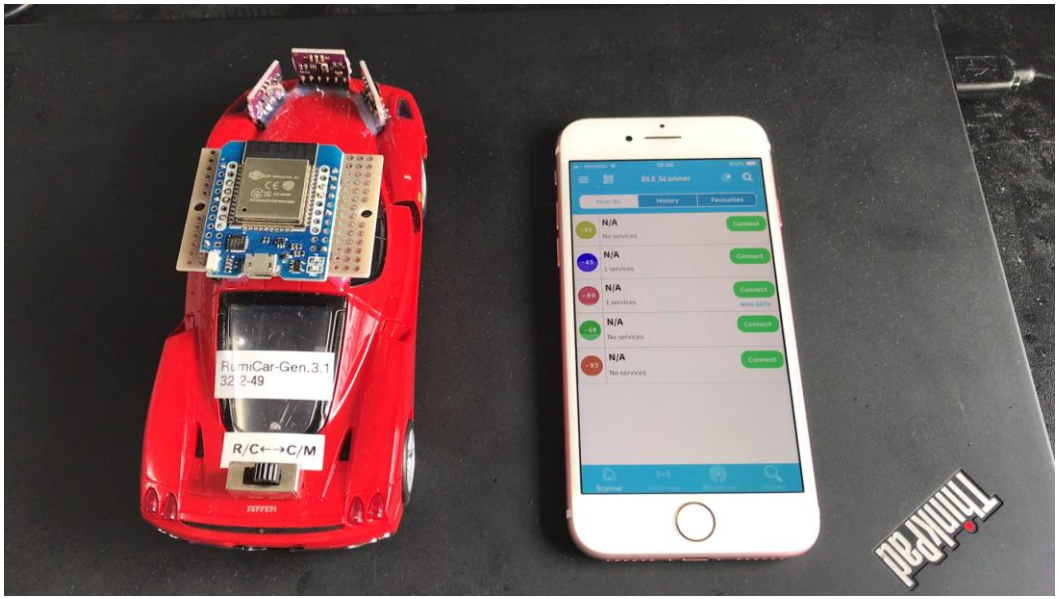
左側がObniz版、右側はラズパイ版

ESP32版 Bluetooth接続（動画）

https://youtu.be/-r6Vrhn5ifQ

https://youtu.be/UpEcTDvwYFw





BLE(Bluetooth Low Energy)マルチコネクト

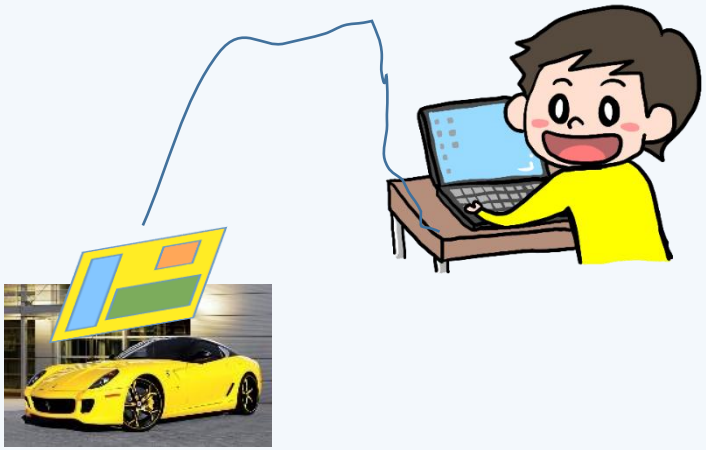
Bluetoothシリアル接続

コンピュータモジュール比較

ボード	Arduino Nano	ESP32	Raspberry Pi Zero W	Obniz	Spresense
電源	7-12V(VIN) (5Vラインへの供給は非推奨)	2.2-3.6V(3V3へ入力の場合) 3.7-6.0V(VINへ入力の場合)	5V	5V	3.6-4.4V
内部動作電圧	5V	3.3V	3.3V	3.3V	
I/Oピン電圧	5V	3.3V	3.3V	5V(電圧選択可能)	5V、3.3V切替
I2C電圧					1.8V
外部電源供給能力	ピン当たり40mA 全体で100mA	最大12mA	ピン当たり16mA 全体で50mA	5Vモードで1A	6mA
消費電力(電流)	65mA	WiFi時160-260mA	125mA WiFi時270mA	平均170mA (WiFi不使用時)	
PWMピン数	6本	4本	ハード2本	6本	
電源関係構成	電源:MT3608 VL/DRV: 5 Vラインより	PQ3RD23 (またはLM2596)	電源:MT3608 (またはTPS63020) VL/DRV:PQ3RD23 (またはSOT-23AP7333)	MT3608 (または TPS63020)	

プログラミングで楽しめるよ

自分でプログラミング
できるよ！



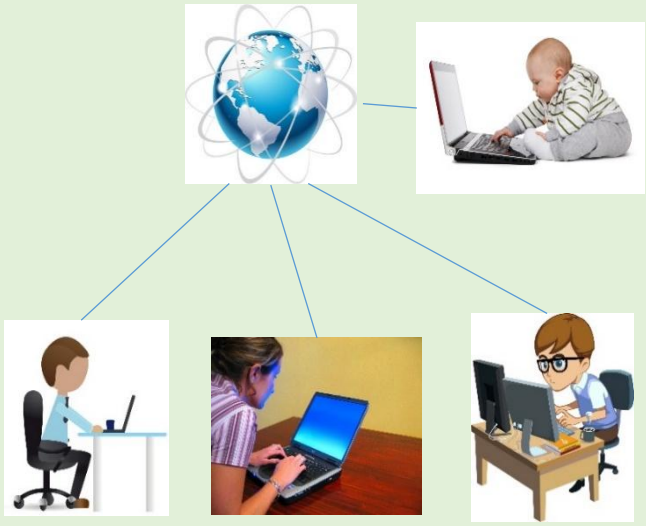
- RumiCarがあれば自分のパソコンでRumiCarへプログラムできるよ、そして自由に走らせよう
- AIに挑戦しよう
- みんながアッと驚くプログラムを作ってみよう！

世界の友達と交流しよう！

予定・計画中です

プログラムのダウンロードや交換が
できるよ

- 自分のプログラムをサーバーにアップしよう
- おもしろそうなプログラムを見つけたら自分のRumiCarにダウンロードして走らせてみよう
- 世界の友達と交流しよう



みんなで学ぼう！

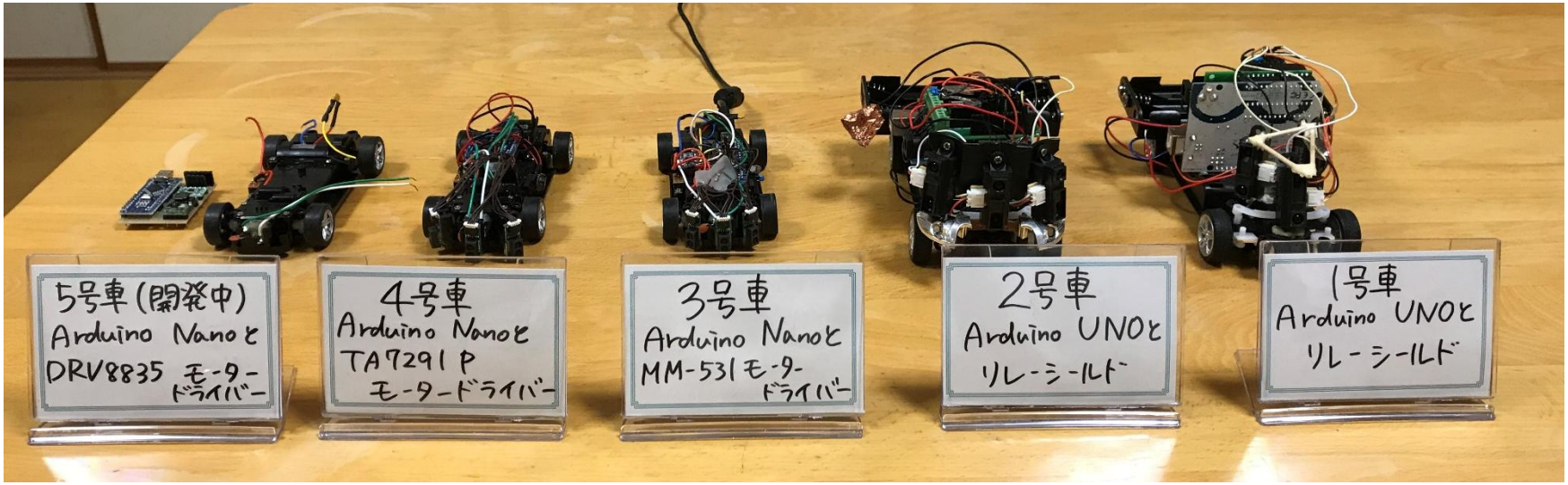
予定・計画中です

教室を開催するよ



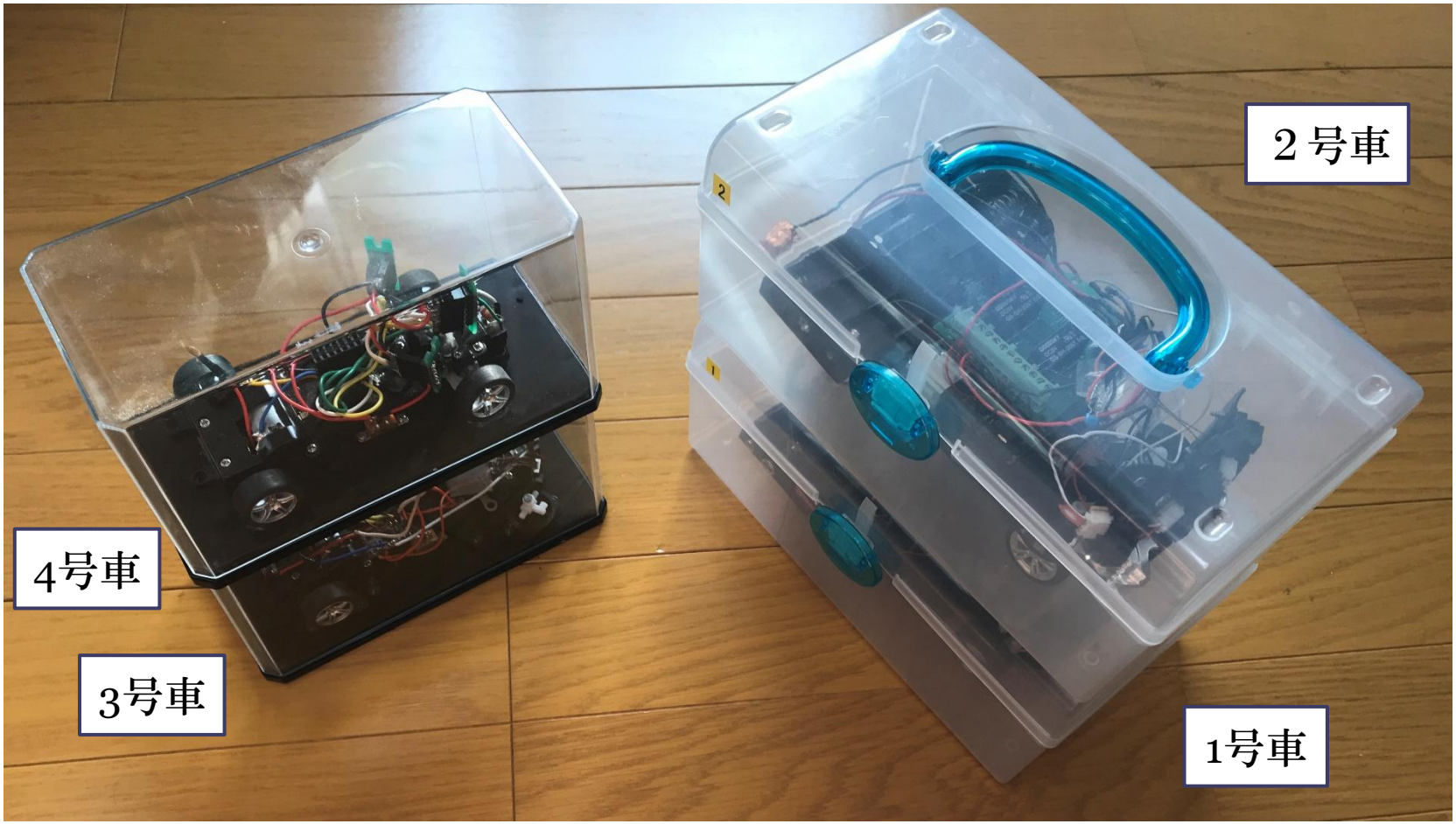
- RumiCarを作ろう！
- コンピュータモジュールを作ろう！
- プログラムを作ろう！

歴代RumiCar(1号車～5号車)

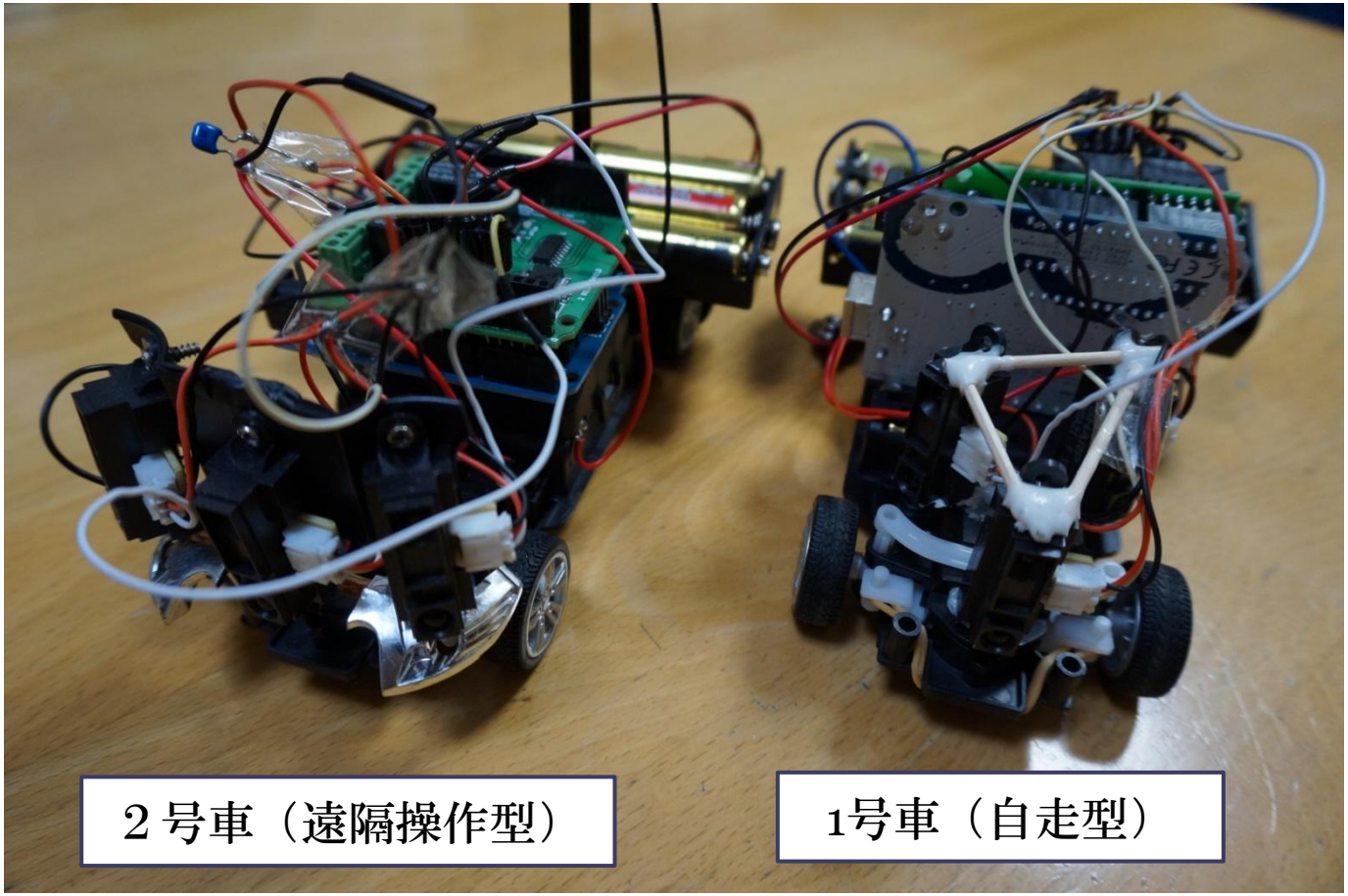


2017年8月3日撮影

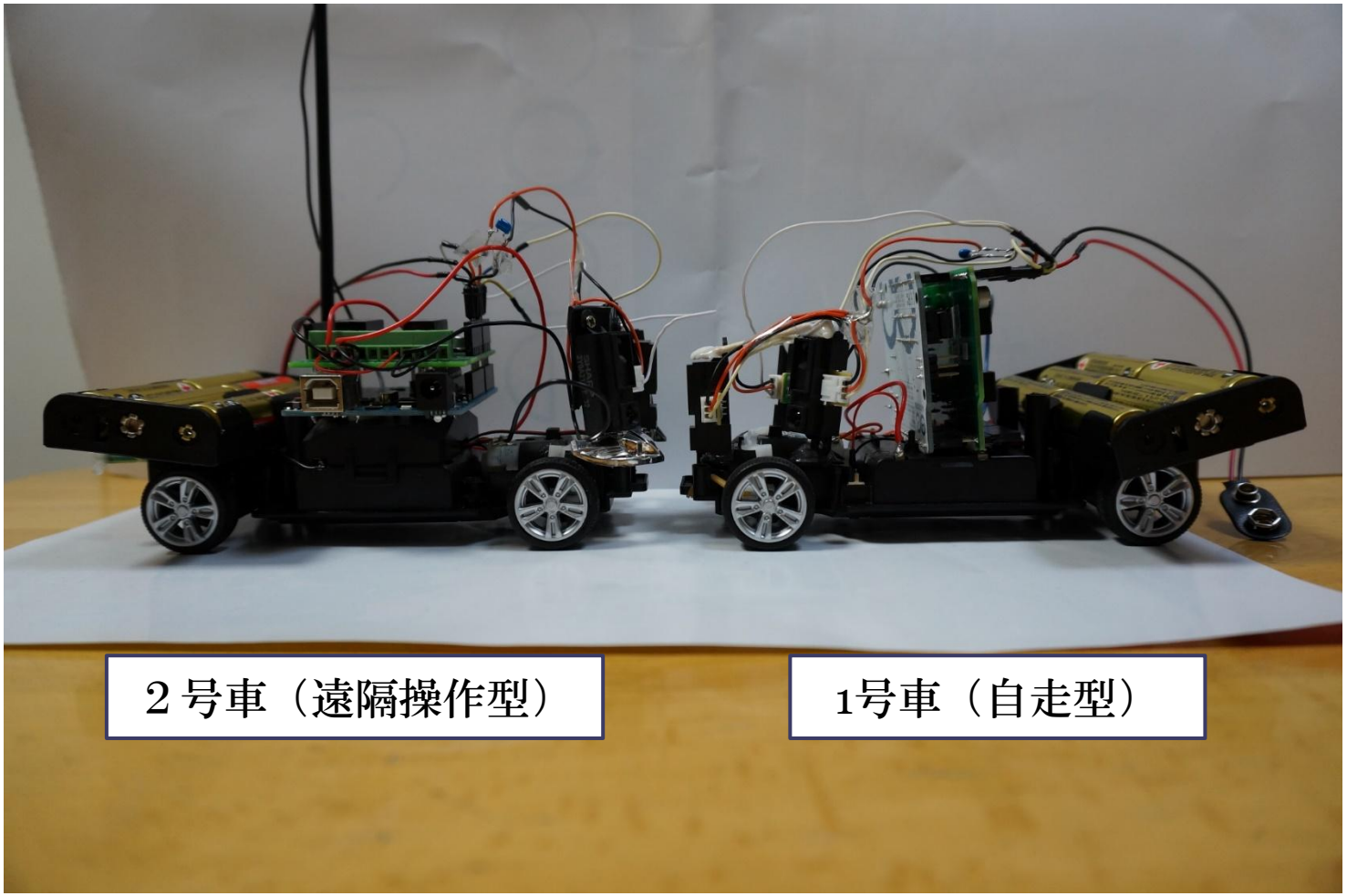
RumiCarの歴史(博物館)



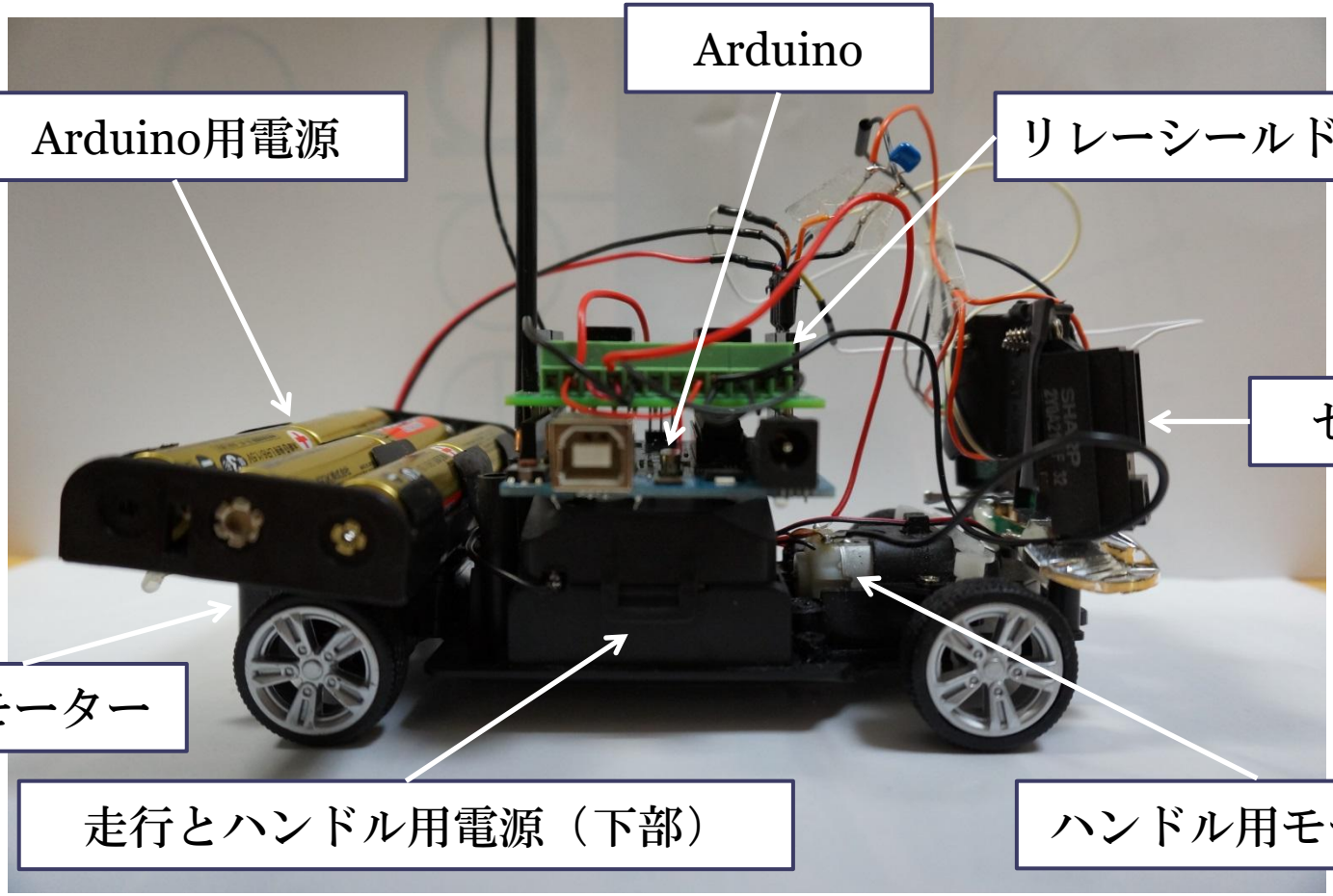
RumiCarの歴史(1号車と2号車)



RumiCarの歴史(1号車と2号車)

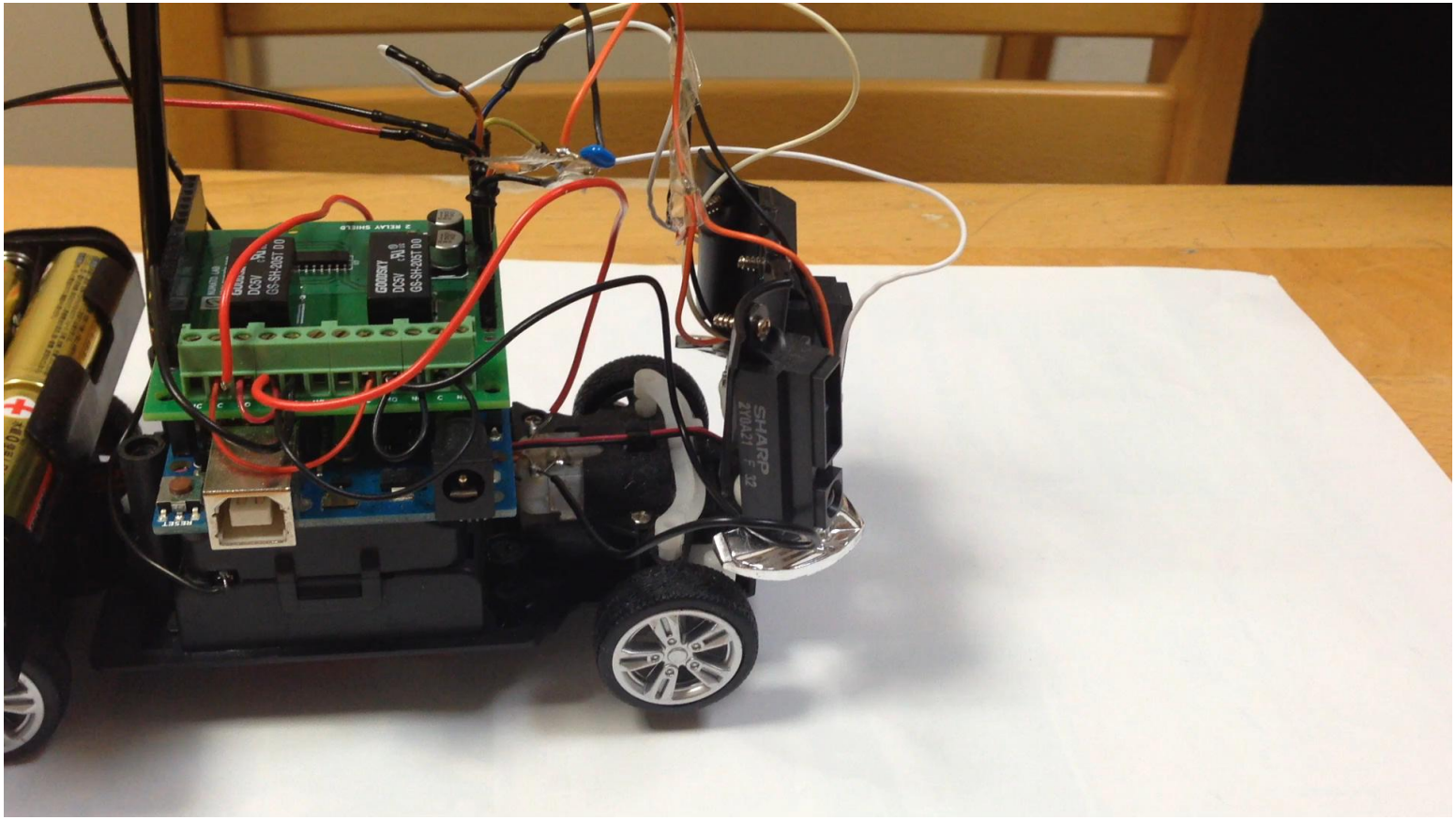


2号車の説明

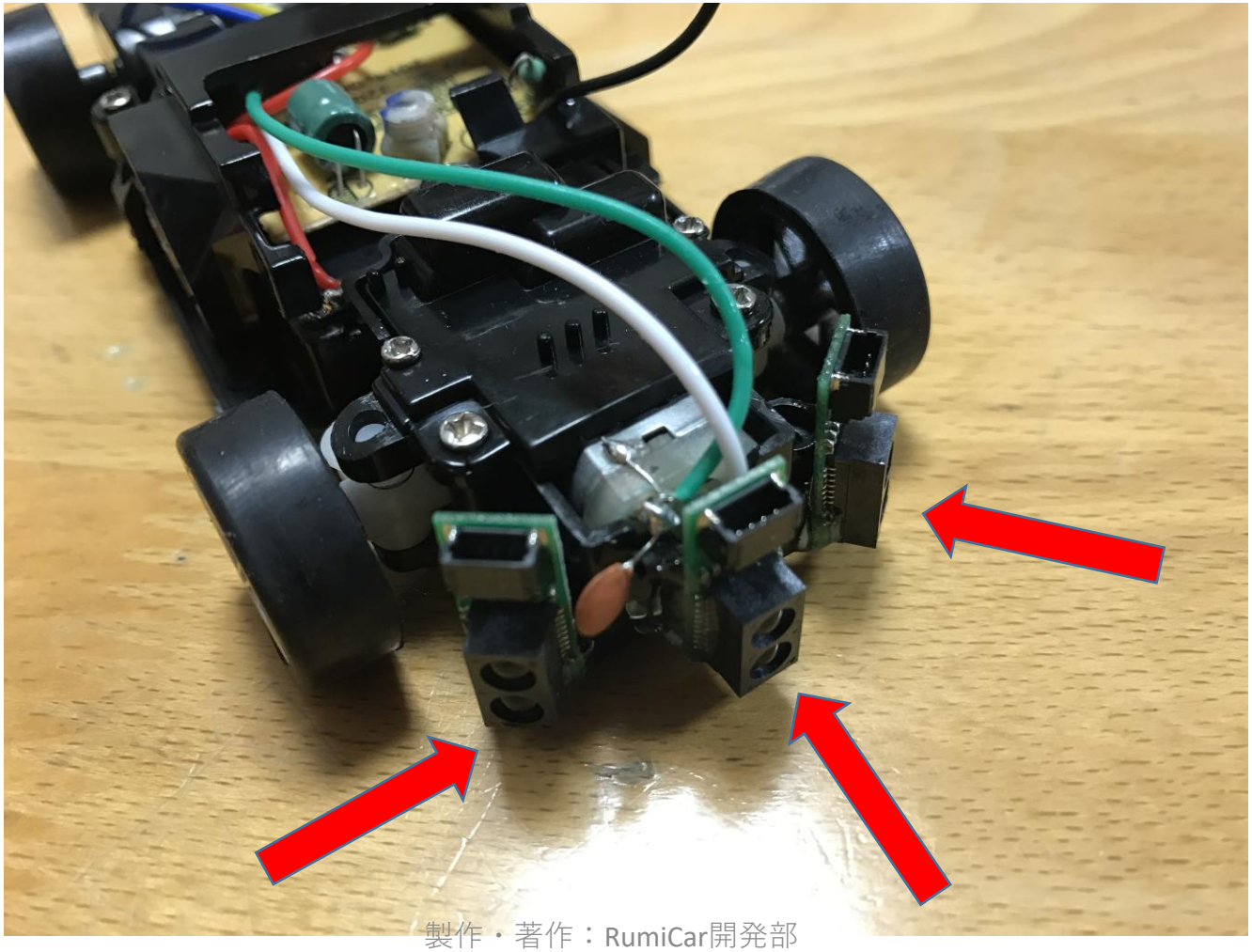

Arduino
Arduino用電源
リレーシールド
センサー
走行用モーター
走行とハンドル用電源（下部）
ハンドル用モーター

センサー動作試験（動画）

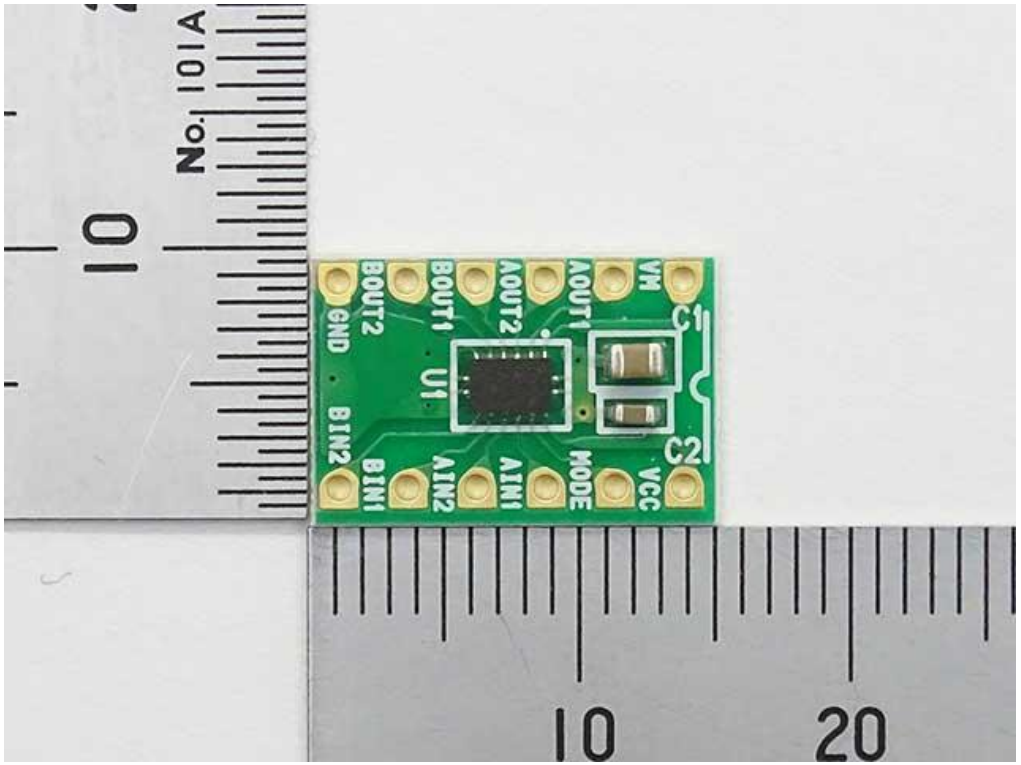
https://youtu.be/vIAL7uUnXTA



３、４号車センサー部分



モータドライバ



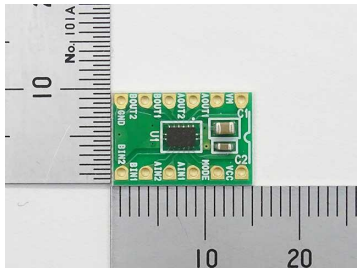
DRV8835

DRV8835 秋月電子通商より

モータドライバ

- RumiCarに搭載されているモータドライバはDRV8835です
- DRV8835の特徴
  - ２チャンネル
  - 各チャンネル1.5Aまで流せる
  - PWM(250kHz)
  - 低消費電力
  - 低電圧降下（モータ側）
  - ロジック電源（VCC）とモータ電源が独立している
    - ロジック電源電圧は2から7V
    - モータ電源電圧は11Vまで

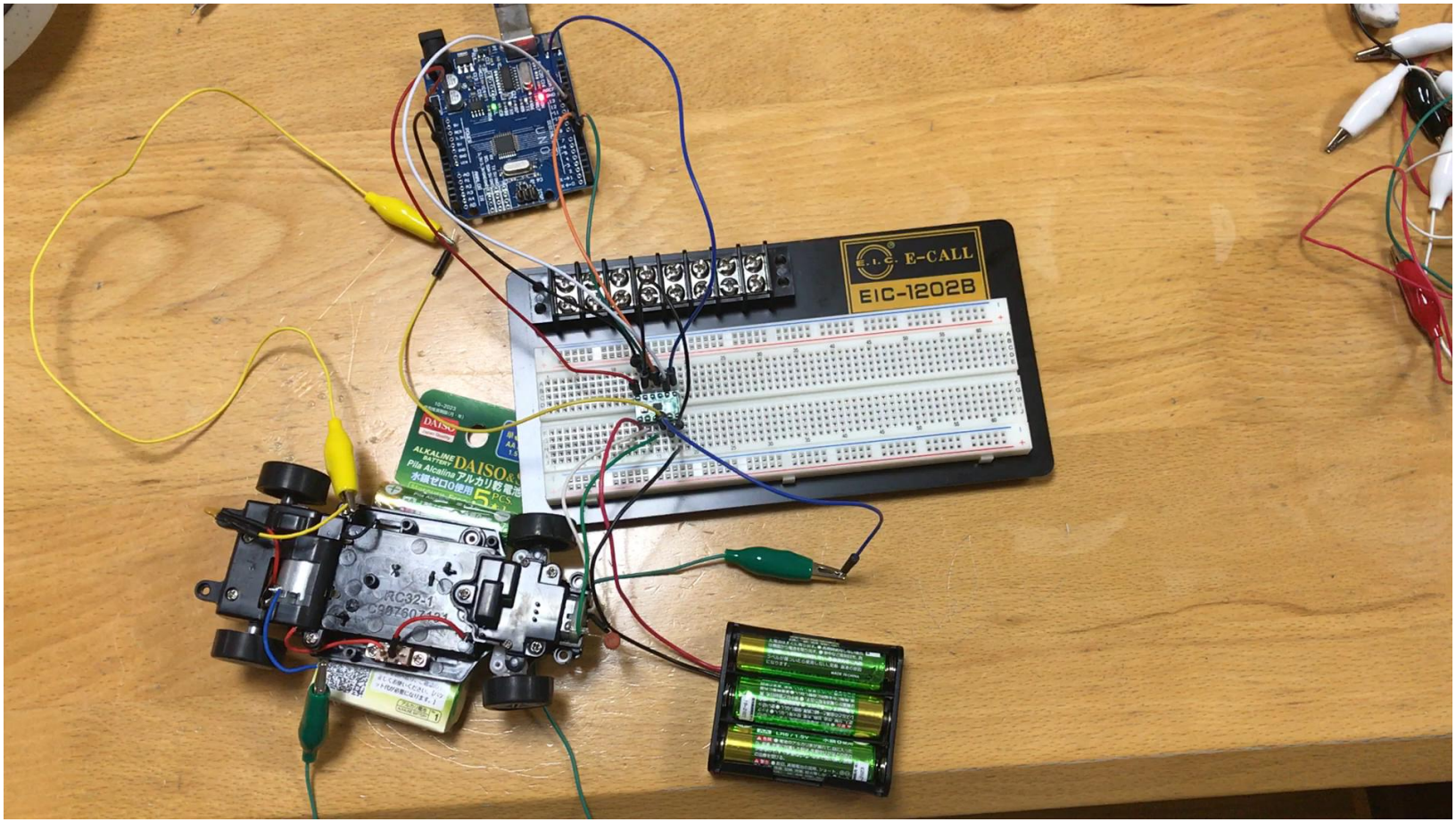
DRV8835



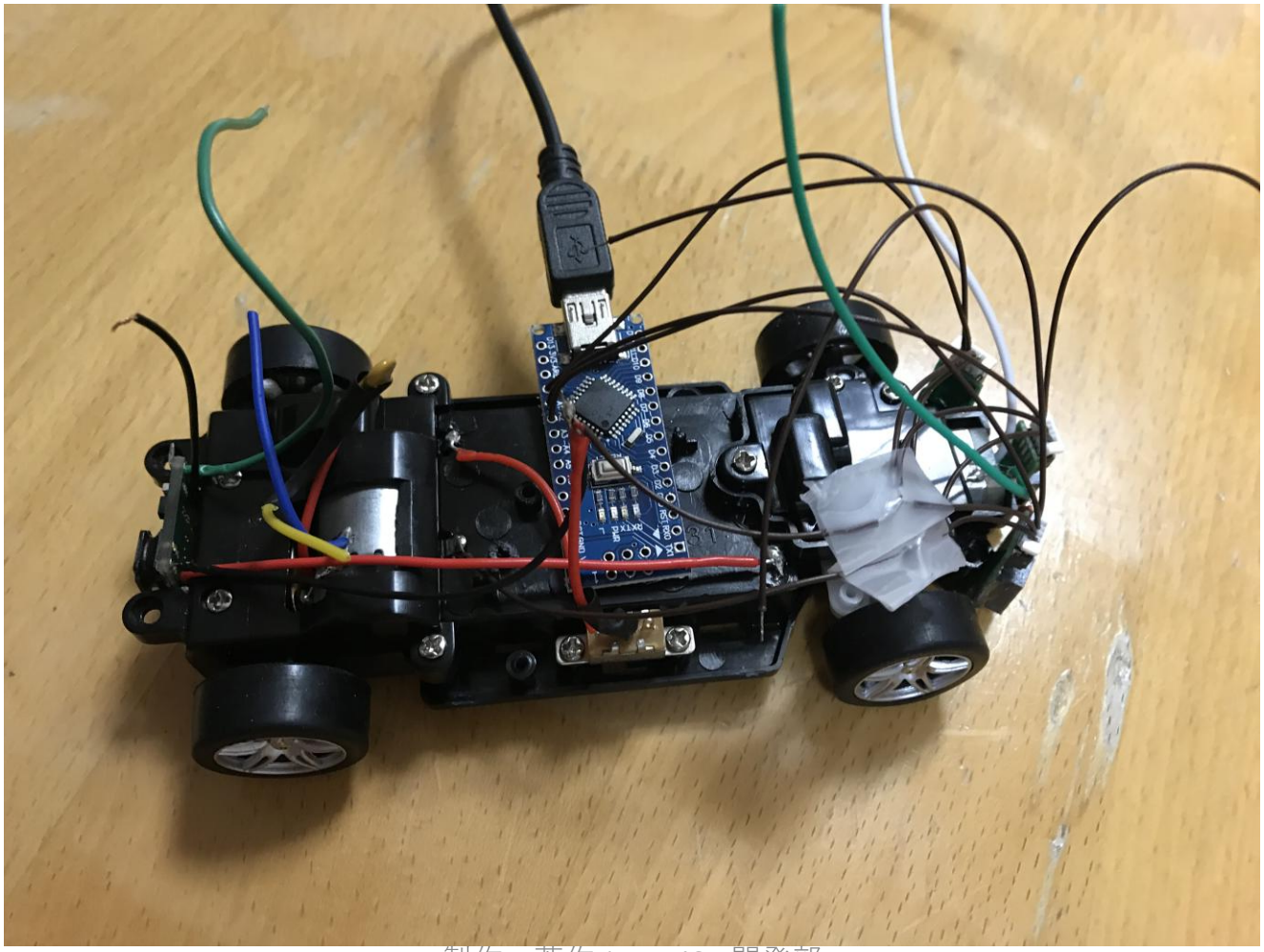
DRV8835 秋月電子通商より

開発中動画（動画）

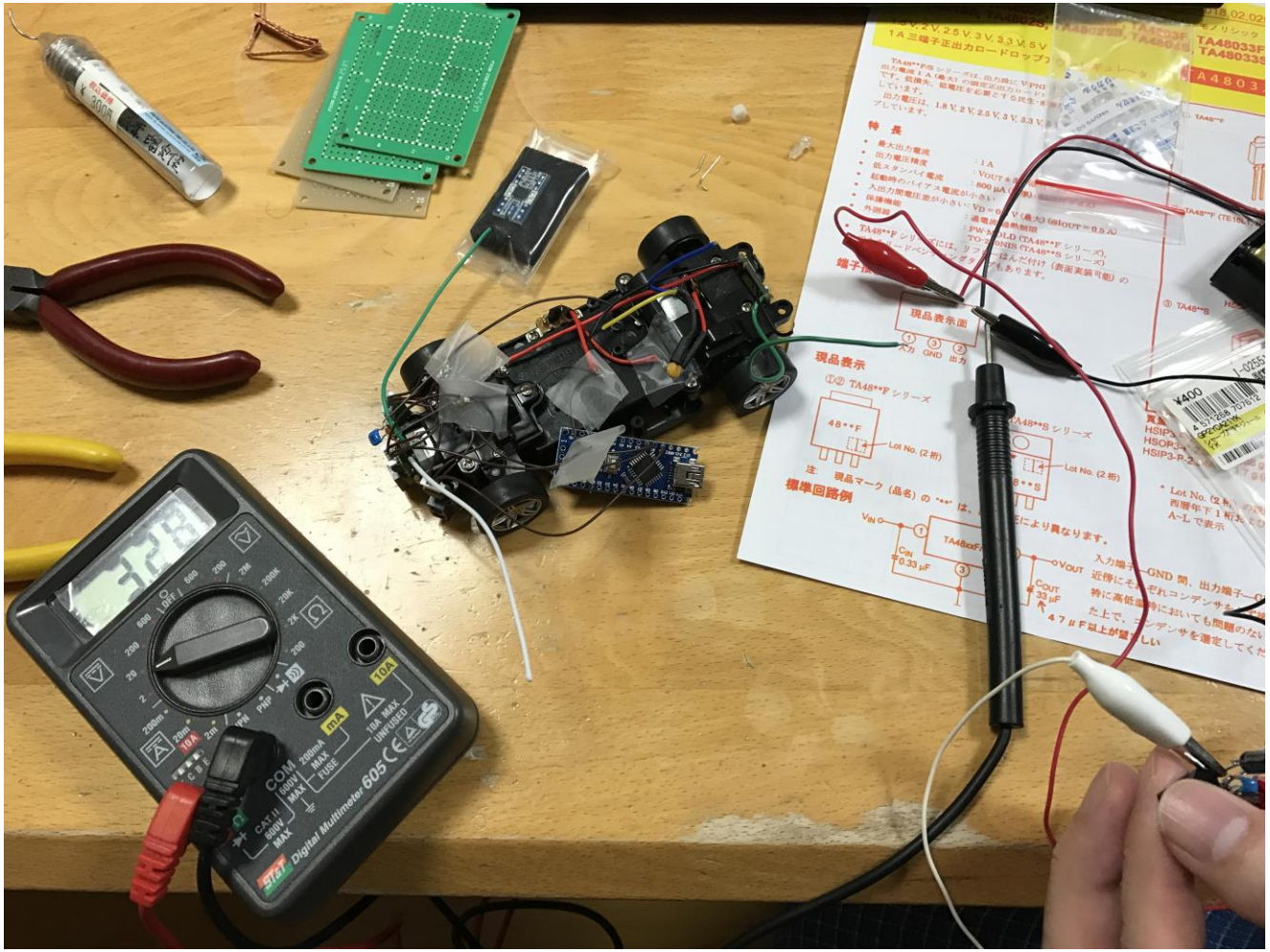
https://youtu.be/8jULg3fZ-jw



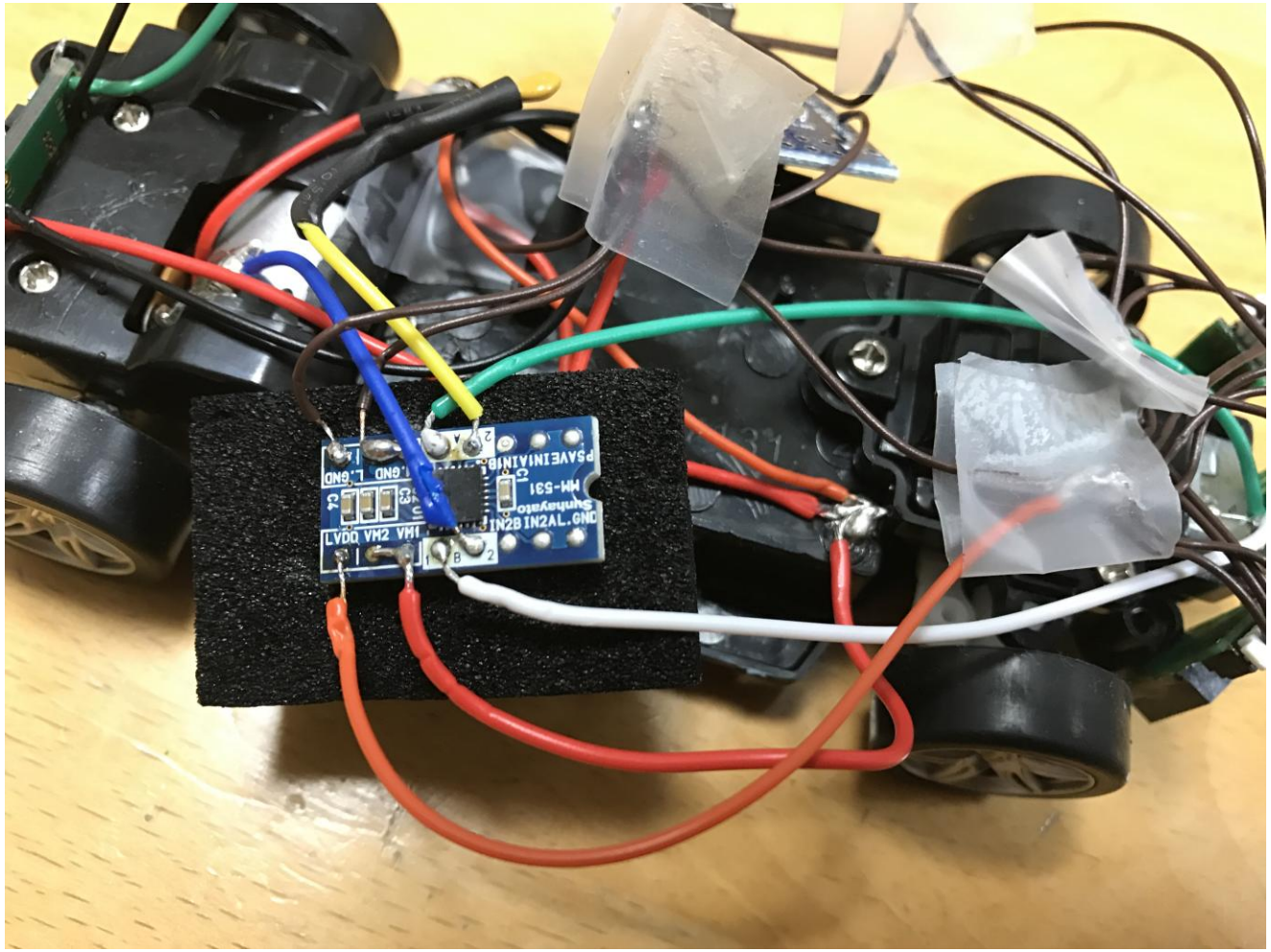
開発中写真



開発中写真

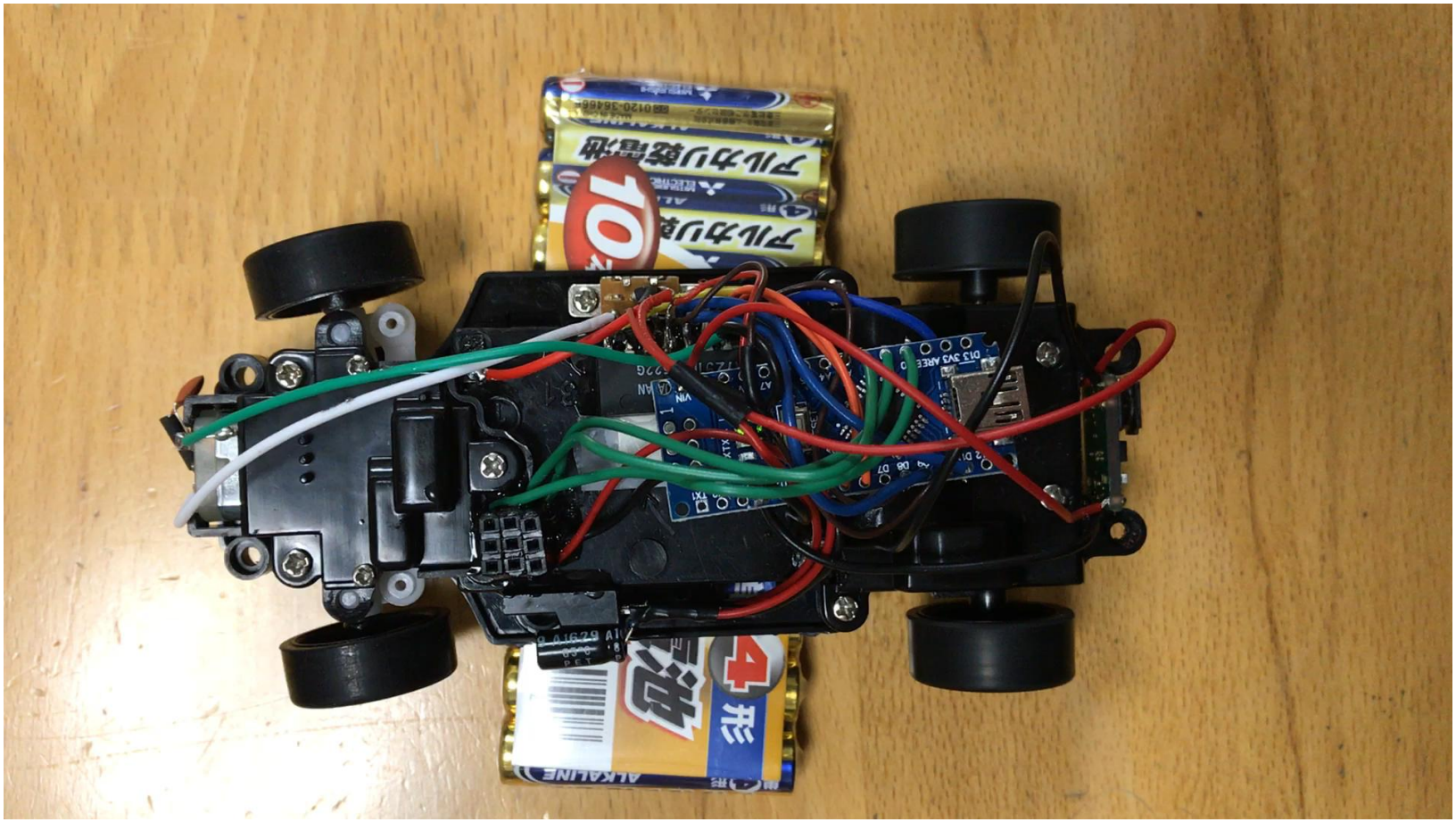


開発中写真



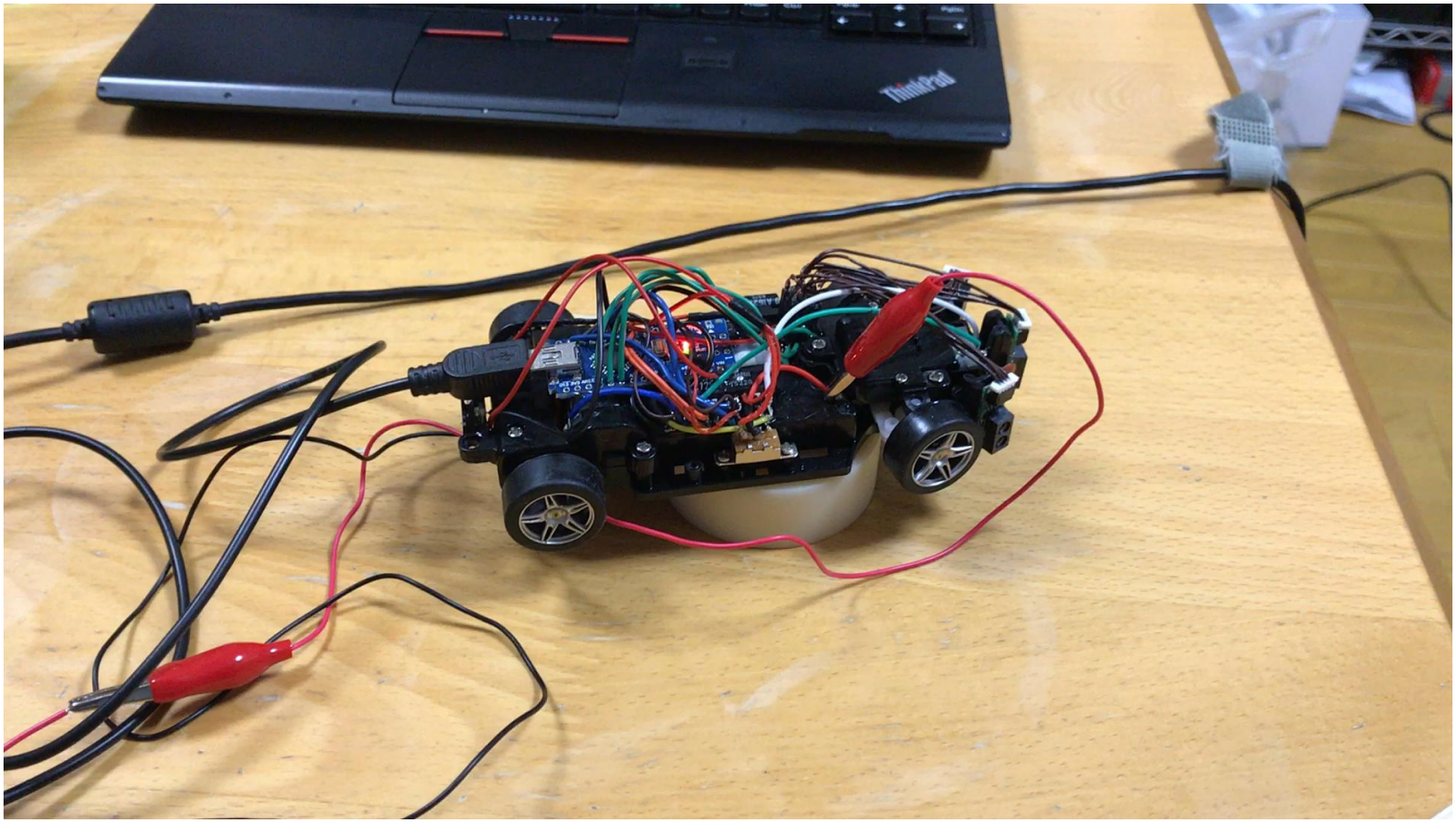
動作試験（動画）

https://youtu.be/8jULg3fZ-jw



動作試験（動画）

https://youtu.be/8jULg3fZ-jw

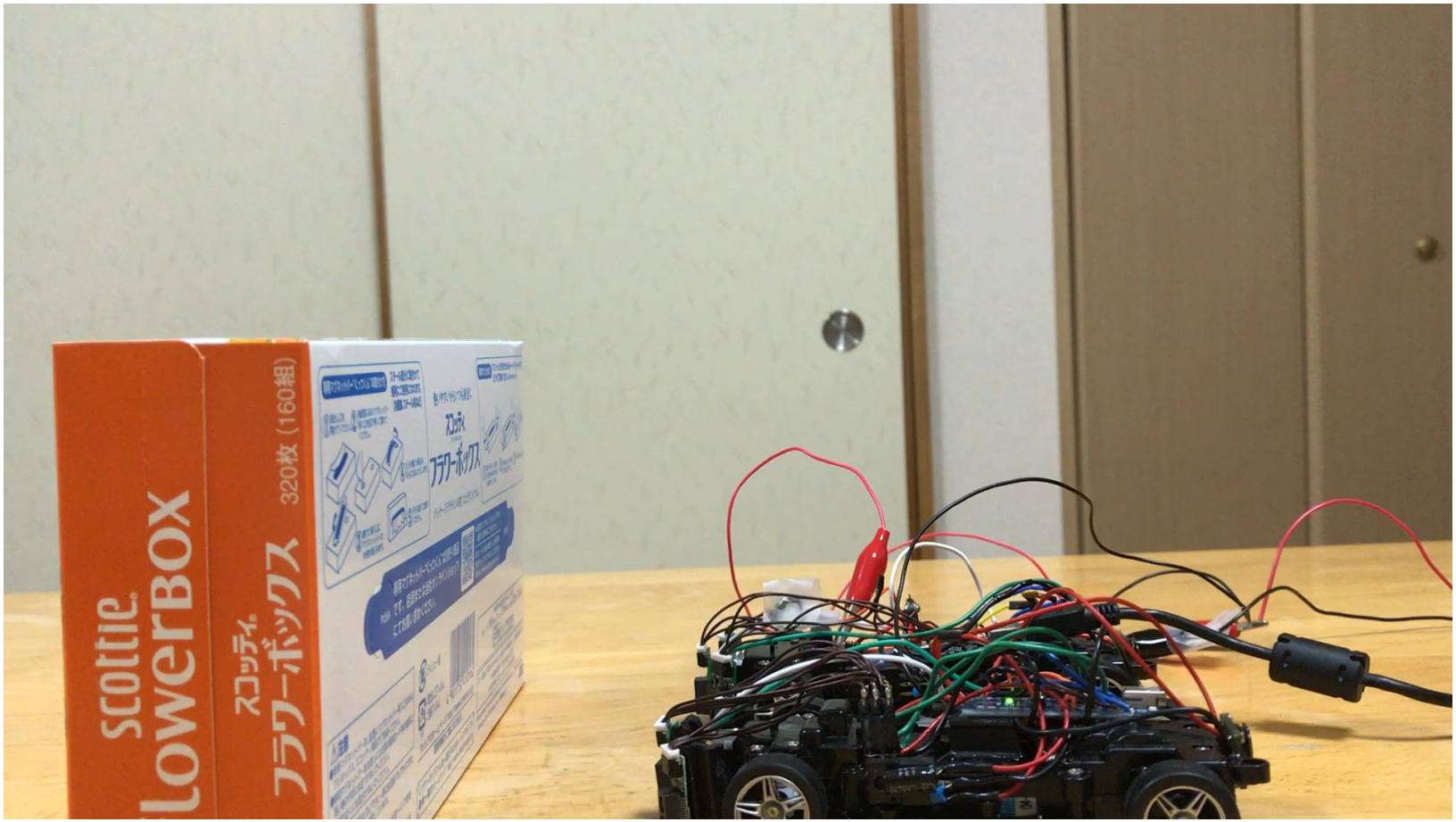


４号車システム部分

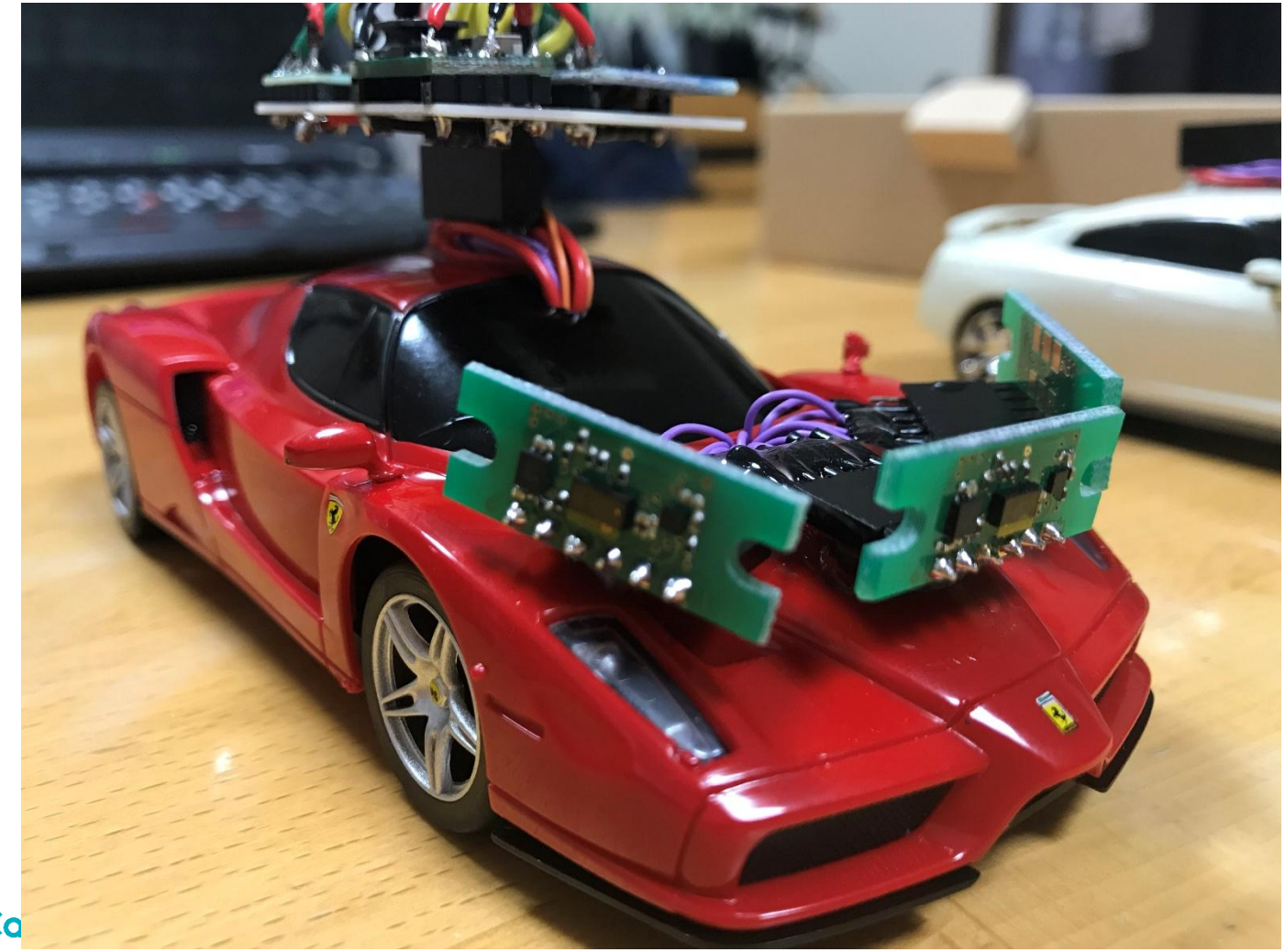


動作試験（動画）

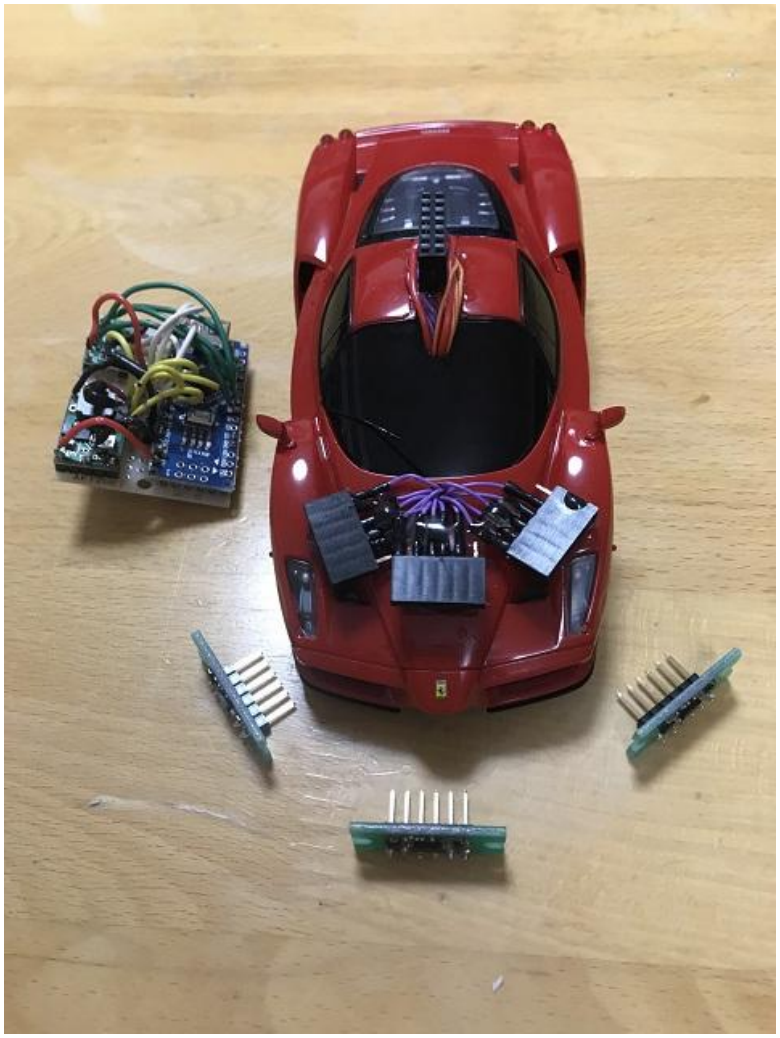
https://youtu.be/dP815X6oKPw



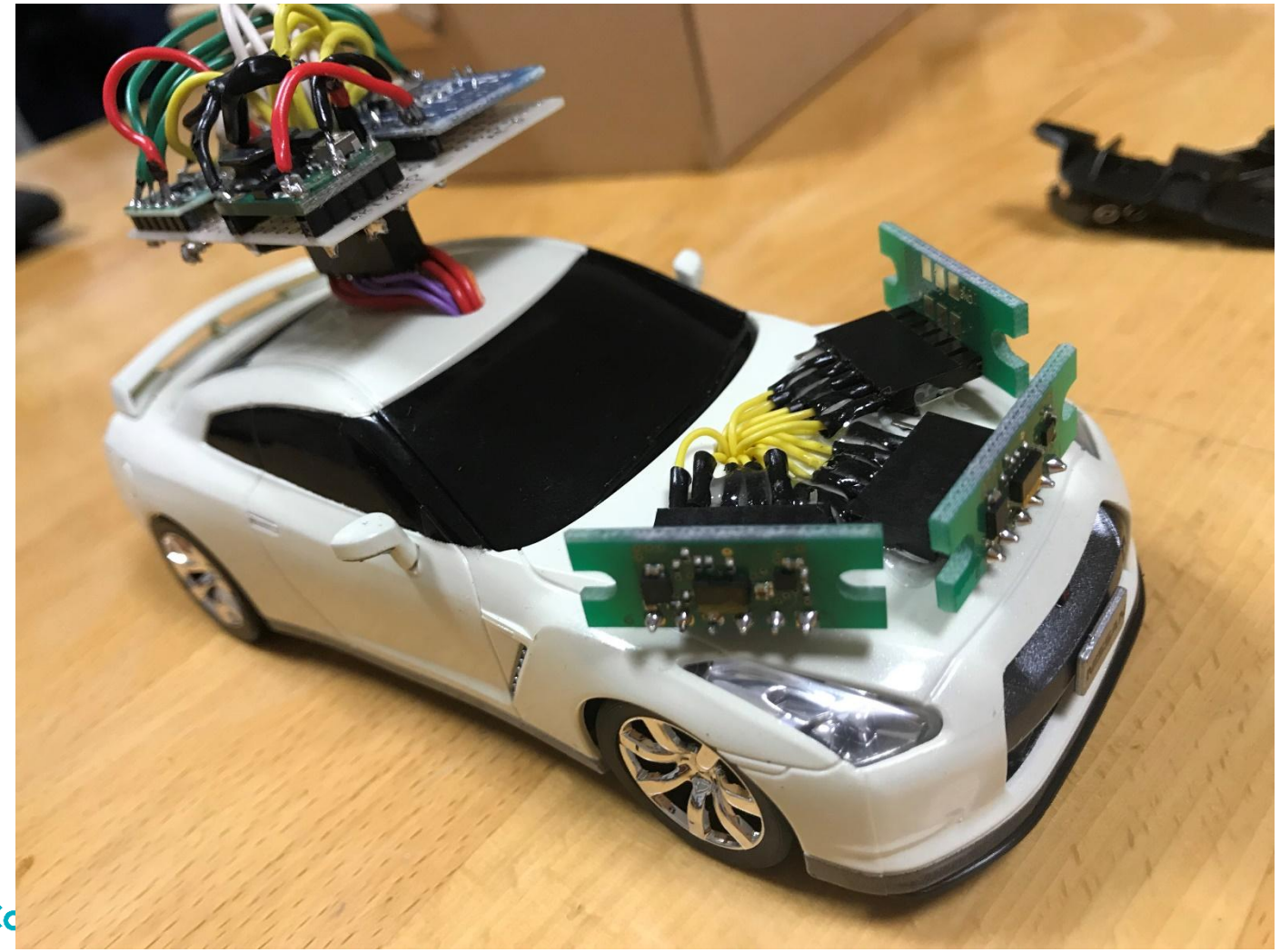
6号車



6号車



7号車



インターフェイスの標準化

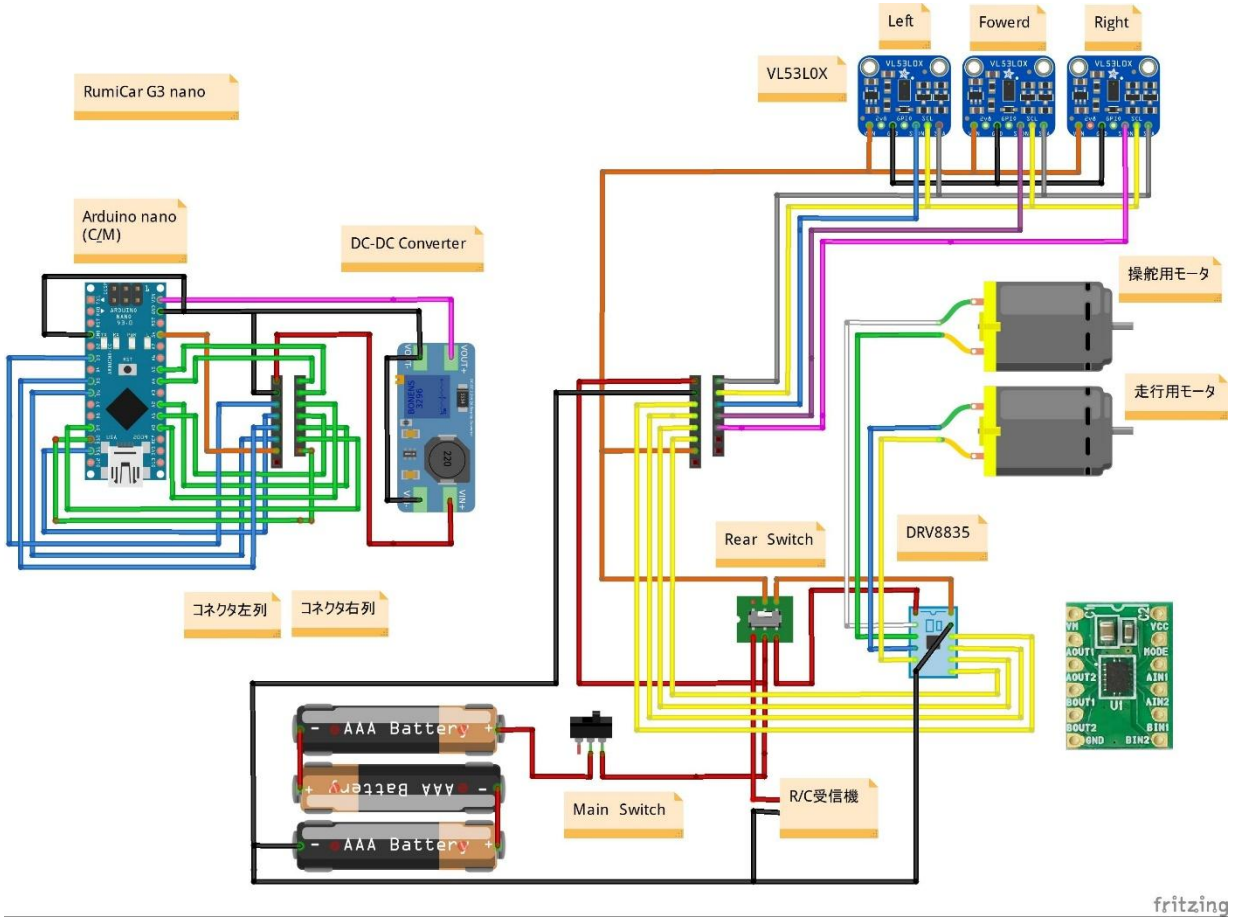


RumiCarの歴史と構成

号車	1号車	2号車	3号車	4号車	5号車
ベース車スケール	1/24		1/32		
コントローラー	Arduino　UNO		Arduino Nano		モジュール式（選択交換可能）
モータ制御	リレーシールド		MM-531	TA7291P	DRV8835
測距モジュール	GP2Y0A21YK		GP2Y0E02A		VL53L0X
測距方式	赤外線				レーザー
Arduino電源	単三乾電池６本（９V）		単四乾電池(4.5V)よりLMR62421を使い７Vに昇圧		
測距モジュール電源	Arduinoより供給		TA48033Sで3.3Vを生成		

モータドライバBD6212HFPは小さすぎて使用断念

第3世代配線図


Left
Fowerd
Right
VL53L0X
RumiCar G3 nano
Arduino nano
(C/M)
DC-DC Converter
操舵用モータ
走行用モータ
Rear Switch
DRV8835
コネクタ左列
コネクタ右列
AAA Battery
AAA Battery
AAA Battery
Main Switch
R/C受信機
fritzing

インターフェイス

旧型式



1 2 3 4 5 6 7

8 9 10 11 12 13 14

NEW



1 2 3 4 5 6 7 7.5

8 9 10 11 12 13 14

誤挿入防止用ダミーピン

8×7ピン　インターフェイス

表 3□第三世代□車体-CM 接続インターフェイス(EX-IF)(旧表記)

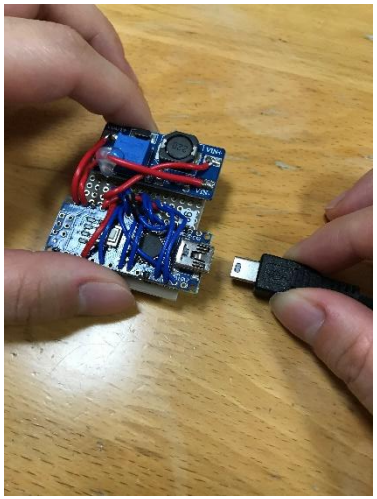
I/O	用途	ピン番号		用途	I/O
O	電源(4.5V)	1	8	SDA:E-GPIO21	I
O	GND	2	9	SCL:E-GPIO22	I
I	AIN1:右折側:A3;·E-GPIO04	3	10	SHDN0(左):E-GPIO19	I
I	AIN2:左折側:A11:·E-GPIO26	4	11	SHDN1(中):E-GPIO18	I
I	BIN1:前進側:A5:·E-GPIO27	5	12	SHDN2(右):E-GPIO05	I
I	BIN2:後進側:A6:·E-GPIO25	6	13	SERVO(操舵)用 A9 番ピン E-GPIO32	I
I	信号最大電圧及び電力供給(通常コンピュータの VCC)(5V または 3.3V)VL53L0X と DRV8835 の VCC へ	7	14	SERVO 用 A10 番ピン E-GPIO33	I
	誤挿入防止用のダミーピン (接続先は無し)	7.5			

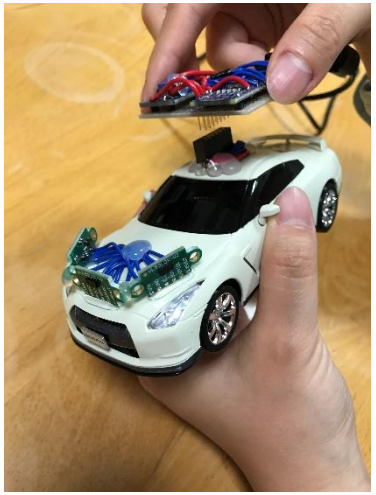
誤挿入防止用7.5をダミーピンとして追加

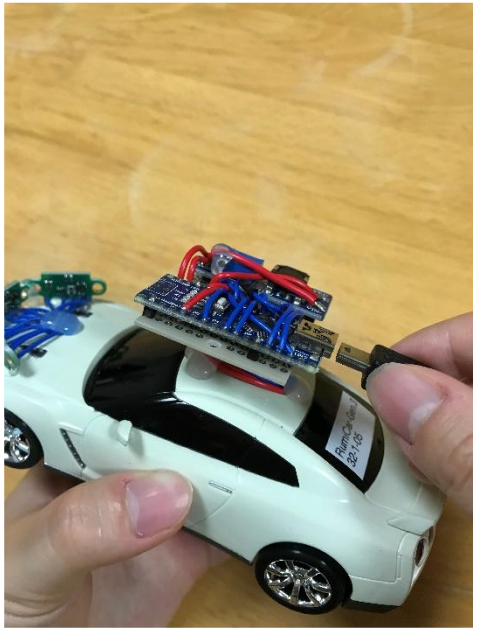
準備

注意点、必要なもの、準備

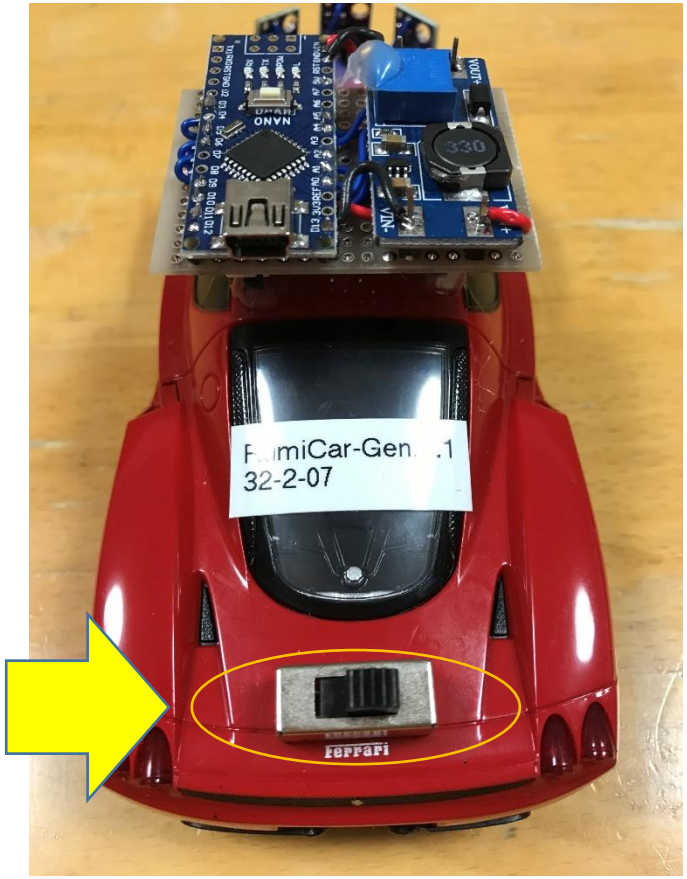
ケーブルの抜き差し注意点







コンピュータ制御へ設定



- （一部のモデルのみ）
- マニュアル制御とコンピュータ制御の切り替えスイッチ好きモデルは切り替えスイッチを右側へ切り替えます
- 右側がコンピュータ制御側です

- 右側にするとCM（ArduinoやESP32）からの命令で走ります
- （左側にするとラジコンで操作できるようになります）

ケーブルの抜き差し注意点

- まず、コンピュータモジュールにUSBケーブルを接続する
- それから、コンピュータモジュール(以下CMと呼びます)を車体に接続する
- 注意！車体にCMを装着した状態でコンピュータモジュールにUSBケーブルを接続しないこと
- 車体にCMを装着したままUSBケーブルを抜かないこと
- CMコネクタ部分に過度な力が加わり破損の原因になります

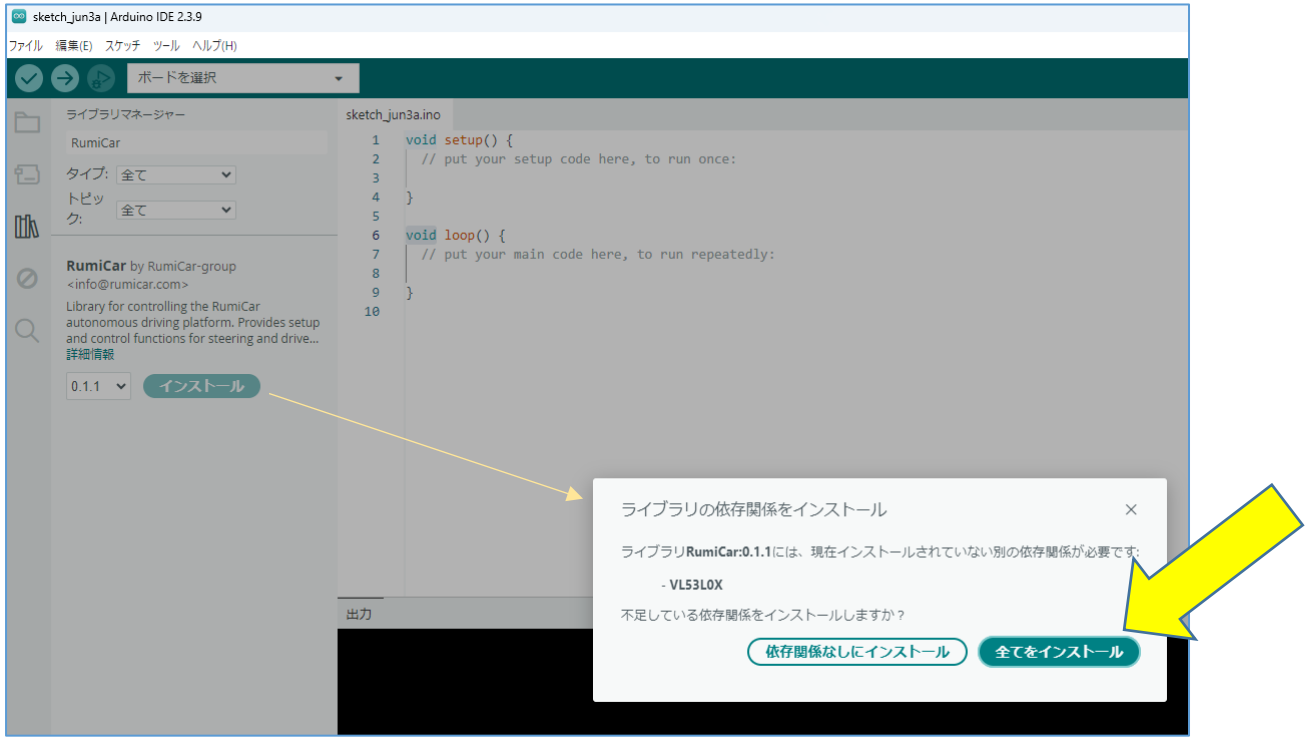
Arduino IDEのインストール

- URL:https://www.arduino.cc/en/Main/Software
- 「Windows Win 10 or newer(64-bit)」をインストール
- (既にインストール済の人はこの作業は飛ばしていただいて結構です)



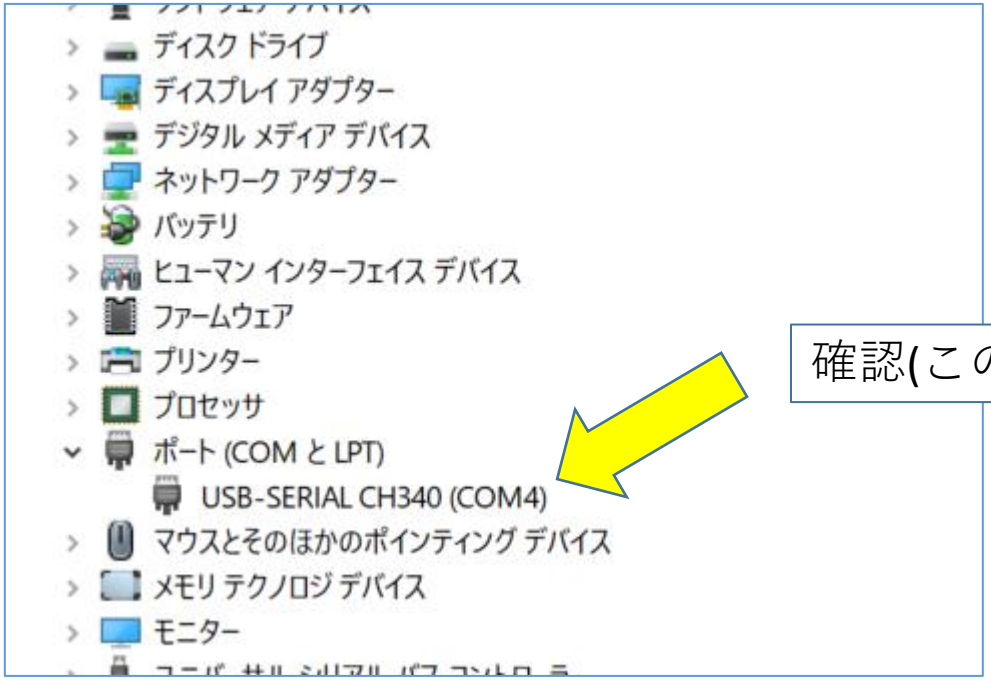
RumiCarライブラリのインストール

- Arduino IDEを開く
  - 「ツール」->「ライブラリを管理...」で「RumiCar」を検索し「全てをインストール」する。入っている人は0.5.0以上に「更新」



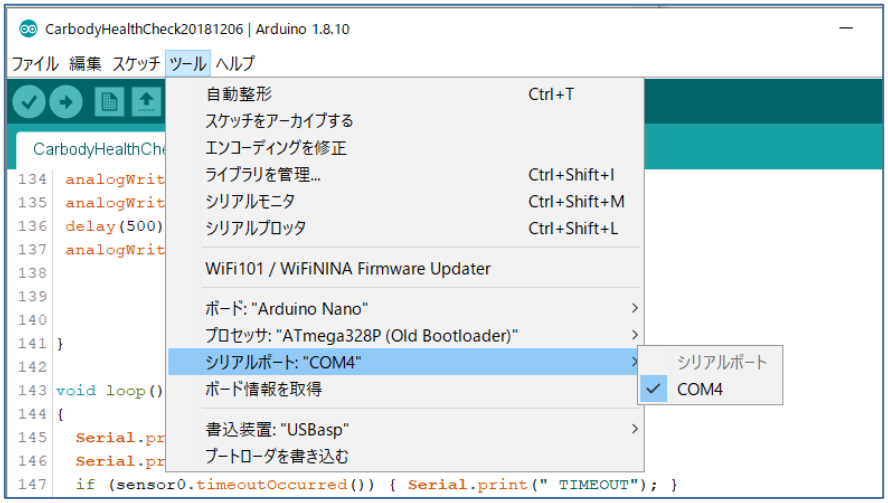
COMポートの確認（Windowsの場合）

- RumiCarのコンピュータモジュールをUSBケーブルでパソコンに接続する
- デバイスマネージャで接続されたCOMポート番号を確認する。CH340を探します（COM番号を覚えておくこと）



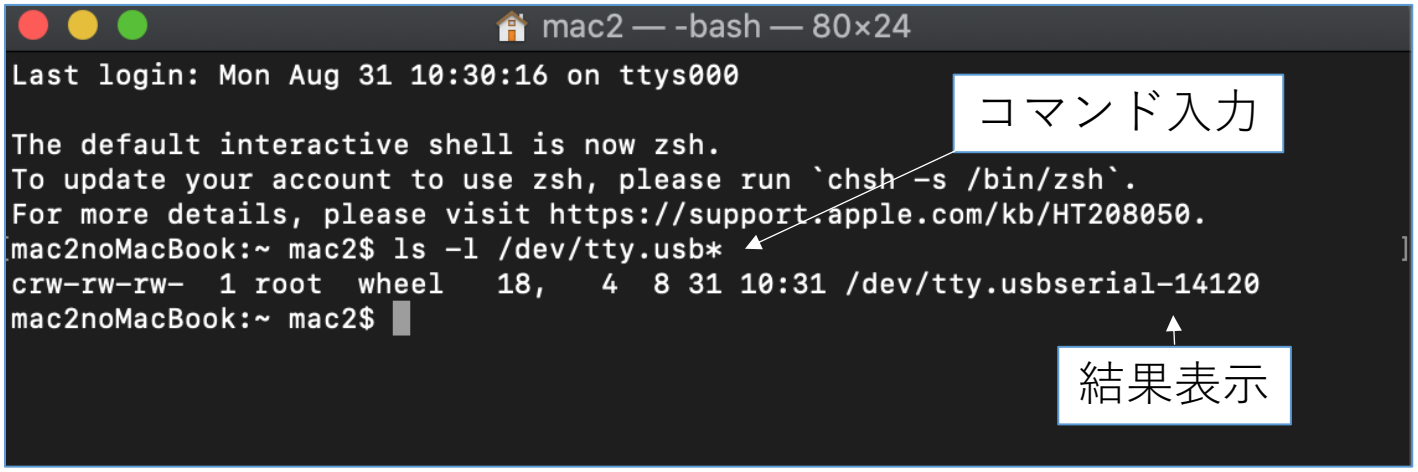
Arduino IDEの設定

- ボード：Arduino Nano
- プロセサ：ATmega328P（Type-C端子のCM）
  - 旧型のCM（USB端子がMiniまたはMicro）は「ATmega328P(Old Bootloader)」。端子の形で見分けます（下の画面例は旧型）
- シリアルポート：デバイスマネージャで確認した番号



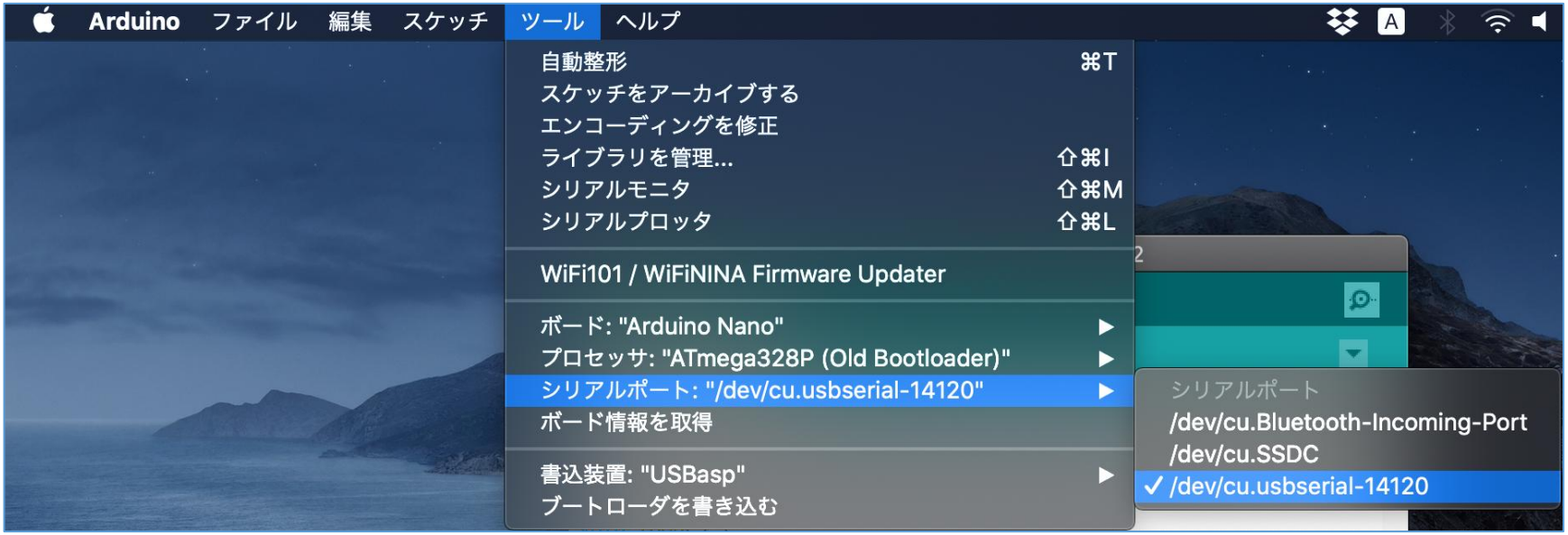
COMポートの確認（Macの場合）

- Terminalソフトを起動します
- コマンド”ls –l /dev/tty.usb*”と入力することで接続されているCOMポートの一覧を表示させます
- Arduinoが接続されたCOMポートを確認する
- 下記の例では「usbserial-14120」（覚えておくこと）



Arduino IDEの設定 (Macの場合)

- ボード：Arduino Nano
- プロセサ：ATmega328P（Type-C）。旧型（Mini/Micro-USB端子）は「ATmega328P(Old Bootloader)」
- シリアルポート：Terminalで確認したデバイス



設定の確認

- 重要！　もう一度、確認しよう！

項目	設定	備考
ボード	Arduino Nano	Arduino UNOでは動きません
プロセッサ	ATmega328P	Type-C端子のCM（標準） 旧型（Mini/Micro-USB端子）は 「ATmega328P(Old Bootloader)」
シリアルポート	“デバイスマネージャで確認した番号”	ボードを変えると、変わったりしますので注意
書込装置	（変更不要）	通常の書込みでは使いません
シリアルポートのボーレート	9600bps	Arduino Nanoでは9600bpsに設定されている必要があります Arduino IDEのシリアルモニタを開いて確認します

CMの端子と設定の対応

CM の端子	見た目	プロセッサの設定	間違えたとき
Type-C（現在の標準）	小さな楕円形。表裏なし	**ATmega328P**	「Old Bootloader」を選ぶと書き込みに失敗
Mini-USB または Micro-USB（旧型）	台形。向きがある（Mini は厚め、Micro は平たい）	**ATmega328P (Old Bootloader)**	「ATmega328P」を選ぶと書き込みに失敗

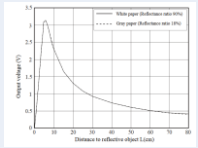
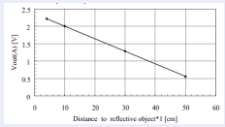
- 「Old Bootloader」＝古い書き込みプログラム（ブートローダ）が入った旧型CM用の設定。端子の形で見分けます。どちらの端子もUSBシリアルは CH340 で、ドライバとデバイスマネージャの表示は同じ
- 書き込みに失敗したときの表示：programmer is not responding → まず「プロセッサ」が端子と合っているか、次にシリアルポートとUSBケーブル（充電専用は通信できません）
- Exercise-4 でAIがコンパイル確認に使う設定も端子で変わります：Type-C は `cpu=atmega328`、旧型は `cpu=atmega328old`

距離の測定

レーザー測距モジュール

測距モジュール

- 測距モジュールの比較

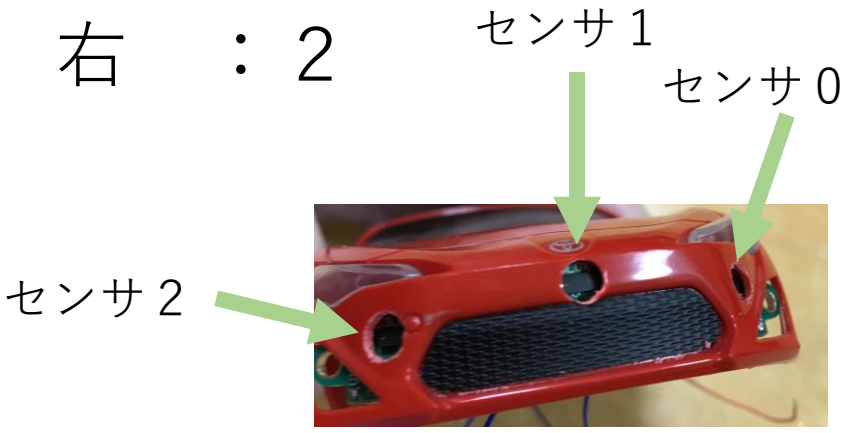
型番	GP2Y0A21YK0F	GP2Y0E02A	VL53L0X	HC-SR04
測定方法	赤外線	赤外線	レーザー	超音波
出力	アナログ 	アナログ 	ディジタル値	アナログ
推奨電源電圧	5V	3.3V	2.6-5.5V (ボード依存)	5V
消費電力	最大40mA	最大36mA	20mA	15mA
測定可能距離	80cm	50cm	2m	2m
RumiCarでの採用	1、2号車	3、4号車	5号車以降	非採用
備考	出力と距離が比例しない			高速移動時のエコーキャンセル処理問題

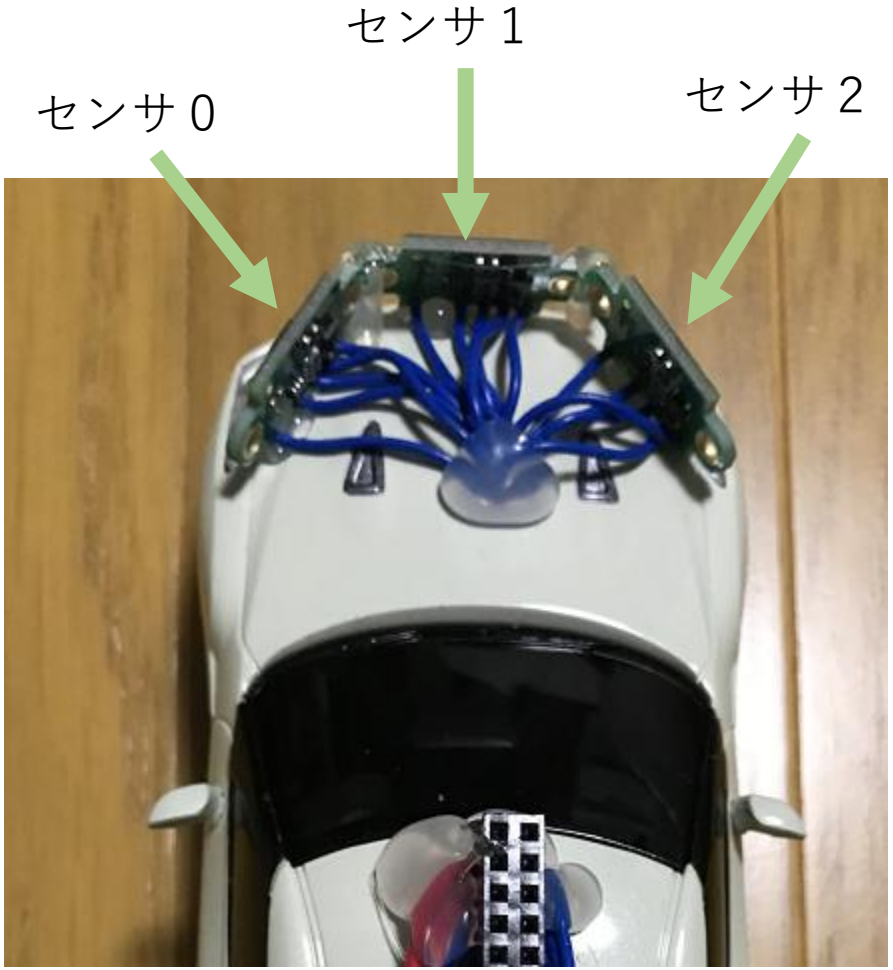
グラフはシャープ社データシートより

レーザー測距モジュール

- 車体前方に３基搭載

左　：０
中央：１
右　：２





VL53L0X

- Rumicarに搭載されているレーザー測距モジュールはVL53L0Xです
- VL53L0Xの特徴
  - 最大２ｍまで計測可能
  - I2Cで測定値を取得可能
  - I2Cアドレスはソフトウエアにより再設定可能
  - パラメータ設定により大まかにモード設定可能
    - 高速測定モード
    - 長距離測定モード
    - 高精度測定モード
    - サンプルプログラムは高速測定モードに設定されてます

測距方法

- 各測距モジュールの測距値の取得は、例えば下記の命令を使います。結果は整数値（int）でｍｍ（ミリメートル）の単位で得ることができます

  readRangeSingleMillimeters()

- センサ1の測定値を取得するには例えば下記のようにします

  i　=　sensor1.readRangeSingleMillimeters();

- （参考）ライブラリの関数RC_readを使っても同じ値を取得できます

  i　=　RC_read(CENTER);

スケッチ例からExerciseを開く

（GitHubからのダウンロードは不要です）

Arduino IDEのメニューから
「ファイル」
▼
「スケッチ例」
▼
「RumiCar」（一覧の下の方）
▼
「Exercise-1.1」など

※RumiCarライブラリのインストール（前の手順）が必要です

【参考】上級編（A2.3など）は
GitHubにあります（このイベントでは使用



Exerciseのヘッダー作製、
リファクタリング、上級編プログラムの追加を
してくださったのは
稲玉　繁樹　氏

Exercise-1

距離を測ってみよう

Exercise-1.1

中央のセンサで測距しよう！

Exercise-1.1
中央のセンサで距離を測る(準備)

- RumiCarの車体の電源をオフにします
- CMをRumiCarの車体より取り外してください
- CMにUSBケーブルを接続します
- USBケーブルをパソコンに接続します
- CMをRumiCarの車体に取り付けます

Exercise-1.1
中央のセンサで距離を測る

- スケッチ例から「Exercise」を開きます
- loop()の中に、計測値を表示するコードを1行追加します

```
 8  void loop()
 9  {
10
11  }
```



```
 8  void loop()
 9  {
10    Serial.println(sensor1.readRangeSingleMillimeters());
11  }
```

- Serial.print命令やSerial.println命令でシリアルモニタにデータを表示できるよ
- Serial.printは改行無し
- Serial.printlnは表示後改行ありだよ

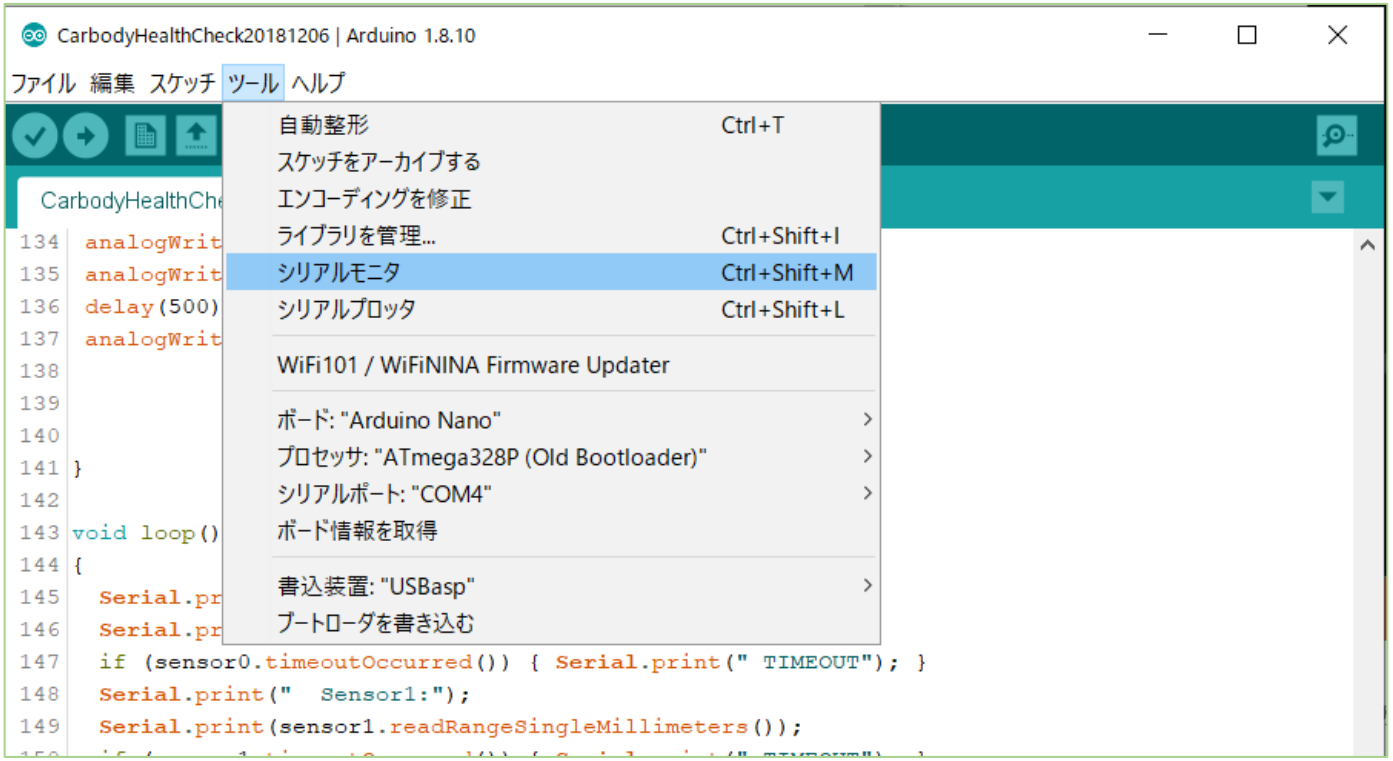
Exercise-1.1
中央のセンサで距離を測る

- コンパイルとボードへの書き込み



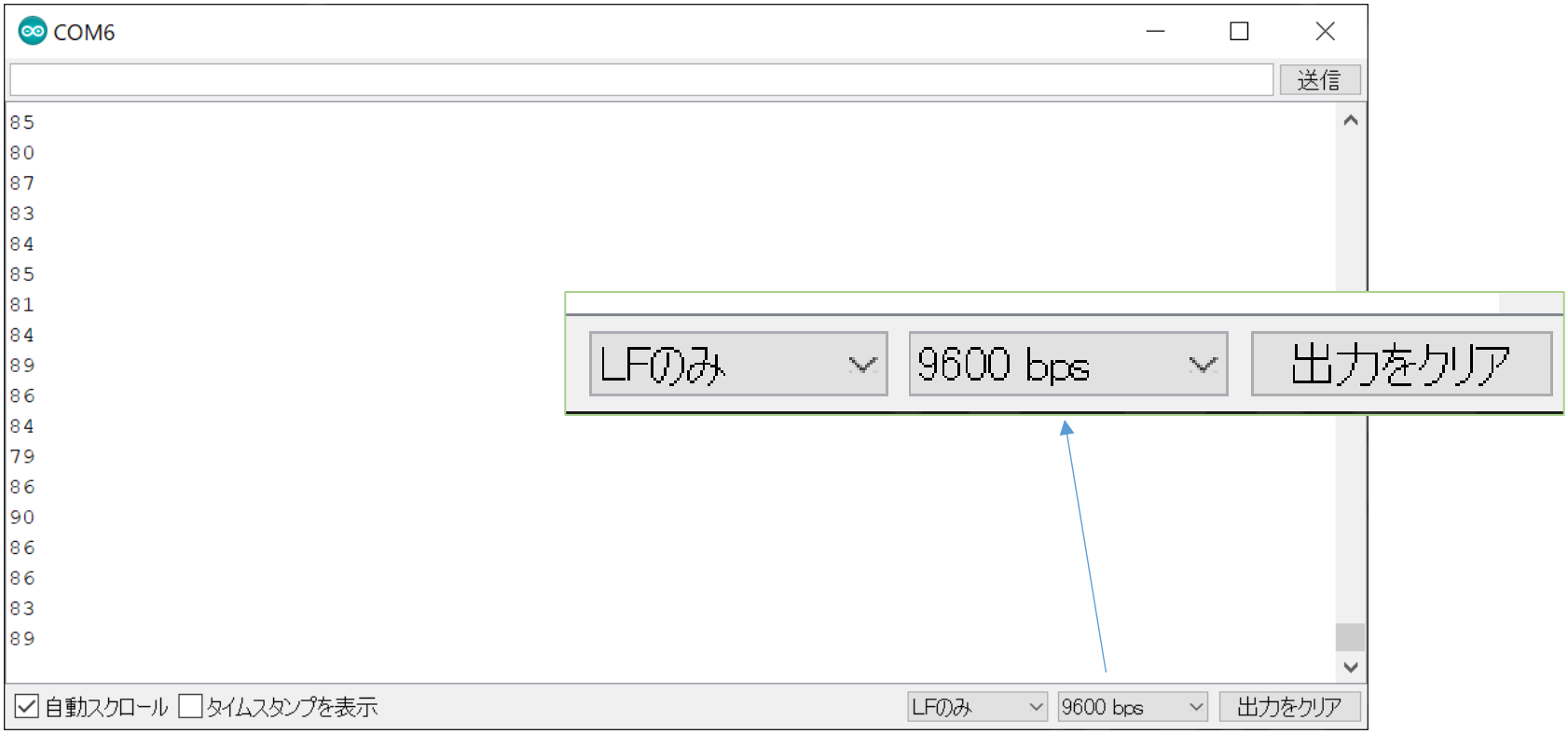
Exercise-1.1
中央のセンサで距離を測る（続き）

- シリアルモニタで測定値を表示させます
  - ツール　->　シリアルモニタ



Exercise-1.1
中央のセンサで距離を測る（続き）

- うまく表示できない場合はボーレートを確認してみてください。（9600 bpsになっている必要があります）



Exercise-1.1
中央のセンサで距離を測る（続き）

- うまく表示できない場合は「準備」の項の「設定の確認」のページの内容を再度確認してください

Exercise-1.2

３つのセンサで測距だ！

Exercise-1.2
3つのセンサで距離を測る

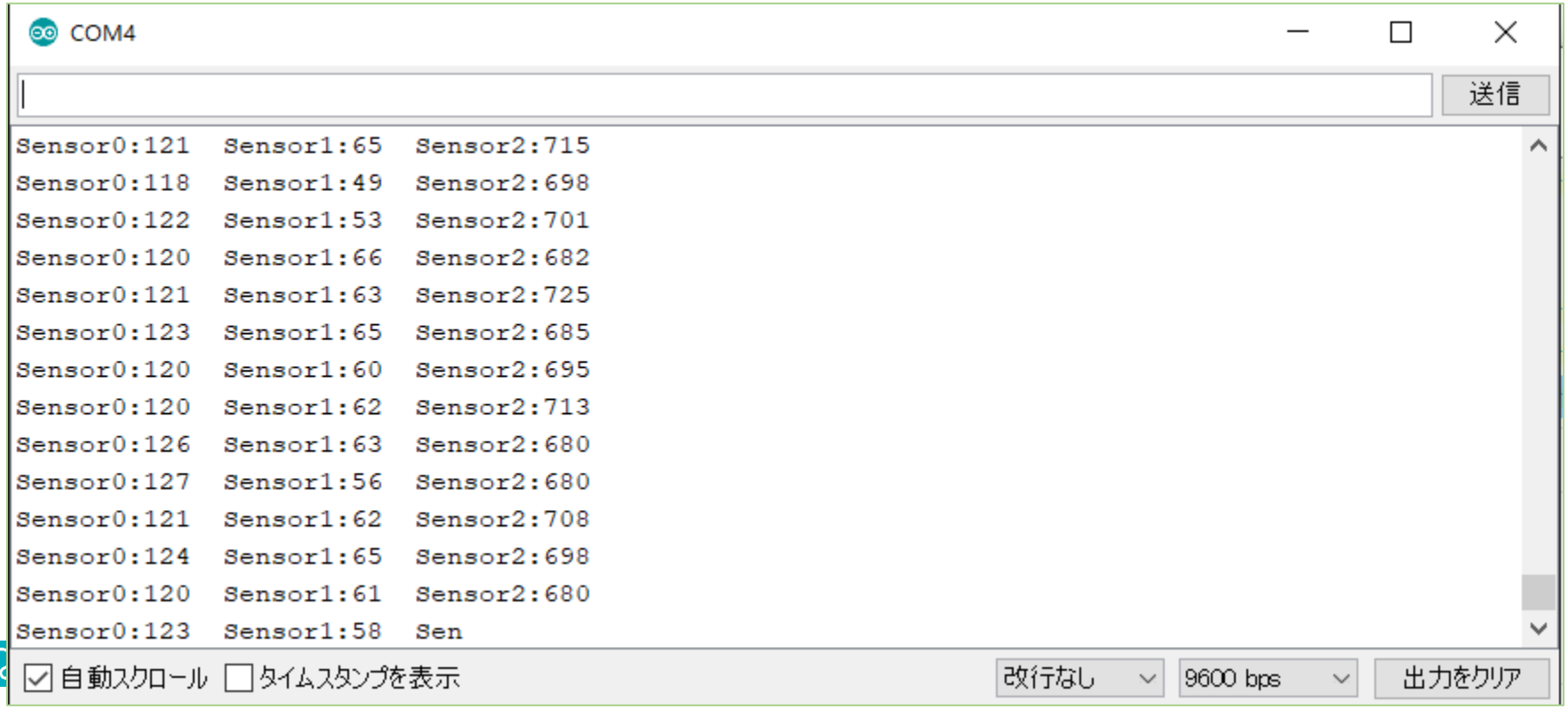
- スケッチ例から「Exercise-1.2」を開きます

センサー	センサーの場所
Sensor0	左
Sensor1	中央
Sensor2	右

```
 8  void loop()
 9  {
10    Serial.print("Sensor0:");
11    Serial.print(sensor0.readRangeSingleMillimeters());
12    Serial.print("  Sensor1:");
13    Serial.print(sensor1.readRangeSingleMillimeters());
14    Serial.print("  Sensor2:");
15    Serial.println(sensor2.readRangeSingleMillimeters());
16  }
```

Exercise-1.2
３つのセンサで距離を測る（続き）

- シリアルモニタで測定値を表示させる
  - ツール　->　シリアルモニタ

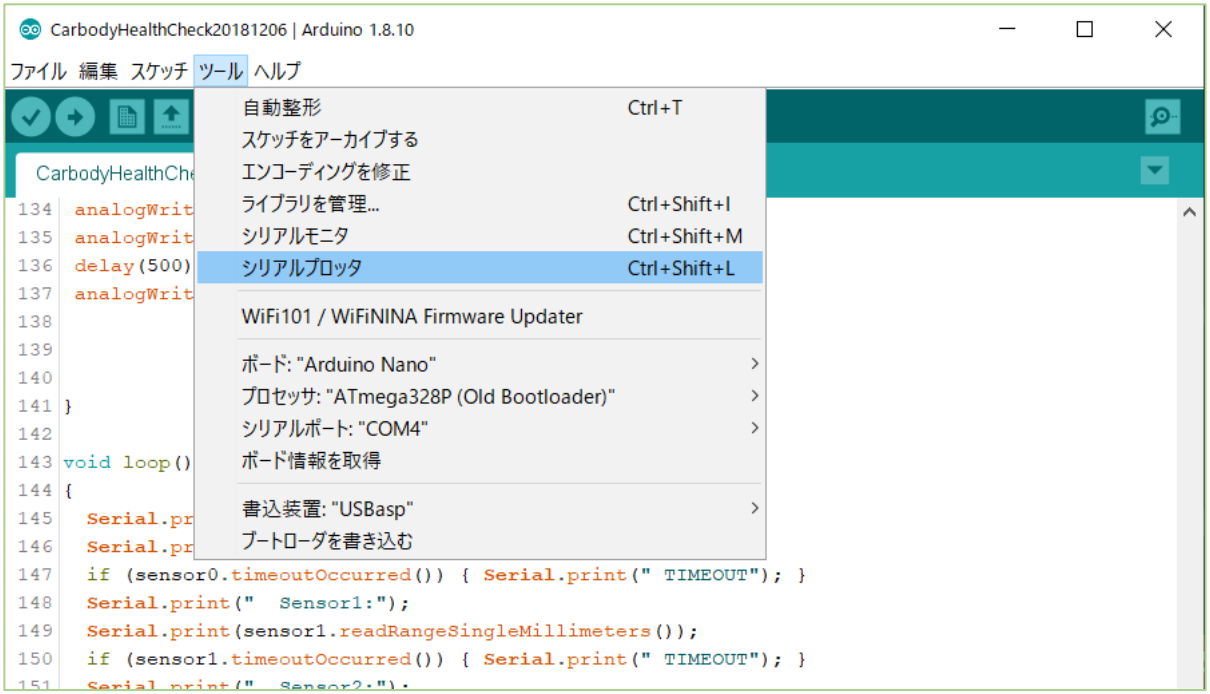


Exercise-1.3

シリアルプロッタを使ってみよう

Exercise-1.3
シリアルプロッタを使ってみよう

- （プログラムはそのままExercise-1.2のものを使います）
- シリルモニタを閉じる
- ツール　->　シリアルプロッタ
  - ※シリアルモニタのウインドウが開いている場合は閉じる必要があります

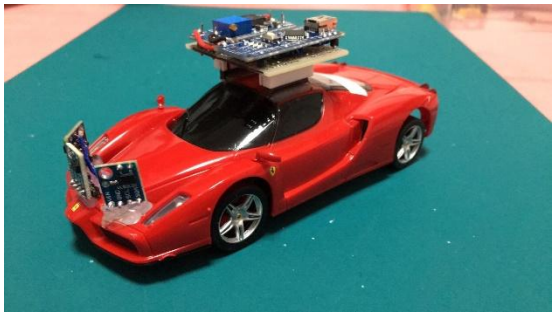


休憩室



	紫外線硬化樹脂	ホットメルト接着剤 （通称グルー）
長所	• 短時間で固まる • 強度も高い • 透明で見栄えが良い • 重ね塗りが可能	• 安い • 失敗しても剥がしてやり直しが効く
短所	• 長期間経過すると脆くなり崩れる	• 透明でないので見栄えが良くない（イ〇スタ映えしない） • 紫外線硬化樹脂程強度がない





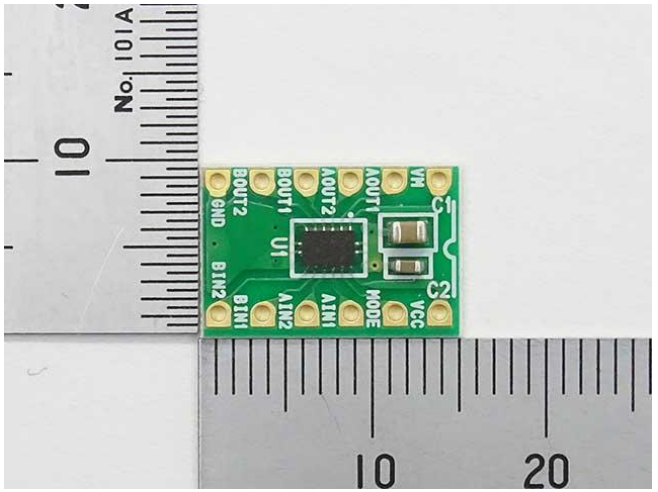
モータ制御

ハンドルを動かす、走る

モータドライバ

- RumiCarに搭載されているモータドライバはDRV8835です
- DRV8835の特徴
  - ２チャンネル
  - 各チャンネル1.5Aまで流せる
  - PWM(250kHz)
  - 低消費電力
  - 低電圧降下（モータ側）
  - ロジック電源（VCC）とモータ電源が独立している
    - ロジック電源電圧は2から7V
    - モータ電源電圧は11Vまで

DRV8835



DRV8835諸元(1/2)

T_A = 25°C (unless otherwise noted)

		MIN	NOM	MAX	UNIT
V_{CC}	Device power supply voltage	2		7	V
V_M	Motor power supply voltage	0		11	V
V_{IN}	Logic level input voltage	0		V_{CC}	V
I_{OUT}	H-bridge output current(1)	0		1.5	A
f_{PWM}	Externally applied PWM frequency	0		250	kHz

(1) Power dissipation and thermal limits must be observed.

DRV8835 Dual Low-Voltage H-Bridge ICデータシート　2016年8月9日　より

DRV8835諸元(2/2)

T_A = 25°C, V_M = 5 V, V_{CC} = 3 V (unless otherwise noted)

PARAMETER		TEST CONDITIONS	MIN	TYP	MAX	UNIT
POWER SUPPLY						
I_{VM}	VM operating supply current	No PWM, no load		85	200	µA
		50 kHz PWM, no load		650	2000	
I_{VMQ}	VM sleep mode supply current	V_M = 2 V, V_{CC} = 0 V, all inputs 0 V		5		nA
		V_M = 5 V, V_{CC} = 0 V, all inputs 0 V		10	95	
I_{VCC}	VCC operating supply current			450	2000	µA
V_{UVLO}	VCC undervoltage lockout voltage	V_{CC} rising			2	V
		V_{CC} falling			1.9	
LOGIC-LEVEL INPUTS						
V_{IL}	Input low voltage				0.3 × V_{CC}	V
V_{IH}	Input high voltage		0.5 × V_{CC}			V
I_{IL}	Input low current	V_{IN} = 0	–5		5	µA
I_{IH}	Input high current	V_{IN} = 3.3 V			50	µA
R_{PD}	Pulldown resistance			100		kΩ
H-BRIDGE FETS						
$R_{DS(ON)}$	HS + LS FET on resistance	V_{CC} = 3 V, V_M = 3 V, I_O = 800 mA, T_J = 25°C		370	420	mΩ
		V_{CC} = 5 V, V_M = 5 V, I_O = 800 mA, T_J = 25°C		305	355	
I_{OFF}	OFF-state leakage current				±200	nA
PROTECTION CIRCUITS						
I_{OCP}	Overcurrent protection trip level		1.6		3.5	A
t_{DEG}	Overcurrent de-glitch time			1		µs
t_{OCR}	Overcurrent protection retry time			1		ms
t_{DEAD}	Output dead time			100		ns
t_{TSD}	Thermal shutdown temperature	Die temperature	150	160	180	°C

RumiCar

DRV8835 Dual Low-Voltage H-Bridge ICデータシート　2016年8月9日　より

IN/INモード

- IN/INモードとPHASE/ENABLEモードを持っています
- RumiCarではDRV8835をIN/INモードで利用します

Table 3. IN/IN Mode

MODE	xIN1	xIN2	xOUT1	xOUT2	FUNCTION (DC MOTOR)
0	0	0	Z	Z	Coast
0	0	1	L	H	Reverse
0	1	0	H	L	Forward
0	1	1	L	L	Brake

DRV8835 Dual Low-Voltage H-Bridge ICデータシート　2016年8月9日　より

Phase/Enableモードの場合

Table 4. Phase/Enable Mode

MODE	xENABLE	xPHASE	xOUT1	xOUT2	FUNCTION (DC MOTOR)
1	0	X	L	L	Brake
1	1	1	L	H	Reverse
1	1	0	H	L	Forward

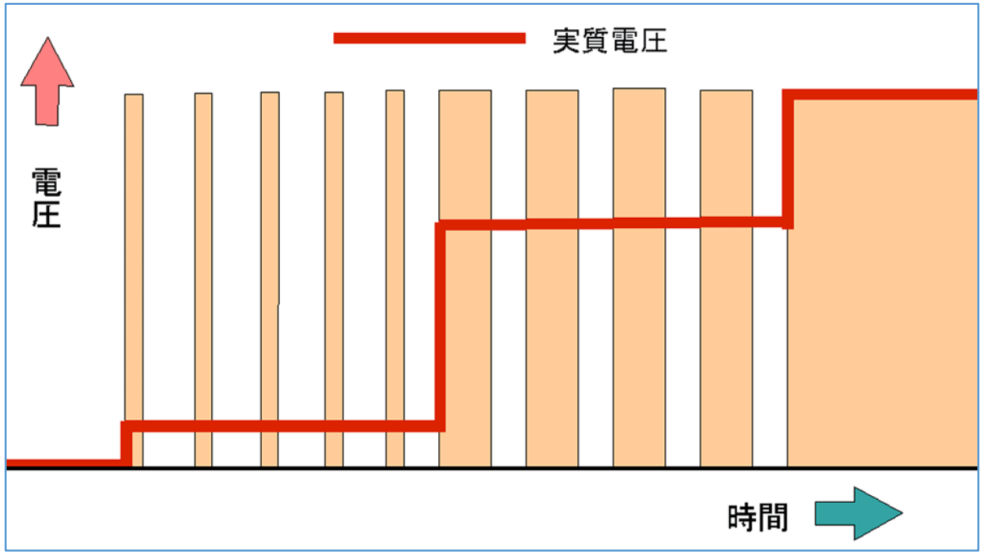
- Enableが1の時、モーターが回転
- Enableが0の時、モーターが停止
- Phaseが1の時は逆回転の向き
- Phaseが0の時は正転の向き

Enableが回転数、Phaseが回転方向

DRV8835 Dual Low-Voltage H-Bridge ICデータシート　2016年8月9日　より

PWMの説明

- RumiCarでは走行速度制御にPWM技術を利用しています
- PWMとは”Pulse Width Modulation”の略語で電圧を短い間隔でオンとオフを繰り返すことによって見かけの電圧を変化させる技術です



図はWiKipediaより

ArduinoのPWM

- ArduinoのPWMはタイマと連動しています
- Arduinoのタイマは3個あります

タイマ	ピン	周波数
タイマ0	5、6	980Hz
タイマ1	9、10	490Hz
タイマ2	3、11	490Hz

- ArduinoのServoライブラリを使うとタイマ1は使えなくなってしまいます

Arudino版でのピンアサイン		
機能	タイマ	ピン
操舵	タイマ2	3、11
走行	タイマ0	5、6

RumiCarでのPWM制御

- RumiCarのプログラムサンプルではピンにマクロ設定をしています
- 例えば右にハンドルを曲げたいときはAIN1をオン、AIN2を”0”に設定します
- ※この例はArduino版のピンアサインです。ESP版などはピンアサインが異なりますがマクロを使ってピンアサインの違いを意識することなくプログラムを開発できるようにしてあります

タイマ	機能	ピン	マクロ設定
タイマ2	右折	3	AIN1
	左折	11	AIN2
タイマ0	前進	5	BIN1
	後進	6	BIN2

RumiCarでのRC関数

- RumiCarではプログラムを開発しやすいように下記関数を用意しました

動作	関数の型	関数名	第一引数 (int)	第二引数 (int)	動作	エラー時の戻り値
操舵	int	RC_steer	RIGHT	(無し)	右操舵	0
			LEFT		左操舵	
			CENTER		中央	
走行	int	RC_drive	FREE	0から255 (PWM値)	ニュートラル	0
			REVERSE		後進	
			FORWARD		前進	
			BRAKE		ブレーキ	

- RC_driveのFREE時とBRAKE時の第二引数の値は指定は必要ですが値自体は無視されます
- RC_driveの第二引数の値はPWM値であり値が大きいほどモータに電力を掛けることができます
- 第一引数はint型です。この表の値を指定すると最終的にマクロで展開されます

RumiCarでのRC関数（測距）

- 距離を測る関数RC_readも用意しています

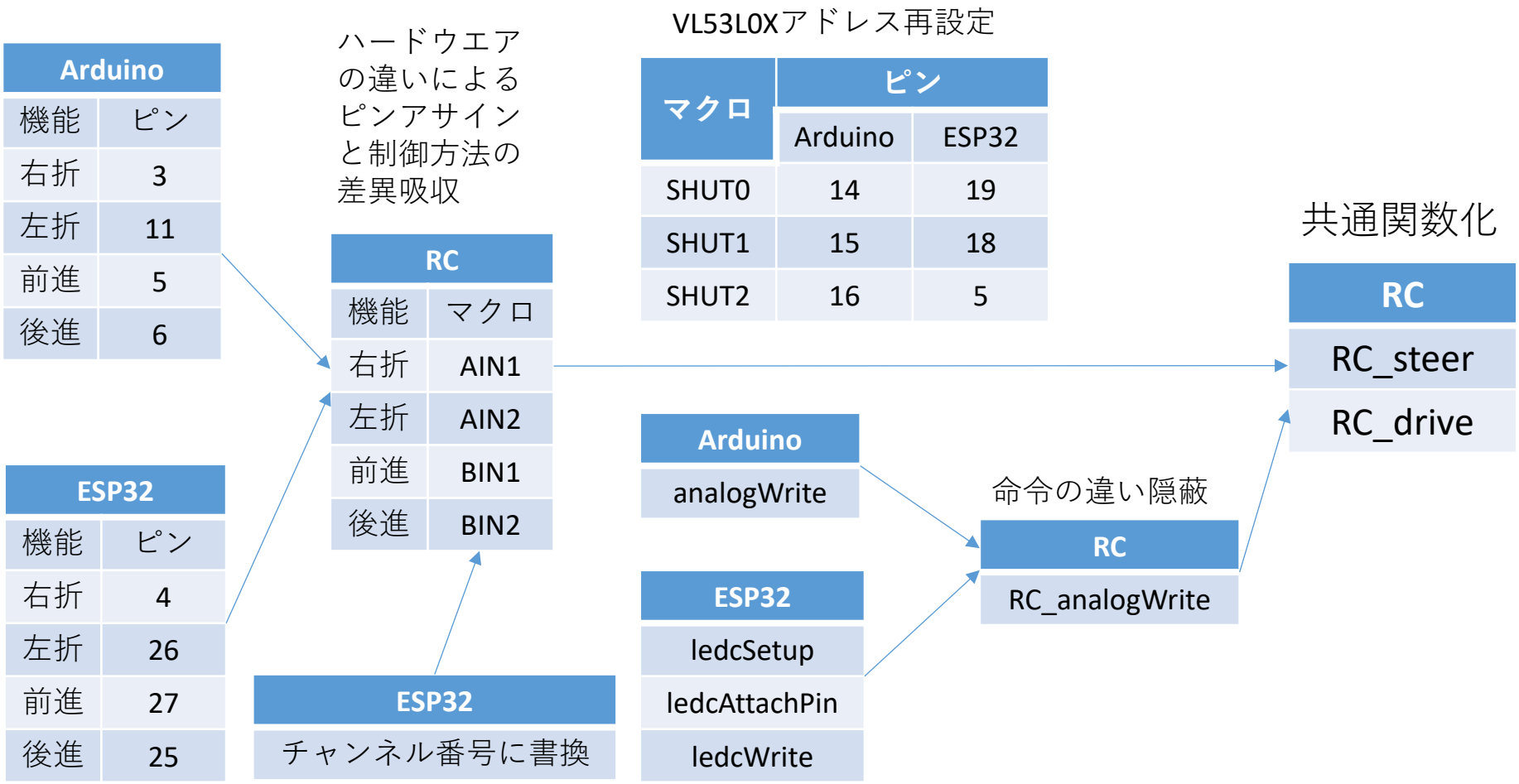
動作	関数の型	関数名	第一引数 (int)	測るセンサ	戻り値（正常）	エラー時の戻り値
測距	int	RC_read	LEFT	左 (Sensor0)	距離 (mm) 0～2000	-1：タイムアウト -2：引数エラー -3：範囲外
			CENTER	中央 (Sensor1)		
			RIGHT	右 (Sensor2)		
			REAR	後部 (Sensor3)		

- 負の値はエラー：-1応答なし（後部センサの無い車でREARを指定した場合も）、-2引数の誤り、-3範囲外・信号が弱い（readRangeSingleMillimeters()で8190前後が出る場合）
- 例：i = RC_read(CENTER);はsensor1.readRangeSingleMillimeters()と同じ距離を返します（Exerciseでは後者を使っています）
- 後部センサ（Sensor3）は搭載車のみ。ライブラリが起動時に自動で検出します

関数を用意することの意義

- RumiCarではマクロ設定や独自関数としてRC関数を追加するなどにより下記のメリットを提供します
  - 複数の命令をまとめているのでプログラムがすっきりし可読性が向上する
  - 固定の引数の記載を省くことによりプログラムがすっきりし可読性が向上する
  - 不要な記載を省くことによるプログラムミス低減
  - ハードウエアの差異によるピンアサインや制御方法等の考慮の不要化
  - コンピュータモジュール間の差異を吸収しプログラムの共通化

共通化ストラクチャ

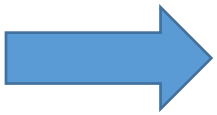

Arduino
機能 ピン
右折 3
左折 11
前進 5
後進 6
ESP32
機能 ピン
右折 4
左折 26
前進 27
後進 25
ハードウエアの違いによるピンアサインと制御方法の差異吸収
RC
機能 マクロ
右折 AIN1
左折 AIN2
前進 BIN1
後進 BIN2
ESP32
チャンネル番号に書換
VL53L0Xアドレス再設定
マクロ ピン
Arduino ESP32
SHUT0 14 19
SHUT1 15 18
SHUT2 16 5
Arduino
analogWrite
ESP32
ledcSetup
ledcAttachPin
ledcWrite
命令の違い隠蔽
RC
RC_analogWrite
共通関数化
RC
RC_steer
RC_drive

RC_steer関数の利用例

- RC_steer関数を利用した操舵プログラミングの例です
- 左側にオリジナル命令での記述を示します

オリジナル命令での記述

```
 8  void loop()
 9  {
10    analogWrite(AIN1,255);
11    analogWrite(AIN2,0);
12    delay(500);
13    analogWrite(AIN1,0);
14    analogWrite(AIN2,255);
15    delay(500);
16  }
```



RC関数利用の記述

```
 8  void loop()
 9  {
10    RC_steer(RIGHT);
11    delay(500);
12    RC_steer(LEFT);
13    delay(500);
14  }
```

Exercise-2

モータ制御

Exercise-2.1

ハンドルを切る

Exercise-2.1
ハンドルを切る(準備)

- RumiCarの車体の電源をオフにします
- CMをRumiCarの車体より取り外してください
- CMにUSBケーブルを接続します
- USBケーブルをパソコンに接続します
- スケッチ例から「Exercise-2.1」を開きます

Exercise-2.1
ハンドルを切る（プログラム）

```
 8  void loop()
 9  {
10    RC_steer(RIGHT);
11    delay(500);
12    RC_steer(LEFT);
13    delay(500);
14  }
```

- delay命令はそのまま何もしないで指定時間だけ待つよ
- delayの引数の単位はミリ秒だよ
- このプログラムはハンドルを切ったまま0.5秒(500ミリ秒)そのままハンドルを保持するよ

- ハンドルを右に切る
- 0.5秒そのまま
- ハンドルを左に切る
- 0.5秒そのまま

繰り返す

Exercise-2.1
ハンドルを切る(試験)

- コンパイルしてプログラムをCMに書き込みます
- CMからUSBケーブルを取り外します
- CMをRumiCarの車体に取り付けます
- RumiCar車体の電源をオンにしてください
- ハンドルが左右に曲がっていますか？

Exercise-2.2

速度制御

Exercise-2.2
速度制御(準備)

- RumiCarの車体の電源をオフにします
- CMをRumiCarの車体より取り外してください
- CMにUSBケーブルを接続します
- USBケーブルをパソコンに接続します
- スケッチ例から「Exercise-2.2」を開きます

Exercise-2.2
速度制御(プログラム)

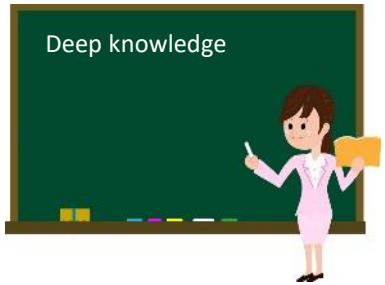
```
 8  void loop()
 9  {
10    RC_drive(FORWARD,255);
11    delay(500);
12    RC_drive(FORWARD,200);
13    delay(500);
14    RC_drive(FORWARD,150);
15    delay(500);
16  }
```

- RC_driveの第二引数、255とか200とか150はPWMの値です
- 255が最大で128が最大値の約半分の値になります
- 値を下げてゆくとタイヤの回転もだんだんと遅くなります

Exercise-2.2
速度制御(実走試験)

- CMからUSBケーブルを取り外します
- CMをRumiCarの車体に取り付けます
- まず、いきなり走らせないでRumiCarの車体を手で持ち上げたままにします
- RumiCar車体の電源をオンにしてください
- タイヤの回転が変化していますか？

- RumiCarをそっと机に置いてみましょう

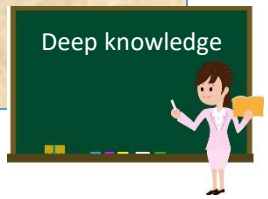


Deep knowledge

直流モータの低回転制御

稲玉　繁樹 氏

RumiCarのモータ制御



- RumiCarは２つのモータを制御しています。
  - ステアリング

```
//操舵の関数
int RC_steer (int direc ){
```

```
RC_analogWrite(AIN1,255);
RC_analogWrite(AIN2,0);
```

オンオフ制御
全力でモータをオンしている

・角度が決められない
・電流が流れすぎている

  - 走行

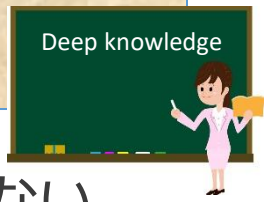
```
//走行の関数
int RC_drive(int direc, int ipwm){
```

```
RC_analogWrite(BIN1,0);
RC_analogWrite(BIN2,ipwm);
```

PWM制御
電圧を変えている

車両条件などで速度安定しない
速度と電圧が比例しない

RumiCarのモータ制御



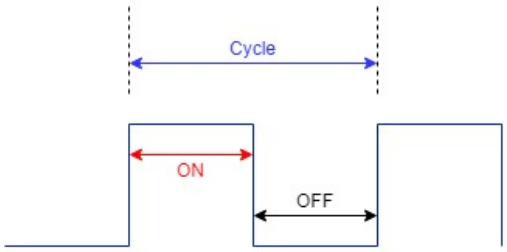
- 低速で動かしたいが，走行指令 120/255 以下は動かない
  - 原因１：PWM周期が高すぎる
  - 原因２：DCモータは始動時に大きい電流が必要

    $Z=j\omega L$ ・・・(1)　$\omega=2\pi f$ ・・・(2)

    ≒ 見かけの抵抗が高くなりPWM指令が低いと電流が小さく始動できない

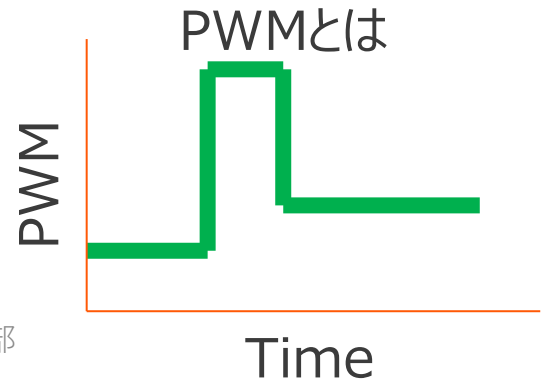
    よって，使えるPWM範囲が狭くなっている。



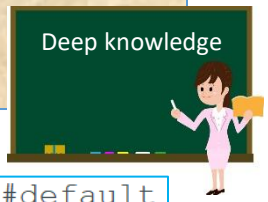


Duty = ON / Cycle
VV = Duty
VF = Cycle

  - 対策１：PWM周期を変更
    - 976kHz → 244Hz
  - 対策２：始動ブーストを導入
    - 始動時の数10ms はPWM大きく



Advanced ソフトの解説



- A2-3 低速で走らせるため

```
//  TCCR0B |= B00000011;                    // 64分周=  16M/  64/256=976Hz  #default
  TCCR0B |= B00000100;                    // 256分周= 16M/ 256/256=244Hz
```

```
//   始動後に低速へ
   RC_drive(FORWARD,200);      RC_delay(20);
   RC_drive(FORWARD,64);       RC_delay(500);
```

PWM周期を下げる
→ 低い値でも電流が流れる
始動時強く，動き出したら弱く
→ 40位まで動くようになりました

- B2-3 低速動作をライブラリへ

```
// Private Variable
#define    UP_SPEED       (200)          // StartUP speed
#define    UP_TIME        (3)            // 30[ms]
```

```
RC_drive(FORWARD,40);       RC_delay(1000);
RC_drive(BRAKE,255);        RC_delay(500);
RC_drive(REVERSE,40);       RC_delay(1000);
RC_drive(BRAKE,255);        RC_delay(500);
```

PWMドライバを定時割込化
始動時ブーストを自動化
→ 意識せずに低速動作が可能

- B2-5 縦列駐車にチャレンジ

動画で確認！

注：上級編はPWM周期を変えるため
RC_delay()を使うこと

ライブラリでの走行モータ制御

- RumiCarライブラリ v0.2.0 以降の RC_drive
  - 走行モータをスローデケイ（駆動とブレーキの切り替え）で制御し、低いPWM値でも走り出しやすくしています
  - 止まっている状態や逆向きから走り出す瞬間だけ、短く強く駆動する「始動キック」を自動で入れます
  - Timer0（PWM周期）は変更しないので、delay()はそのまま正確に使えます
  - RC_drive(BRAKE, …)は第二引数に関係なく確実に停止します

- 上級編（Exercise-A2.3、A3.1、B2.3～B2.5）
  - ライブラリを使わない独自のプログラムです。前のページの説明どおりRC_delay()を使います

Exercise-2.3

前進と後進

Exercise-2.3
前進と後進(準備)

- RumiCarの車体の電源をオフにします
- CMをRumiCarの車体より取り外してください
- CMにUSBケーブルを接続します
- USBケーブルをパソコンに接続します
- スケッチ例から「Exercise-2.3」を開きます

Exercise-2.3
前進と後進(プログラム)

- RC_driveの第一引数がFORWARD を指定した時に前進
- 第一引数がREVERSEの時に後進

```
 8  void loop()
 9  {
10    RC_drive(FORWARD,255);
11    delay(500);
12    RC_drive(REVERSE,255);
13    delay(500);
14  }
```

- 前進する
- 0.5秒そのまま（前進し続ける）
- 後進する
- 0.5秒そのまま（後進し続ける）

繰り返す

Exercise-2.3
前進と後進(実走試験)

- コンパイルしプログラムをCMへ書きこみます
- CMからUSBケーブルを取り外します
- CMをRumiCarの車体に取り付けます
- まず、いきなり走らせないでRumiCarの車体を手で持ち上げたままにします
- RumiCar車体の電源をオンにしてください
- タイヤが前進と後進を繰り返してますか？

- RumiCarをそっと机に置いてみましょう

Exercise-2.3
前進と後進(発展問題)

- （発展問題）なるべくスリップしないようにして前進と後進を繰り返すようにするにはどうしたら良いでしょうか？

Exercise-2.4

ジグザグ走行

Exercise-2.4
ジグザグ走行（準備）

- RumiCarの車体の電源をオフにします
- CMをRumiCarの車体より取り外してください
- CMにUSBケーブルを接続します
- USBケーブルをパソコンに接続します
- スケッチ例から「Exercise-2.4」を開きます

Exercise-2.4
ジグザグ走行(プログラム)

```
 8  void loop()
 9  {
10    RC_drive(FORWARD,255);
11    RC_steer(RIGHT);
12    delay(500);
13    RC_steer(LEFT);
14    delay(500);
15  }
```

- 前進する
- ハンドルを右に切る
- 0.5秒そのまま（右に進み続ける）
- ハンドルを左にきる
- 0.5秒そのまま（左に進み続ける）

繰り返す

Exercise-2.4
ジグザグ走行(実走試験)

- コンパイルしプログラムをCMへ書きこみます
- CMからUSBケーブルを取り外します
- CMをRumiCarの車体に取り付けます
- まず、いきなり走らせないでRumiCarの車体を手で持ち上げたままにします
- RumiCar車体の電源をオンにしてください
- RumiCarは予想通りの動きをしていますか？

- RumiCarをそっと机に置いてみましょう

自律走行基礎

センサによる車体制御

自律走行基礎

- 自律走行はセンサ等で得られた情報や、既に自分が持っている情報など、またそれらを組み合わせて自動車を自動で最適に運転させることです
- この章では自律走行の基礎として簡単な例を考えてみます

自律走行基礎(状況の仮定)

- 次のような単純な自律走行の状況を仮定します
  - 前進走行中に前方に障害物を検知したら障害物に衝突しないように安全に停車する
  - 障害物を検知して停車中に障害物がなくなったら再度前進を再開する

Exercise-3

自律走行プログラムを開発しよう！

Exercise-3.1

安全に停止する車

Exercise-3.1
安全に停止する車

- 単純な条件として具体的に下記を仮定します
  - RumiCarの中央センサ(Sensor1)で障害物を検知しながら、障害物が検知されない場合は前進します
  - 障害物の例えば10cm手前で停車、障害物がなくなったら再度前進再開します

Exercise-3.1
RumiCarでの開発にあたっての考慮事項

- 距離の測定誤差
  - サンプルプログラムのVL53L0X設定である高速測定モードでは高速に測定するため若干精度が犠牲になります。そのため測距時に1~2cmの誤差が発生します。余裕を見込んだ距離判定をしましょう

- 測定不能時の処理
  - 測定結果で8190前後の値はレーザーの反射が得られない場合の値です。つまりは測定範囲内に障害物を検知できなかった場合の値になります
  - （RC_read関数を使った場合は-3が返ります）

Exercise-3.1
RumiCarでの開発にあたっての考慮事項(続き)

- 地面等の検知
  - センサのレーザーは円錐形に照射されます。RumiCarは小さいので車上のセンサ搭載位置が低く円錐形に照射されたレーザーはいずれ距離測定限界より先に地面や床等を捉えてしまいます。この距離はRumiCarモデル32型でだいたい30cm前後です。これらより障害物としての検知距離は30cm以下としてください

- 急停止処理
  - 走行状態から停止や後進命令を出しても慣性（勢い）が付いているため直ぐには停止できません。できれば上手な減速や停止の方法を考慮しましょう

Exercise-3.1
プログラミング上の条件まとめ

項目	条件
使用センサ	中央センサ(Sensor1)のみ使用する
挙動	設定距離内に障害物を検知したら停止する
設定距離	10cmとする。ただし誤差を考え実範囲は10±2cmとする
信頼距離	30cm以内とする
測定不能時の値	8190、但し誤差を含むことがあるので注意（RC_readでは-3）

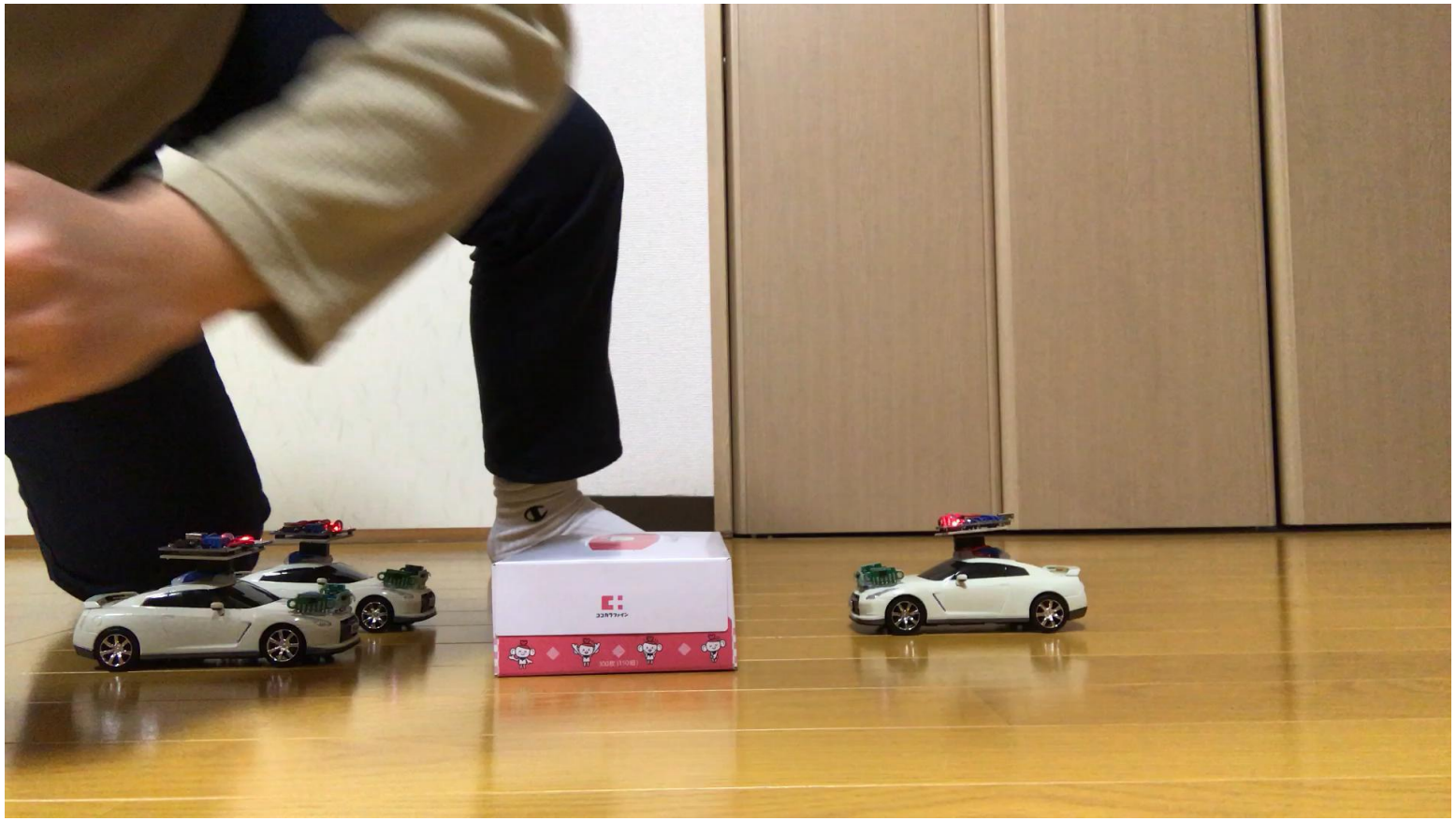
Exercise-3.1
開発時のちょっと一工夫

- 判断条件を単純に障害物がない、とのロジックとしてしまうと前方に障害物がない場合はRumiCarがどこまででも走っていってしまいます。これだとうっかり机の上などにRumiCarをおいてしまうとRumiCarが机から飛び降りてしまいます。机の上で動作テストなどをする可能性がある場合においては安全も考えて例えば30cm以内に障害物がある場合のみ走るなどと条件を追加するなど工夫すると良いでしょう。開発時にRumiCarをうっかり破損させてしまう危険性を低減できます

Exercise-3.1 https://youtu.be/95pc_4Wf14U
安全に停止する車（動画）



Exercise-3.1 https://youtu.be/95pc_4Wf14U
安全に停止する車（動画）



Exercise-3.1
安全に停止する車(準備)

- RumiCarの車体の電源をオフにします
- CMをRumiCarの車体より取り外してください
- CMにUSBケーブルを接続します
- USBケーブルをパソコンに接続します
- スケッチ例から「Exercise-3.1」を開きます

Exercise-3.1
安全に停止する車(プログラム)

```
 8  void loop()
 9  {
10
11    int ispeed = 255;
12    int idist1;
13    idist1=sensor1.readRangeSingleMillimeters();
14    Serial.println(idist1);
15    if ( idist1 < 300 ){
16      if ( idist1 > 120 ){
17        RC_drive(FORWARD,ispeed);
18      }else if (idist1 < 80){
19        RC_drive(REVERSE,ispeed);;
20      }else{
21        RC_drive(BRAKE,ispeed);
22      }
23    }else{
24      RC_drive(FREE,ispeed);
25    }
26  }
```

- 障害物が30cm以内にあるか？
- （無い場合は何もしない）
- （ある場合は次を実行）
  - 障害物が12cmを超える->前進
  - 障害物が8cm未満->後進
  - 8cm以上12cm以下->ブレーキ

繰り返す

Exercise-3.1
安全に停止する車(実走試験)

- コンパイルしプログラムをCMへ書きこみます
- CMからUSBケーブルを取り外します
- CMをRumiCarの車体に取り付けます
- RumiCar車体の電源をオンにしてください
- 手を中央センサーに近づけたり話したりしてみましょう。RumiCarは予想通りの動きをしていますか？

- RumiCarをそっと机に置いてみましょう

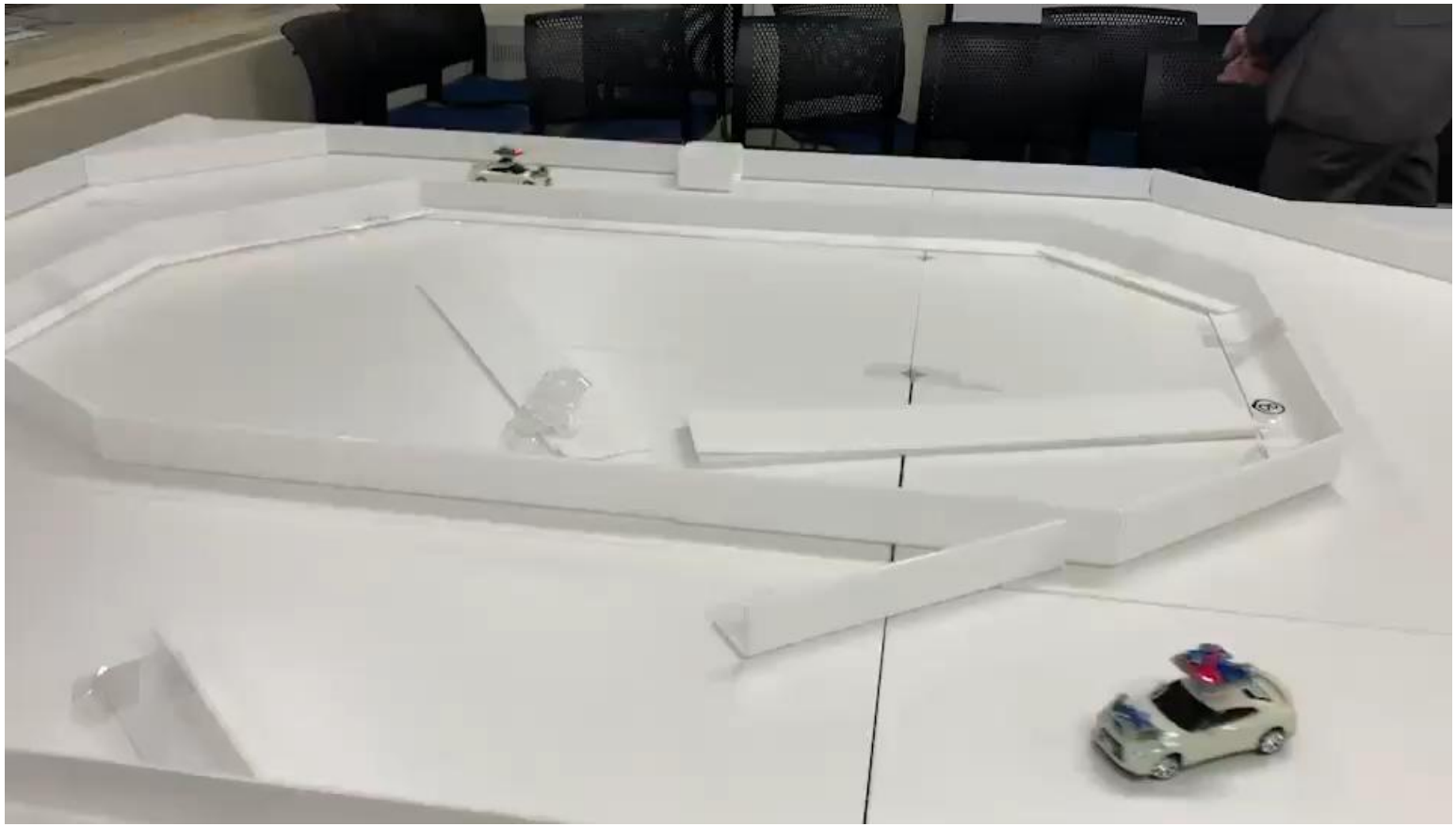
Exercise-3.2

市街地を走る車

Exercise-3.2
市街地を走る車

- 単純な条件として具体的に下記を仮定します
  - 市街地に見立てた、道路の両側に壁がある道路を走ります
  - 両側の壁の距離を測って道路の中心を走ります
  - 先行車の例えば10cm手前で停車、先行車がなくなったら再度前進再開します

Exercise-3.2 https://youtu.be/jyq4Ph0mCDM
市街地を走行する車（動画）



Exercise-3.2
市街地を走行する車(準備)

- RumiCarの車体の電源をオフにします
- CMをRumiCarの車体より取り外してください
- CMにUSBケーブルを接続します
- USBケーブルをパソコンに接続します
- スケッチ例から「Exercise-3.2」を開きます

Exercise-3.2
市街地を走行する車(プログラム)

```
 8  void loop()
 9  {
10
11    int ibound =250;
12    int s0, s1, s2;
13    s0=sensor0.readRangeSingleMillimeters();
14    s1=sensor1.readRangeSingleMillimeters();
15    s2=sensor2.readRangeSingleMillimeters();
16
17
18    Serial.print("Sensor0:");
19    Serial.print(s0);
20    Serial.print("  Sensor1:");
21    Serial.print(s1);
22    Serial.print("  Sensor2:");
23    Serial.println(s2);
24
25    if(s1<100){
26      RC_drive(REVERSE,150);
27    }else if (s1<150){
28      RC_drive(FORWARD,150);
29    }else if (s1<250){
30      RC_drive(FORWARD,200);
31    }else{
32      RC_drive(FORWARD,255);
33    }
34    if(s0>s2){
35      RC_steer(LEFT);
36    }else{
37      RC_steer(RIGHT);
38    }
39  }
```

- 速度のパラメータ（150や200、255）を変更してみよう！
- 距離のパラメータ（100や150、250）を変更してみよう！
- どうなるかな？？

- 障害物が10cm未満->ゆっくり後進
- 障害物が15cm未満->ゆっくり前進
- 障害物が25cm未満->ちょっと速めに前進
- 上記以外は全速前進

- 左右の壁の距離を比較して
- 壁から近い方から離れる（壁から遠い方へハンドルを切る）

Exercise-3.2
市街地を走行する車(実走試験)

- コンパイルしプログラムをCMへ書きこみます
- CMからUSBケーブルを取り外します
- CMをRumiCarの車体に取り付けます
- RumiCar車体の電源をオンにしてください

- RumiCarをそっとコースに置いてみましょう

Exercise-4

AIと一緒にプログラムを改良しよう！

AIペアプログラミングとは

- やりたいことを日本語で（声でも）伝えると、AIがプログラムを書き換えてくれます
  - 方針を考えるのは人、書くのはAI、確かめるのは人
- この章では、これまでのExerciseをAIと一緒に改良します
  - Exercise-4.1　スリップしない前進と後進（2.3の発展）
  - Exercise-4.2　はまったら切り返す車（3.2の発展）
- 道具はGoogleの「Antigravity CLI」（コマンド名 agy、無料プラン可）
  - 13歳以上は自分のGoogleアカウント。小学生は保護者と一緒に
- 学ぶのは「頼み方」と「確かめ方」。AIの答えをそのまま信じない

事前準備の確認（宿題）

確認すること	確認のしかた
RumiCarライブラリが 0.5.0 以上	「ライブラリを管理…」で RumiCar を検索 → 版の表示。古ければ「更新」
スケッチ例に Exercise-4.1〜4.2-Answer がある	「ファイル」→「スケッチ例」→「RumiCar」
Antigravity CLI が入っている	ターミナルで agy --version と入力して版が出る
Google アカウントでサインイン済み	agy を起動して質問に答えられる（会社・学校のアカウントは不可。個人の Gmail で）
音声入力が使える	agy で /voice と入力し、一言話して Enter → 文字が出る（出ないときは「声で頼む」のページ）
授業の週はクォータを温存	無料プランは週ごとの上限あり。前の週に使い切らない

できていない人は手を挙げてください。講師のPCで代わりに改良して、プログラムをお渡しします

進め方（4.1・4.2 共通）

順番	すること	どうやって
1	スケッチ例を開く	「ファイル」→「スケッチ例」→「RumiCar」→「Exercise-4.x」
2	名前を付けて保存	「ファイル」→「名前を付けて保存」→ そのまま保存
3	フォルダを開く	「スケッチ」→「スケッチのフォルダを表示」→ 右クリック →「ターミナルで開く」
4	agy を起動	ターミナルに agy と入力。初回は「信頼する」に はい
5	声か文字で頼む	/voice で話す（または入力）→ 文字を確かめて Enter。例：「加減速をもう少しスムースにして」
6	差分を読んで承認	Ctrl+R で変更点を表示 → 内容が分かったら Y
7	IDE で書き込み	IDE の画面が自動で新しくなる → いつも通り「書き込み」
8	持ち上げて確認	車体を持ち上げたまま電源オン → 動きを見てから床へ

声で頼む（/voice）

順番	すること
1	agy の入力欄に /voice と入力して Enter。表示が「Recording」になるまで待つ
2	マイクに向かって、頼みたいことを話す（例：「加減速をもう少しスムースにして」）
3	Enter で録音を止める → 話した内容が入力欄に文字で入る（まだ送られない）
4	文字を読み、間違いはキーボードで直す（Esc＝録音をやめる、Ctrl+Z＝取り消す）
5	Enter で AI に送る

- 声で入れた文字は自動では送られません。送る前に必ず読む（聞き間違い防止）
- 関数名や数字（RC_drive、255 など）は聞き間違えやすい → キーボードで直す
- 周りの声も拾うので、マイクに近づいて話す（ヘッドセットがあれば使う）。録音は1回最大3分30秒
- F5 キーでも始められます（F5 が効かないときは /voice）
- 始まらないとき：マイクを許可（Windows：設定 › プライバシーとセキュリティ › マイク で「デスクトップ アプリがマイクにアクセスできるようにする」をオン／Mac：システム設定 › プライバシーとセキュリティ › マイク でターミナルをオン）→ だめなら /logout してサインインし直す

AIを使うときの約束

- 意味が分からない変更は承認しない（「もっと簡単に」「説明して」と頼む）
- 変えてよいのは .ino だけ。AIへのお約束は隣のタブ AGENTS.md（これは変えない）
- 書き込みは自分で。AIには車への書き込みをさせません
- 動かすときは、まず車体を手で持ち上げたまま
- 元のスケッチ例は残っているので、壊しても開き直せます
- やり取りはAIの改善に使われる設定が既定。個人情報は書かない
- うまくいかなければ頼み方を変える → だめなら講師を呼ぶ

AGENTS.md とは

- Exercise-4.1 と 4.2 のスケッチ例に入っているファイル
- AIは作業フォルダのこのファイルを自動で読みます
- IDE では .ino の隣のタブに見えます（コンパイルには使われません）
- 「名前を付けて保存」すると一緒にコピーされます
- 中身を変える必要はありません

AGENTS.md に書いてあること
RumiCar の関数（RC_setup / RC_steer / RC_drive / RC_read）だけを使う
変えてよいのは .ino だけ。新しいファイルは作らない
後退には時間か回数の上限。止まらない繰り返しを作らない
PWM は 0〜255、delay は数秒以内
変更したら arduino-cli でコンパイル確認（端子別の設定つき）
車への書き込みはしない。返事は日本語で短く

Exercise-4.1
スリップしない前進と後進(準備)

- Exercise-2.3 の発展問題を、今度はAIと一緒に解きます
- RumiCarの車体の電源をオフにします
- CMをRumiCarの車体より取り外し、USBケーブルでパソコンにつなぎます
- スケッチ例から「Exercise-4.1」を開きます（中身は Exercise-2.3 と同じ）
- 「名前を付けて保存」で保存します（スケッチ例のままではAIの変更がIDEに映りません）
- 「スケッチ」→「スケッチのフォルダを表示」→ ターミナルで開く → agy

Exercise-4.1
AIに頼む（頼み方で結果が変わる）

頼み方 その1
「スリップしないようにして」
→ 何をどう変えるかはAI任せ。結果を説明できない

頼み方 その2（方針つき）
「前進と後進を、いきなり全速にせず少しずつ速くして、止まる前に少しずつ遅くしてください。向きを変える前に一度ブレーキで止めてください。RC_drive と delay だけで書いて」
→ 何を・どのくらい・何を変えないか が伝わる

- まず その1 で頼んで動かし、次に その2 で頼み直して、差を見比べましょう
- **頼むときの3点セット：**何を（加速・減速）／どのくらい（例：1秒かけて）／何を変えないか（関数は RC_drive だけ）
- 返事が長いときは「3行で説明して」。分からない言葉は「小学生にも分かるように」

Exercise-4.1
差分を読んでから承認する

確認すること	見るところ
速さの値が 0〜255 の範囲か	RC_drive(…, 速さ) の2つ目の数
待ち時間が長すぎないか	delay(…) の数。数千（数秒）を超えていたら理由を聞く
向きを変える前に止まっているか	FORWARD と REVERSE の間に BRAKE があるか
setup() と loop() が残っているか	全体の形。RC_setup() が消えていないか
知らない関数が増えていないか	RumiCar の関数（RC_〜）以外が出てきたら質問する

Ctrl+R で差分（変わった行）が出ます。納得できたら Y、できなければ N で「〜を直して」と頼み直す

Exercise-4.1
実走試験と次の依頼

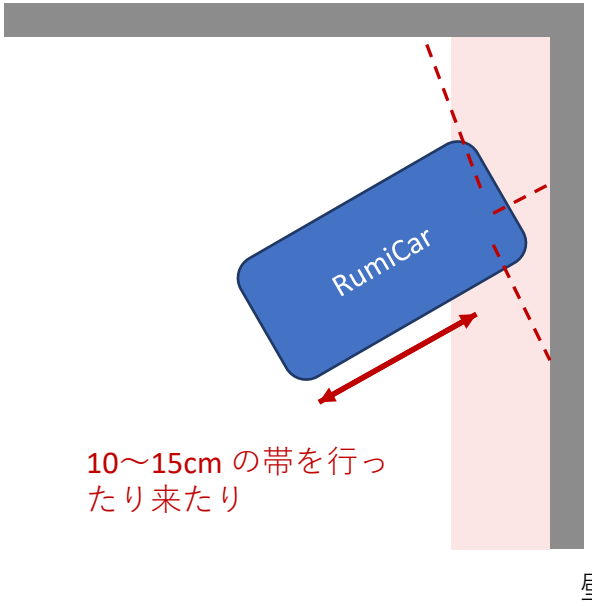
- IDEにAIの変更が映っていることを確認し、コンパイルしてCMへ書き込みます
- CMからUSBケーブルを外し、車体に取り付けます
- まず車体を手で持ち上げたまま電源をオンにします
- タイヤが、ゆっくり速く → ゆっくり遅く になっていますか？
- そっと床に置いて、スリップが減ったか見てみましょう
- もっと良くするなら、もう一度 agy で頼みます
  - 「止まるときも、もう少しゆっくり」「全速で走る時間を長く」

Exercise-4.1-Answer
解答例（スケッチ例にあります）

```
21  void rampDrive(int direc)
22  {
23    for (int pwm = MIN_PWM; pwm <= MAX_PWM; pwm += STEP_PWM) {    // 加速
24      RC_drive(direc, pwm);
25      delay(STEP_MS);
26    }
27    RC_drive(direc, MAX_PWM);
28    delay(HOLD_MS);
29    for (int pwm = MAX_PWM; pwm >= MIN_PWM; pwm -= STEP_PWM) {    // 減速
30      RC_drive(direc, pwm);
31      delay(STEP_MS);
32    }
33    RC_drive(BRAKE, 0);                                          // 止まってから向きを変える
34    delay(STOP_MS);
35  }
```

- loop() は rampDrive(FORWARD); rampDrive(REVERSE); の2行。速さを STEP_PWM（15）ずつ STEP_MS（40ms）ごとに変え、最高速で HOLD_MS 走り、止まる前に減速、向きを変える前に BRAKE。MIN_PWM（70）より遅いとモーターが回らない
- AIの答えと比べてみよう。違っていても、どちらも正解

Exercise-4.2
はまったら切り返す車



- Exercise-3.2 の車は、カーブで外側の壁に斜めに当たると…
- 前が 10cm を切る → 後進
- 10cm を超える → すぐ前進 → また当たる
- 前後に小刻みに震えるだけで、抜け出せません
- これを「はまった」と呼ぶことにします
- **今日の課題：はまっても自力で抜け出す車にする**

Exercise-4.2
なぜはまるのか

- 3.2 の判断は「前の距離」と「左右どちらが遠いか」だけ
  - 前 < 10cm → 後進、< 15cm → ゆっくり前進、それ以上 → 速く前進
  - ハンドルは「遠い側」へ切ったまま
- 壁に斜めに当たると、後進しても向きがほとんど変わらない
  - 同じ角度で同じ場所にまた当たる → 10〜15cm の帯を往復
  - 切り替えのたびに始動キックが入るので「震える」ように見える
- **足りないもの：**①はまったと気づく仕組み　②向きを変える動作

考えてみよう①
はまったと、どう気づく？

使えるもの（RumiCar の標準の車体）

- 前・左・右の距離センサー（ToF）3本の値
- 自分がいまモーターに何を命じているか（前進／後進、いつ切り替えたか）
- 時計（millis()：起動してからの時間）

使えないもの

- タイヤが何回転したか（回転計がない）、傾きや加速度（センサーがない）、カメラ

この材料だけで「はまっている」と判断する方法を、隣の人と2分で考えてみましょう

ヒント：はまっているとき、センサーの値と自分の命令は、どんな様子になっている？

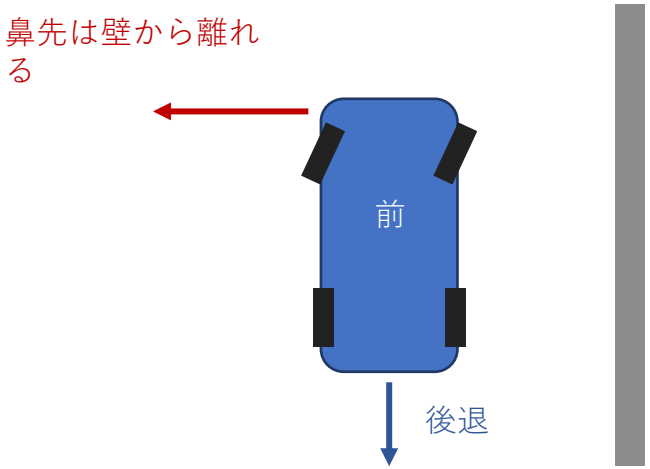
はまりを知る3つの方法

方法	考え方	良いところ	弱いところ
① 前後の切り替えを数える	3秒のあいだに前進⇄後進が4回以上	症状そのもの。変数2つで書ける	押し付けられて空転（切り替えなし）は見つけられない
② 距離が変わらない	前進や後進を命じているのに、前の距離が1秒で20mm未満しか変わらない、が2回続く	①が見逃す「空転」を見つけられる	検知に約3秒。前が遠いときは判定しない
③ 三方が近い	前・左・右がすべて12cm未満	一瞬で分かる。if文1つ	狭い所で誤検知。しきい値はコース幅の半分未満に

どれか1つでも当てはまれば「はまり」。数値は目安で、自分のコースで決めるのも課題。脱出後2秒は再判定しない

考えてみよう②
どうやって抜け出す？ ＝ 切り返し

後退するとき：近い壁の側にハンドル



右に切って下がると、鼻先は左（壁から離れる向き）に振れる

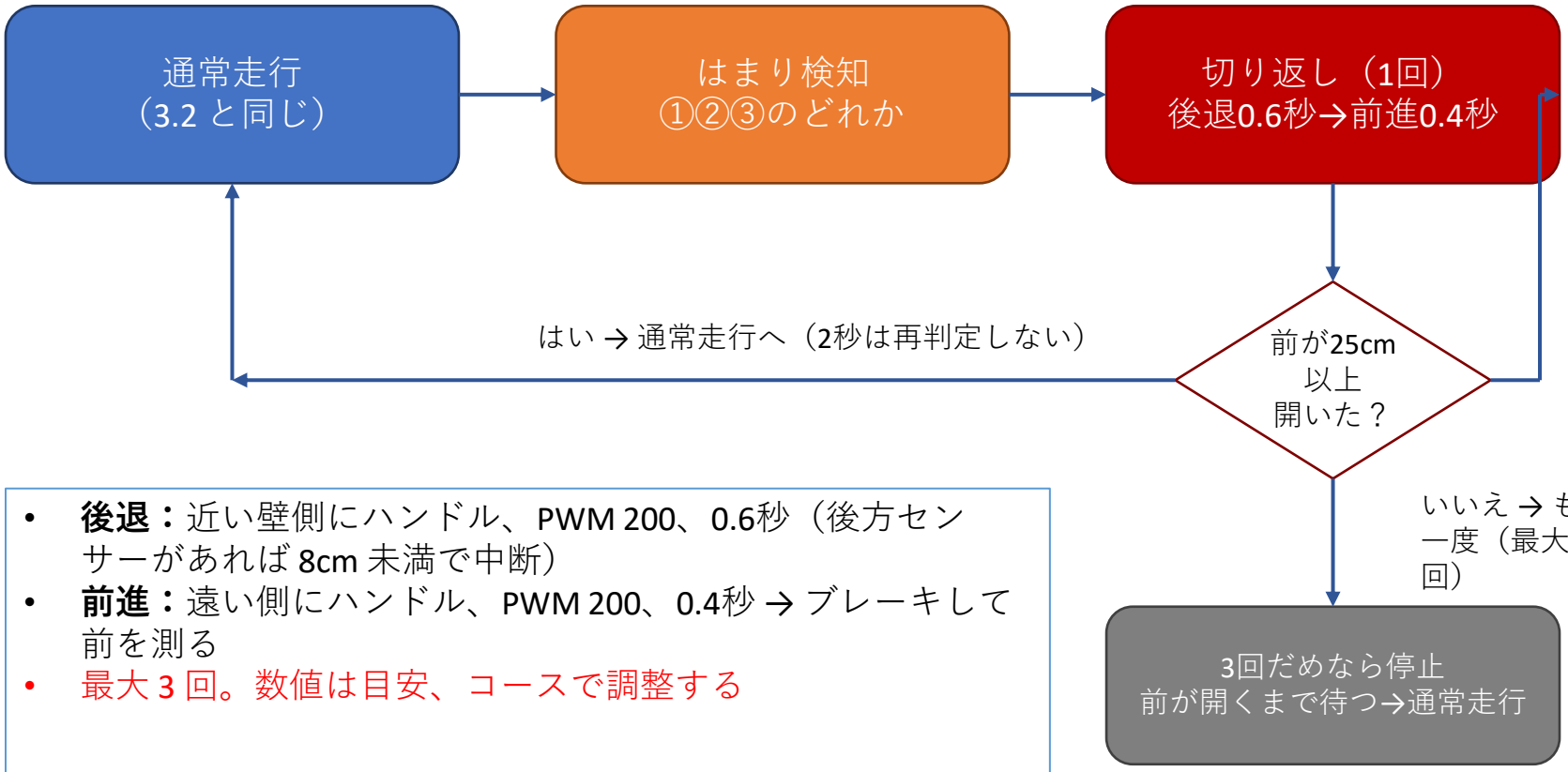
前進するとき：遠い側にハンドル



左に切って進むと、壁から離れていく

- 3.2 が抜け出せなかったのは、後退のときも「遠い側」にハンドルを切っていたから
- コツ：後退は近い壁側、前進は遠い側。これが「切り返し」（車の運転と同じ）

脱出の手順
状態の流れと数値



- **後退：**近い壁側にハンドル、PWM 200、0.6秒（後方センサーがあれば 8cm 未満で中断）
- **前進：**遠い側にハンドル、PWM 200、0.4秒 → ブレーキして前を測る
- 最大 3 回。数値は目安、コースで調整する

Exercise-4.2
はまったら切り返す車(準備)

- RumiCarの車体の電源をオフにします
- CMをRumiCarの車体より取り外し、USBケーブルでパソコンにつなぎます
- スケッチ例から「Exercise-4.2」を開きます（中身は Exercise-3.2 と同じ）
- 「名前を付けて保存」で保存します
- 「スケッチ」→「スケッチのフォルダを表示」→ ターミナルで開く → agy
- 考えてみよう①②で決めた方針を、メモしておきましょう（次のページで使います）

Exercise-4.2
AIに頼む（丸投げ と 方針つき）

丸投げ
「カーブではまらないようにして」
→ 何かは書いてくれる。でも、なぜ動くのか説明できない

- 方針つきのほうが自分のプログラムになります
- ①〜③を全部書かず、まず①だけで頼んで動かし、あとから②③を足してもOK
- 長い依頼は、メモ帳に書いてから貼り付けると楽

方針つき（自分で考えた方針を伝える）
「Exercise-4.2 に、はまったときの切り返しを足してください。はまった判定は (ア) 3秒以内に前後が4回切り替わった (イ) 前進中なのに前の距離が1秒で20mmも変わらない状態が2回続いた (ウ) 前・左・右が全部12cm未満、のどれか。脱出は、近い壁側にハンドルを切って0.6秒後進、反対側に切って0.4秒前進、前が25cm以上開くまで最大3回、だめなら停止。RumiCar.h の関数だけを使い、.ino 以外は変えないで」

Exercise-4.2
差分の確認観点

確認すること	なぜ大事か
終わらない繰り返しがないか。後退や切り返しに回数・時間の上限があるか	上限がないと、はまったまま後ろへ走り続ける
通常走行（3.2 の部分）に戻る条件があるか	元の走り方に戻らないと先へ進めない
速さが 0〜255、時間が数秒以内か	大きすぎると危ない／遅すぎるとモーターが回らない
後退中に後ろを見ているか（後方センサーがある車体）	後ろの壁や人にぶつからないため
3.2 の走り方そのものが壊れていないか	「はまらない」より「ちゃんと走る」が先
測れなかった値（-1 / -3 / 8190）の扱いがあるか	「遠い」を「近い」と勘違いすると後進し続ける

Exercise-4.2
はまったら切り返す車(実走試験)

- コンパイルしてCMへ書き込み、CMを車体に取り付けます
- まず車体を持ち上げたまま電源オン。前のセンサーを手でふさいで前後を4回切り替えさせる → 切り返しが始まる？
- コースに置き、わざとカーブの外側の壁に斜めに当ててみましょう
  - 抜け出せたら成功。だめなら、次を1つずつ頼みます
- 「判定をもう少し早く（3回で）」「後退の時間を0.8秒に」「切り返しを5回まで」
- シリアルモニタの STUCK / ESCAPED の表示で確かめましょう

Exercise-4.2-Answer
解答例の骨格

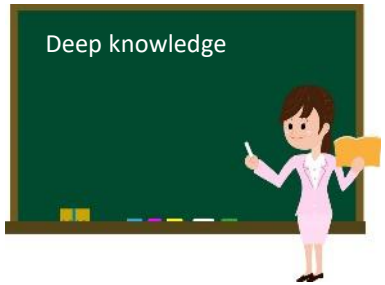
```
198  void loop()
199  {
200    int s0 = RC_read(LEFT);
201    int s1 = RC_read(CENTER);
202    int s2 = RC_read(RIGHT);
210    bool canJudge = (millis() >= cooldownEnd);    // 脱出の直後は判定しない
211    if (canJudge && (stuckByFlip()
212                     || stuckByNoProgress(s1)
213                     || stuckByThreeSides(s0, s1, s2))) {
215      escape(s0, s2);                             // 切り返し（最大 ESCAPE_TRIES 回）
216      return;
217    }
219    driveLikeExercise32(s0, s1, s2);              // 3.2 と同じ走り方
220  }
```

- stuckByFlip()＝①切り替え回数、stuckByNoProgress()＝②距離が変わらない、stuckByThreeSides()＝③三方が近い。escape()＝近い壁側に切って後退→遠い側に切って前進、最大3回
- 判断を関数に分けると1つずつ試せる。数値は先頭の const に。全文はスケッチ例 Exercise-4.2-Answer で（シリアル表示の行は省略）

Exercise-4 まとめ
人とAIの役割分担

だれが	なにを
人	何をしたいか決める。方針を考える（どう気づく？ どう抜ける？）
AI	方針をプログラムにする。コンパイルまで確かめる
人	差分を読む。分からなければ聞く。実機で走らせて確かめる。数値を調整する

- AIの提案を鵜呑みにしない。動いた理由を自分の言葉で言えたら合格
- うまく頼むコツ：何を・どのくらい・何を変えないか。数値は自分の車とコースで決める
- 家でも続きができます。AGENTS.md があるスケッチ例から始めましょう

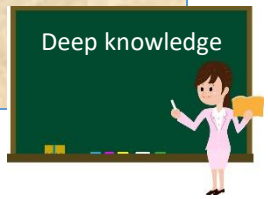


Deep knowledge

ラズパイサンプルプログラムに関して

石垣　翔子 氏

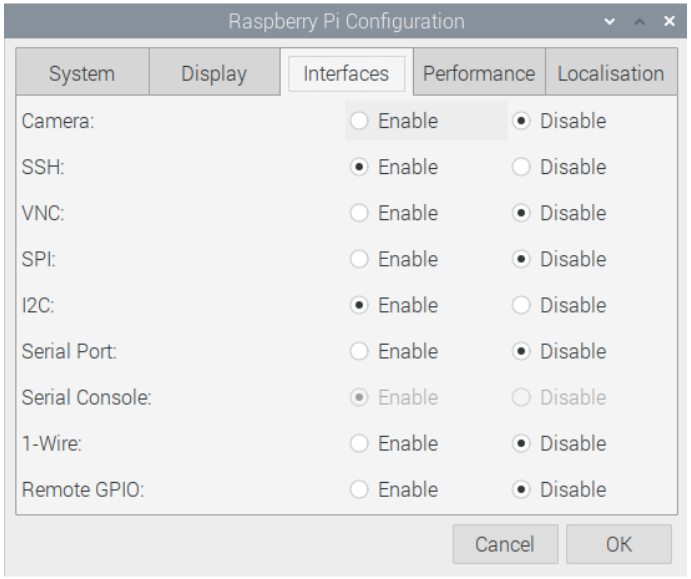
ラズパイサンプルプログラムに関して



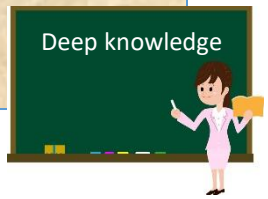
- PWM制御を含め全てのGPIO制御に
  pigpioライブラリを使用しています。
  プログラム実行前にpigpiodを起動し
  てください。

  起動コマンド：sudo pigpiod

- 測距モジュールはI2Cを使用するため、
  事前にI2Cを有効にしてください。



ラズパイサンプルプログラムに関して

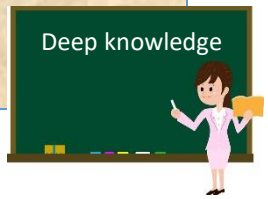


- ソフトウェアPWMとハードウェアPWM
  - ラズパイで独立して制御できるハードウェアPWMは２つ

    12/32番ピンと33/35番ピンがHardware PWMとして使用可能
  - ソフトウェアPWMは全てのGPIOピンで使用できます
  - 急停止など正確な制御を要する走行用モータにハードウェアPWMを

    使用します

ラズパイサンプルプログラムに関して



VL53L0Xのプログラム実行に必要なライブラリのインストール方法

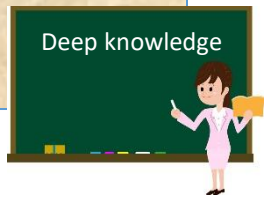
- GitHubに公開されているライブラリを使用します

1. git clone https://github.com/johnbryanmoore/VL53L0X_rasp_python
2. cloneしたVL53L0X_rasp_pythonフォルダ内でmakeを実行

注意点

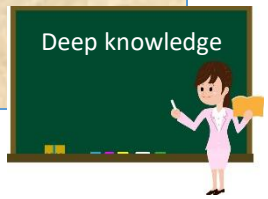
- サンプルプログラムは上記レポジトリが/home/pi/Documentsにcloneされていることを前提としています。必要に応じてVL53L0X.py内の共通ライブラリのパスを変更してください。
- VL53L0XのVccにラズパイの3.3V出力以外を使用する場合はラズパイのGNDピンをVL53L0Xと共通のGNDに接続してください

Azure Iot Hub



- IoTデバイス（ラズパイなど）とAzureの双方間通信
- 走行データの可視化や遠隔操作が可能に
- Azure Machine Learningなどの他のAzureサービスとの連携

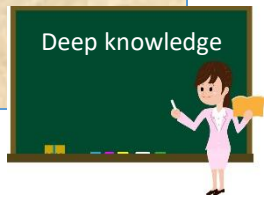
device-to-cloudメッセージ



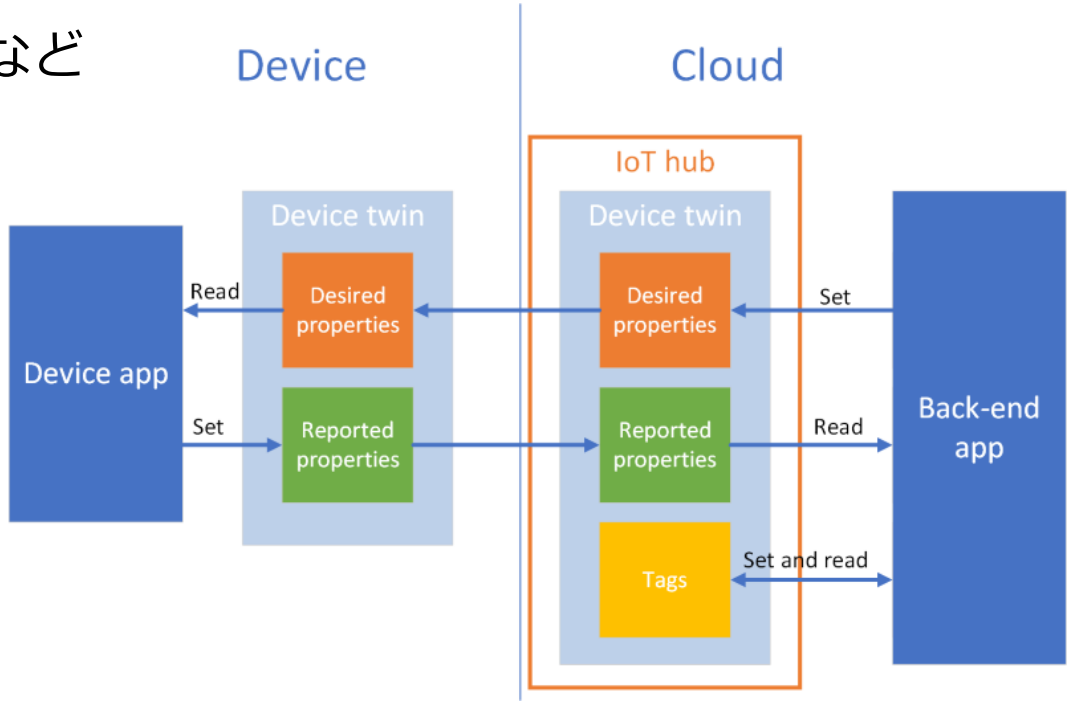
- センサーデータの送信と監視
- 最大256-KBのデータまで送信可能

```
ShokonoMacBook-Pro:~ $ az iot hub monitor-events --hub-name raspi-ishigaki --device-id raspi
Starting event monitor, filtering on device: raspi, use ctrl-c to stop...
{
    "event": {
        "origin": "raspi",
        "interface": "",
        "payload": "{\"distance\": 424, \"control\": FREE}"
    }
}
{
    "event": {
        "origin": "raspi",
        "interface": "",
        "payload": "{\"distance\": 422, \"control\": FREE}"
    }
}
{
    "event": {
        "origin": "raspi",
        "interface": "",
        "payload": "{\"distance\": 422, \"control\": FREE}"
    }
}
```

desired properties

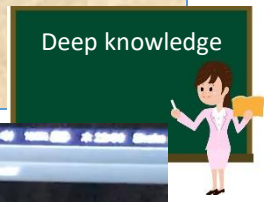


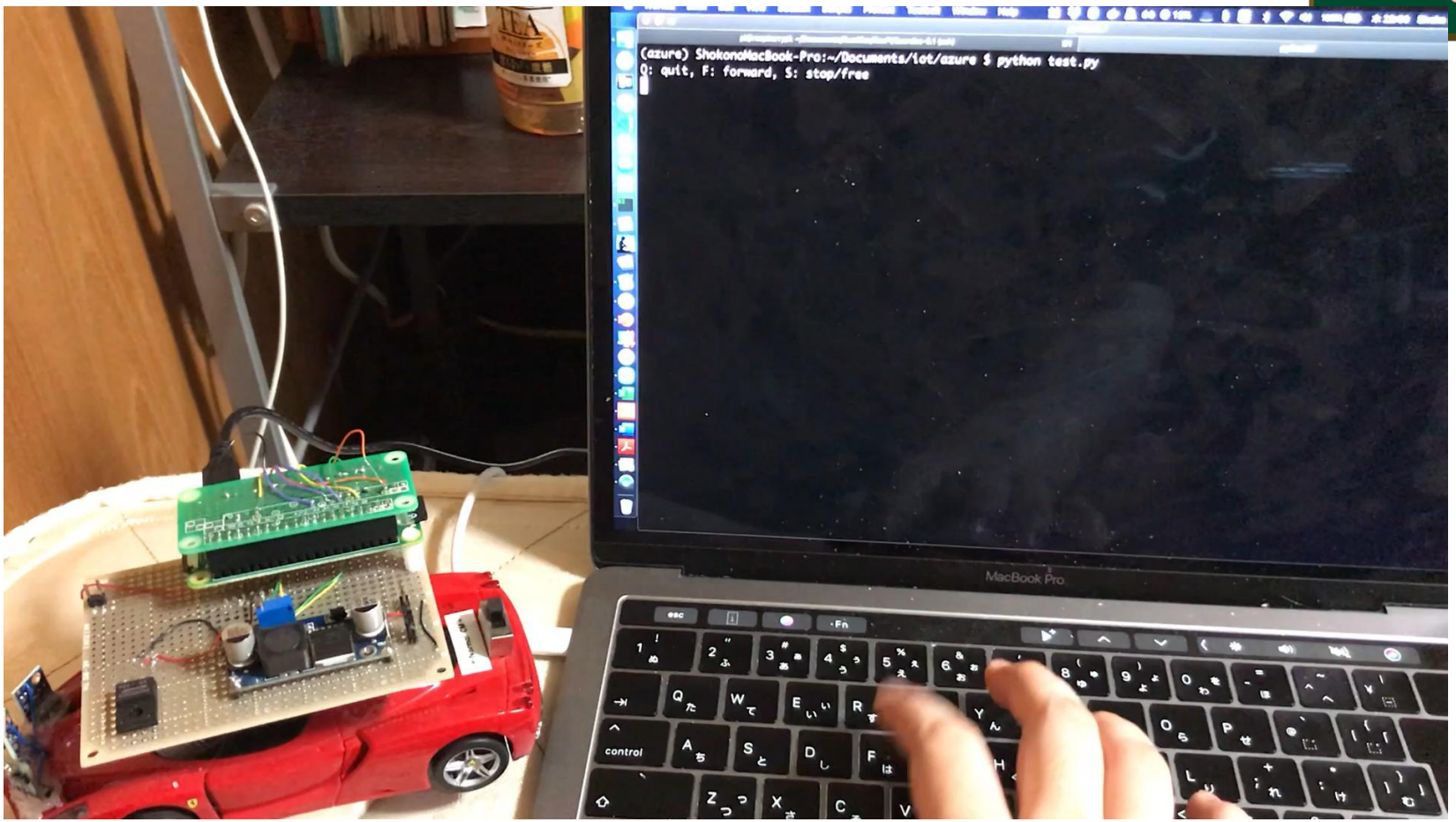
- クラウドからデバイスへの通知
- 実行開始や停止の制御命令など



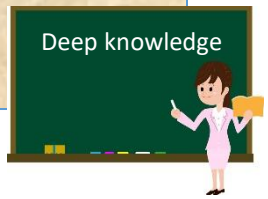
https://docs.microsoft.com/en-us/azure/iot-hub/tutorial-device-twins

動作動画（動画）


Deep knowledge


(azure) ShokonoMacBook-Pro:~/Documents/iot/azure $ python test.py
Q: quit, F: forward, S: stop/free
MacBook Pro
control

参考情報



- Azure IoT Hub のチュートリアル
  - https://docs.microsoft.com/ja-jp/azure/iot-hub/quickstart-send-telemetry-cli
- ソースコード（ラズパイ）
  - https://github.com/RumiCar-group/RumiCar/tree/master/RasPi/misc
  - クラウドへの送信：azure_d2c.py
  - Desired property：azure_property.py（ラズパイ側）、azure_service.py（PCなど制御が側）

※Azure IoT Hubの作成が必要です。

付録

I2Cアドレス

- RumicarではI2C通信を利用します。現在使用中のアドレスです

センサ			アドレス	設定方法
VL53L0X (レーザ測距モジュール)	Sensor0	左	0x14	ソフトウエアにより変更
	Sensor1	中央	0x15	
	Sensor2	右	0x16	
	Sensor3	後部	0x17	CMの後部に設置（任意）。起動時に0x29から変更
BMX055 (９軸センサ)	加速度センサ		0x19	基板上にてジャンパ設定
	ジャイロセンサ		0x69	
	磁気センサ		0x13	

分野別開発協力者

氏名	内容
片岡　秀明	• RumiCar車体作製 • CM開発
大栄　豊	• センサー基板設計開発 • Arduino Nano版CM基板設計開発
石垣　翔子	• ラズパイ版CM開発 • Pythonプログラム開発 • Microsoft Azure IoT Hub連携
稲玉　繁樹	• モータ低速回転制御 • ESP32版基板設計開発 • M5StickC版基板設計開発 • センサー基板設計開発
高瀬　英希	• ライブラリ化
千葉　吉輝	• Arduino公式ライブラリ登録・保守

イベント履歴

	日付	イベント	開催場所	備考
1	2017年8月5-6日	MakerFaireTokyo2017	東京ビックサイト	
2	2018年5月18-20日	MakerFaireBayArea2018	San Mateo, USA	
3	2018年8月4-5日	MakerfaireTokyo2018	東京ビックサイト	
4	2019年3月15日	Hello Tomorrow Global Challenge	Paris, France	投資家イベント
5	2019年8月3-4日	MakerFiareTokyo2019	東京ビックサイト	
6	2019年11月30日	IoT ALGYANつくるよ２！	情報科学専門学校	
7	2020年4月18日	IoT ALGYAN RumiCarハンズオン中継	オンライン	
8	2020年4月25日	IoT ALGYAN RumiCarハンズオン中継	オンライン	
9	2020年6月6日	IoT ALGYAN RumiCarハンズオン	オンライン	実機貸出でのオンライン

イベント履歴

	日付	イベント	開催場所	備考
10	2020年6月20日	【リアル】マイRumiCar試走会！RumiCar開発部メンバーと一緒に、自慢の愛機を走らせよう！	川崎市麻生区	
11	2020年8月10日	RumiCar実機ハンズオン&走行会	横浜市開港記念会館	
12	2020年8月30日	RumiCar走行会	麻生文化センター	
13	2020年11月13日	～RumiCar を使って自動運転車の制御の基本を学ぼう～ 第一回　部品確認、ハードウェアの作成	Zoom	主催:一般社団法人 広島県情報産業協会 技術委員会 協賛:RumiCar開発部
14	2020年12月4日	～RumiCar を使って自動運転車の制御の基本を学ぼう～ 第二回　ハードウェア作成の続き、基本的なソフトウェアのテスト		

イベント履歴

	日付	イベント	開催場所	備考
15	2021年1月10日	第15回イベント:RumICarハンズオン＆走行会(300人用会場で開催します!)	川崎市麻生区	
16	2021年9月26日	RumiCar会議(conference)	オンライン	
17	2021年11月28日	RumiCar走行会(良いにゃの日)	麻生文化センター	
18	2021年12月12日	RumiCar会議(conference)12/12	オンライン	Youtubeライブ
19	2022年4月29日	メイカーズフリマ	技研ベース	
20	2022年8月7日	第17回イベント：RumiCar試走会	東京都品川区大崎第二区民集会所	イベント座長井上氏
21	2022年9月3-4日	Maker Faire Tokyo2022	東京ビッグサイト	
22	2022年10月23日から27日	第一回RumiCar画廊(個展)	隅田公園リバーサイドギャライー	

イベント履歴

	日付	イベント	開催場所	備考
23	2022年11月3日、5日、6日、12日、13日	第二回RumiCar画廊(個展)	隅田公園リバーサイドギャライー	
24	2022年12月31日	コミックマーケット101	東京ビックサイト	
25	2023年2月1-3日	テクニカルショウヨコハマ2023	パシフィコ横浜	R2ブースで展示
26	2023年3月25日	自動運転アルゴリズムプラットフォーム"ルミカー" 講演会＆ハンズオン	ソニーシティ大崎 BRIDGE TERMINAL	
27	2023年7月19-21日	Robot Innovation Week 2023	名古屋国際会議場	R2ブースで展示
28	2023年10月14-15日	Maker Faire Tokyo 2023	東京ビッグサイト	

イベント履歴

	日付	イベント	開催場所	備考
29	2023年10月26-27日	燕三条ものづくりメッセ2023	燕三条地場産業振興センター	RumiCar新潟が鯛車版を展示
30	2023年11月3-4日	HANAZONO EXPO 2023	花園中央公園（東大阪市）	R2ブースで展示
31	2023年11月26日	自動運転ミニカーバトル2023 小学生部門	スカイホール豊田	車体にRumiCar採用（トヨタ技術会）
32	2023年11月29日-12月2日	2023国際ロボット展	東京ビッグサイト	R2のエリアで展示
33	2023年12月9-10日	あいちロボフェス2023	愛知県国際展示場	トヨタ技術会がワークショップ開催

イベント履歴

	日付	イベント	開催場所	備考
34	2023年12月16日	BLACK TECH SATURDAYS	Detroit, USA	Detroit Autonomous Vehicle Group が展示
35	2024年1月21日	第2回トビタテ文化祭	文部科学省 東館	展示とマイクパフォーマンス
36	2024年2月3-4日	YOXO FESTIVAL 2024	横浜みなとみらい地区	展示・ワークショップ、ピッチ決勝進出

イベント履歴

	日付	イベント	開催場所	備考
37	2024年2月24日	【トヨタ技術会presents】自動運転ミニカーをつくろう！	SMBCパーク栄（名古屋市）	トヨタ技術会がワークショップ開催
38	2024年3月2-3日	Kariya Micro Maker Faire 2024	刈谷市産業振興センター	SPRESENSEのブースで展示
39	2024年5月20日	Driving the Future（AutoSens USA 2024）	Detroit, USA	小学生向けハンズオンで使用
40	2024年7月6-7日	テックシーカー2024 ボードコンピュータ・プログラミング教室	ATC（大阪市）	SPRESENSE版でワークショップ

イベント履歴

	日付	イベント	開催場所	備考
41	2024年7月	センサーを使った「自動運転」で車を動かしてみよう！（授業）	芝浦工業大学附属中学高等学校	SPRESENSE版で授業
42	2024年7月24日	同上 本番のレース	ソニー本社	タイムアタック
43	2024年9月21-22日	Maker Faire Tokyo 2024	東京ビッグサイト	
44	2024年9月23日	Robo RAVE神戸大会2024	神戸国際会館	
45	2024年10月6日	ゆるメカトロ展示会2	東京テクニカルカレッジ	RumiCar新潟が鯛車版を展示
46	2024年10月18-20日	Maker Faire Bay Area 2024	Vallejo, USA	Blue Ribbon受賞

イベント履歴

	日付	イベント	開催場所	備考
47	2024年10月26-27日	つくろか！3	大阪市立中央図書館	
48	2024年10月27日	自動運転ミニカーバトル2024 小学生部門	スカイホール豊田	車体にRumiCar採用（トヨタ技術会）
49	2024年12月14-15日	あいちロボフェス2024	愛知県国際展示場	トヨタ技術会がワークショップ開催
50	2024年度	巻東中学校「地域サークル活動」	新潟市立巻東中学校	
51	2025年2月8日	【トヨタ技術会presents】自動運転ミニカーをつくろう！	SMBCパーク栄（名古屋市）	トヨタ技術会がワークショップ開催

イベント履歴

	日付	イベント	開催場所	備考
52	2025年3月8日	ラズパイ13歳お誕生会（Raspberry JAM Tokyo 2025）	ソニーシティ大崎 BRIDGE TERMINAL	LT登壇
53	2025年3月16日	第8回WE Love とよたフェスタ	スカイホール豊田	自動運転ミニカークラブのブースで展示
54	2025年7月12日	小中学生向けプログラミング教室	FUN＋TECH LABO（相模原市）	SPRESENSE版でワークショップ
55	2025年7月27日	RumiCar Program an Autonomous Vehicle	Cambridge, UK	ワークショップ開催
56	2025年11月8日	とうかい環境フェスタ2025	東海村役場	
57	2026年3月28日	NT福島2026	會津稽古堂（会津若松市）	

イベント履歴

	日付	イベント	開催場所	備考
58	2026年4月22日	STEAM Fair 2026	Northville, USA	
59	2026年5月10日	Tech Festival	Cambridge, UK	
60	2026年8月11日	RumiCarプログラミング教室（午前・午後）、第18回RumiCar走行会	北とぴあ（東京都北区）	
61	2026年8月30日	RumiCar車体制作教室	bono相模大野サウスモール	
62	2026年9月23日	RumiCarプログラミング教室（午前・午後、英語クラスあり）、第19回RumiCar走行会	北とぴあ（東京都北区）	

メディア掲載

	メディア	日付	内容	ページ
1	I/O 第47巻7月号 通巻549号	2022年7月号	100円ロボット部!	118-119
2	I/O 第47巻8月号 通関550号	2022年8月号	RumiCar	10-13
3	「Raspberry Pi」から広がる電子工作の世界	2022年11月30日発売		

提携・協業等

	メディア	日付	内容
1	R2		
2			
3			

最後に

RumiCar関連情報

より良く楽しんでいただくために

- RumiCar開発部では多くの方々に楽しんでいただくために標準化が大切と考えています
- 標準化がなされていると車体やコンピュータモジュールのハードウエアの互換性が向上し、プログラムの再利用性が高まります
- これらにより、多くの人が変更の手間を掛けずに機能向上、機能追加、高機能化ができるようになります
- 多くの人により楽しんで頂くためにコンピュータモジュールのピンアサインやピン配置はRumiCar標準に準拠いただき開発をお願いします

RumiCar関連情報



- RumiCar開発部Webページ
  - https://www.rumicar.com/
- Facebook　RumiCarグループ
  - https://www.facebook.com/groups/rumicar
- RumiCar開発部　連絡先メールアドレス
  - info@RumiCar.com
- RumiCar YouTube
  - https://www.youtube.com/channel/UCVg3CBSVBcc_00FdC6q2wDg
- GitHub
  - https://github.com/RumiCar-group/RumiCar
  - https://github.com/RumiCar-group/RumiCar-lib

RumiCarハンズオン！にご参加頂きましてどうもありがとうございました

これで終了です